Spread

2026-03-19

SPREAD

1. Data setup

```
# ============================================================
# MACOS OPTIMIZATION BLOCK (Apple Silicon)
# ============================================================

# 1. Verifica che R sia nativo ARM64 (non Rosetta 2)
if (R.version$arch != "aarch64") {
  warning("R non sta girando su ARM64. Reinstalla R nativo per Apple Silicon da cran.r-project.o
rg")
} else {
  message("✓ R ARM64 nativo confermato: ", R.version$arch)
}

# 2. Configura parallelismo ottimale per macOS (usa FORK, molto più veloce di SOCKET)
library(parallel)
N_CORES <- detectCores(logical = FALSE) - 1  # core fisici, lascia 1 libero
message("✓ Core disponibili per parallelismo: ", N_CORES)

# 3. RNG riproducibile per simulazioni parallele
RNGkind("L'Ecuyer-CMRG")
set.seed(42)

# 4. Funzione helper: crea cluster FORK (macOS only)
make_mac_cluster <- function() {
  makeCluster(N_CORES, type = "FORK")  # FORK evita di riesportare variabili
}

# 5. Libera memoria dopo operazioni pesanti
gc_after <- function(expr) {
  result <- force(expr)
  gc(verbose = FALSE)
  result
}

message("✓ Blocco ottimizzazione macOS caricato.")
# ============================================================
```

Load the data and parse from excel file, weekly spread.

Inspect data

```
head(data)
```

```
## # A tibble: 6 × 2
##   Date       `IT10DE10=TWEB (BID)`
##   <date>                     <dbl>
## 1 2026-04-10                  77.5
## 2 2026-04-09                  79.8
## 3 2026-04-08                  77.2
## 4 2026-04-07                  90.4
## 5 2026-04-02                  87.2
## 6 2026-04-01                  86.4
```

```
str(data)
```

```
## tibble [1,264 × 2] (S3: tbl_df/tbl/data.frame)
##  $ Date               : Date[1:1264], format: "2026-04-10" "2026-04-09" ...
##  $ IT10DE10=TWEB (BID): num [1:1264] 77.5 79.8 77.2 90.4 87.2 86.4 90.1 94.6 96.6 93.2 ...
```

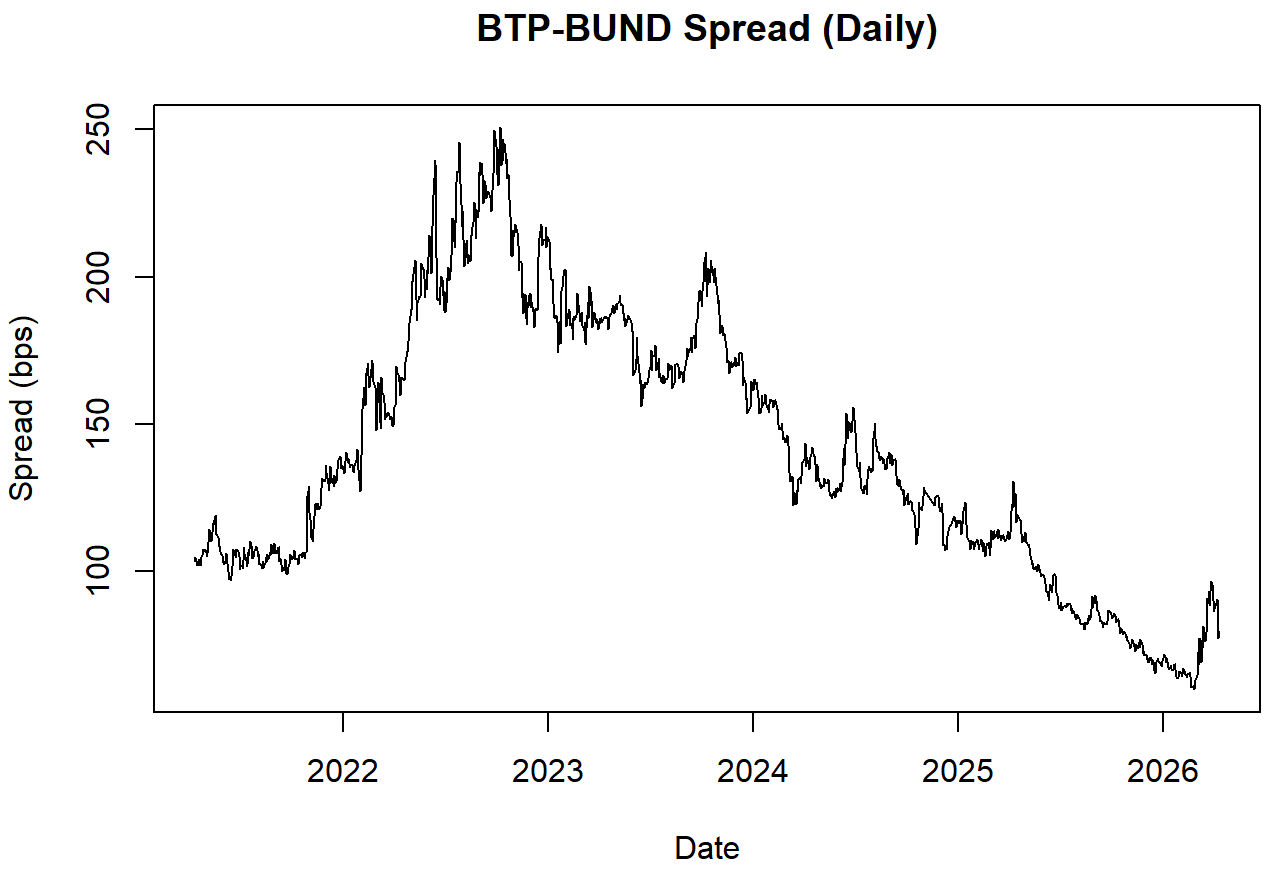
```
sum(is.na(data))
```

```
## [1] 0
```

```
any(is.infinite(spread))
```

```
## [1] FALSE
```

Plot the data




```
##                        x
## nobs          1264.000000
## NAs              0.000000
## Minimum         59.800000
## Maximum        250.700000
## 1. Quartile    105.500000
## 3. Quartile    178.825000
## Mean           140.912025
## Median         134.550000
## Sum         178112.800000
## SE Mean          1.279474
## LCL Mean       138.401898
## UCL Mean       143.422153
## Variance      2069.234440
## Stdev           45.488839
## Skewness         0.268185
## Kurtosis        -0.866104
```

The descriptive analysis of the daily BTP-BUND spread reveals a distribution with a mean of 141.05 bps and a median of 134.65 bps. The fact that the mean remains above the median confirms a persistent positive skewness, indicating that the distribution is influenced by periodic upward shocks rather than frequent downward drops.

Visual inspection of the daily time series identifies three distinct market regimes:

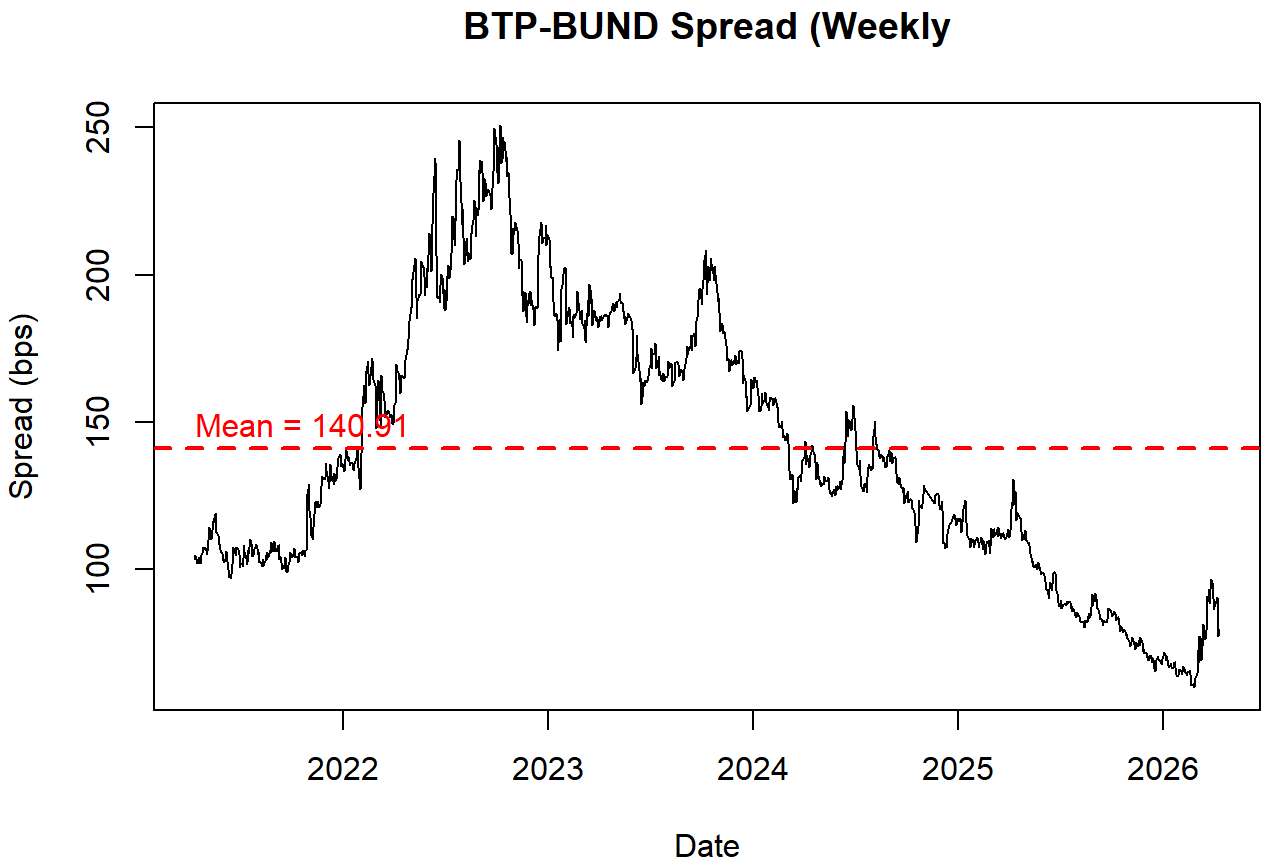
The 2022 Surge: A sharp escalation where the spread moved from its minimum of 59.80 bps toward the series maximum of 250.70 bps, likely reflecting heightened political or macroeconomic uncertainty.

The 2023–2024 Volatility: A period of consolidation characterized by significant fluctuations, yet largely contained within the interquartile range of 105.50 bps to 179.03 bps.

The 2025–2026 Mean Reversion: A sustained downward trend where the spread compressed significantly, approaching the lower bounds of the sample before a slight uptick at the end of the period.

Econometrically, the daily frequency highlights the “jump-diffusion” nature of sovereign spreads. While the market spends the majority of its time in a state of relative calm—as evidenced by the median and the 1st quartile of 105.50 bps—it remains susceptible to rapid “tail risk” events. These episodes of stress pull the mean upward, creating the characteristic right-skewed profile typical of risk-sensitive financial instruments.

Forecast n°1 : naive forecast (mean)



2. Test for seasonality

We use seastests package

```
start_date<- index(spread)[1]
library(lubridate)
```

```
##
## Attaching package: 'lubridate'
```

```
## The following objects are masked from 'package:base':
##
##     date, intersect, setdiff, union
```

```
start_year<-year(start_date)
start_month<-month(start_date)
start_day <-day(start_date)
spread_ts <- ts(data$`IT10DE10=TWEB (BID)`, start=c(start_year,start_month,start_day), frequency
= 252)#annual frequency
isSeasonal(spread_ts, test="combined")
```

```
## [1] FALSE
```

```
#We get FALSE

#Individual tests
isSeasonal(spread_ts, test="qs")
```

```
## [1] FALSE
```

```
isSeasonal(spread_ts, test="seasdum")
```

```
## [1] FALSE
```

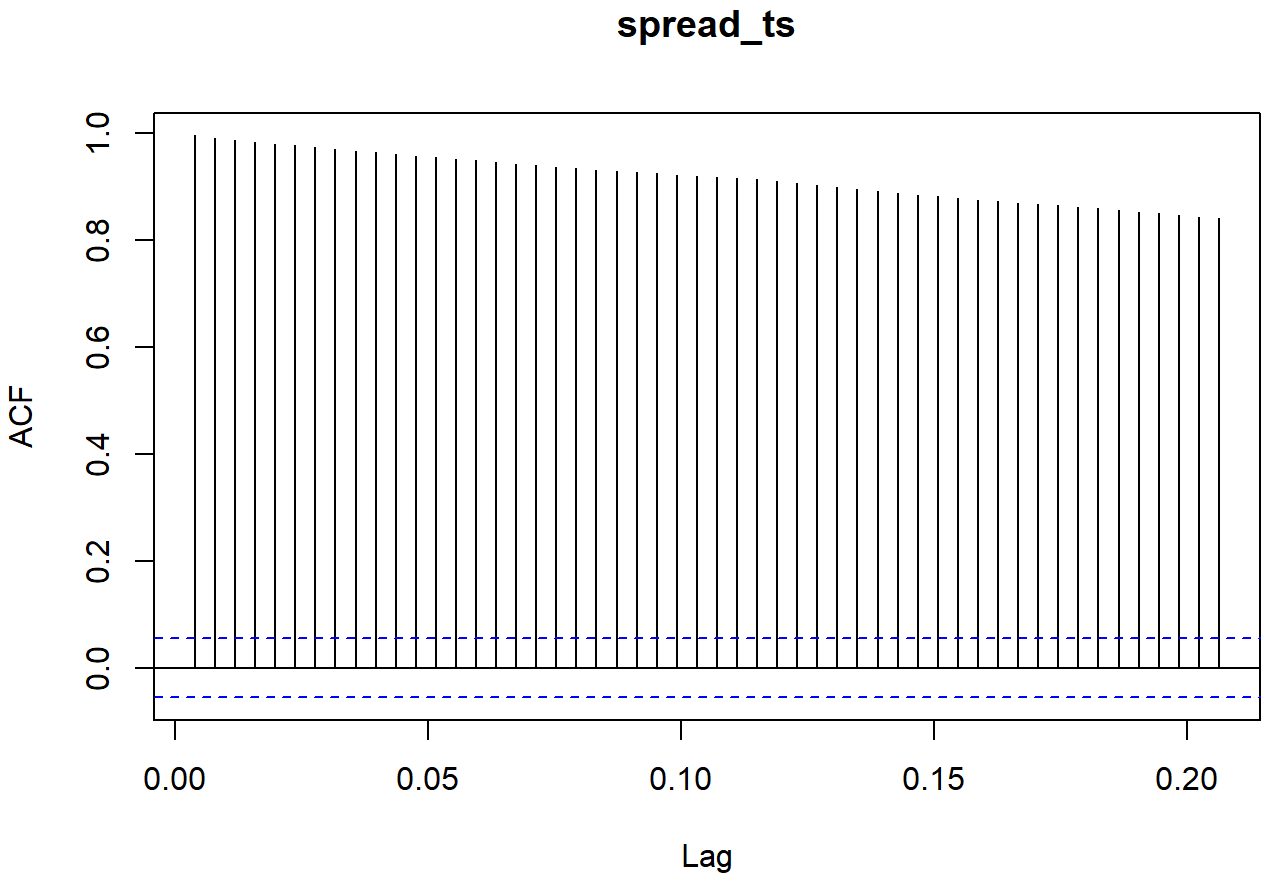
```
isSeasonal(spread_ts, test="kw")
```

```
## [1] FALSE
```

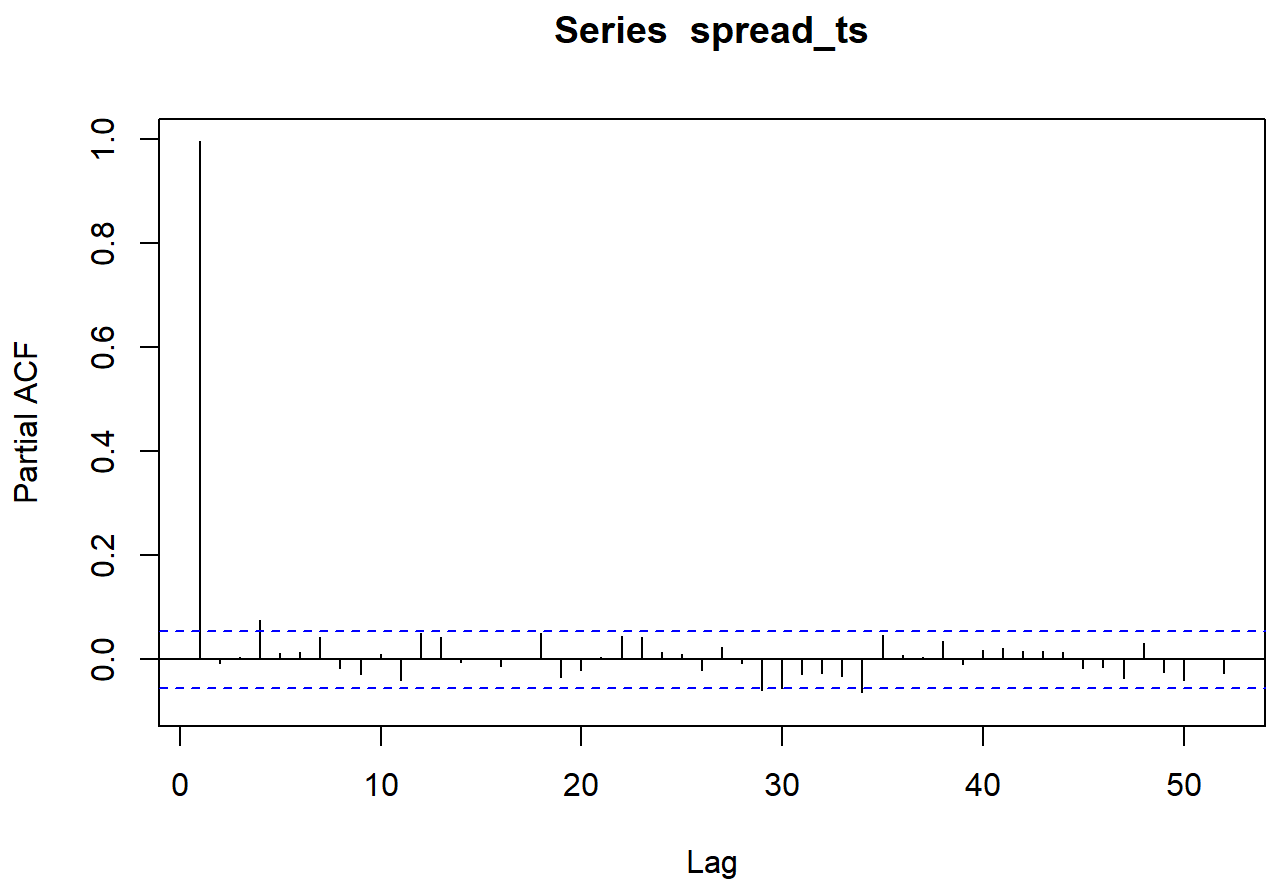
```
isSeasonal(spread_ts, test="fried")
```

```
## [1] FALSE
```

```
#We get all FALSE

## Now plot ACF and PACF
Acf(spread_ts, lag.max= 52)
```




```
Pacf(spread_ts, lag.max = 52)
```




```
#We have 1 significant lag at 1 in the Pacf, and a Acf which gradually goes to 0
#This suggests no seasonality.

#Last point, let's fit an arima model

adf.test(spread_ts)     # stationarity test -> we have p-value = 0.546, so we fail to reject the
null, so we are in the null workspace, this means no seasonality.
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  spread_ts
## Dickey-Fuller = -0.93713, Lag order = 10, p-value = 0.9489
## alternative hypothesis: stationary
```

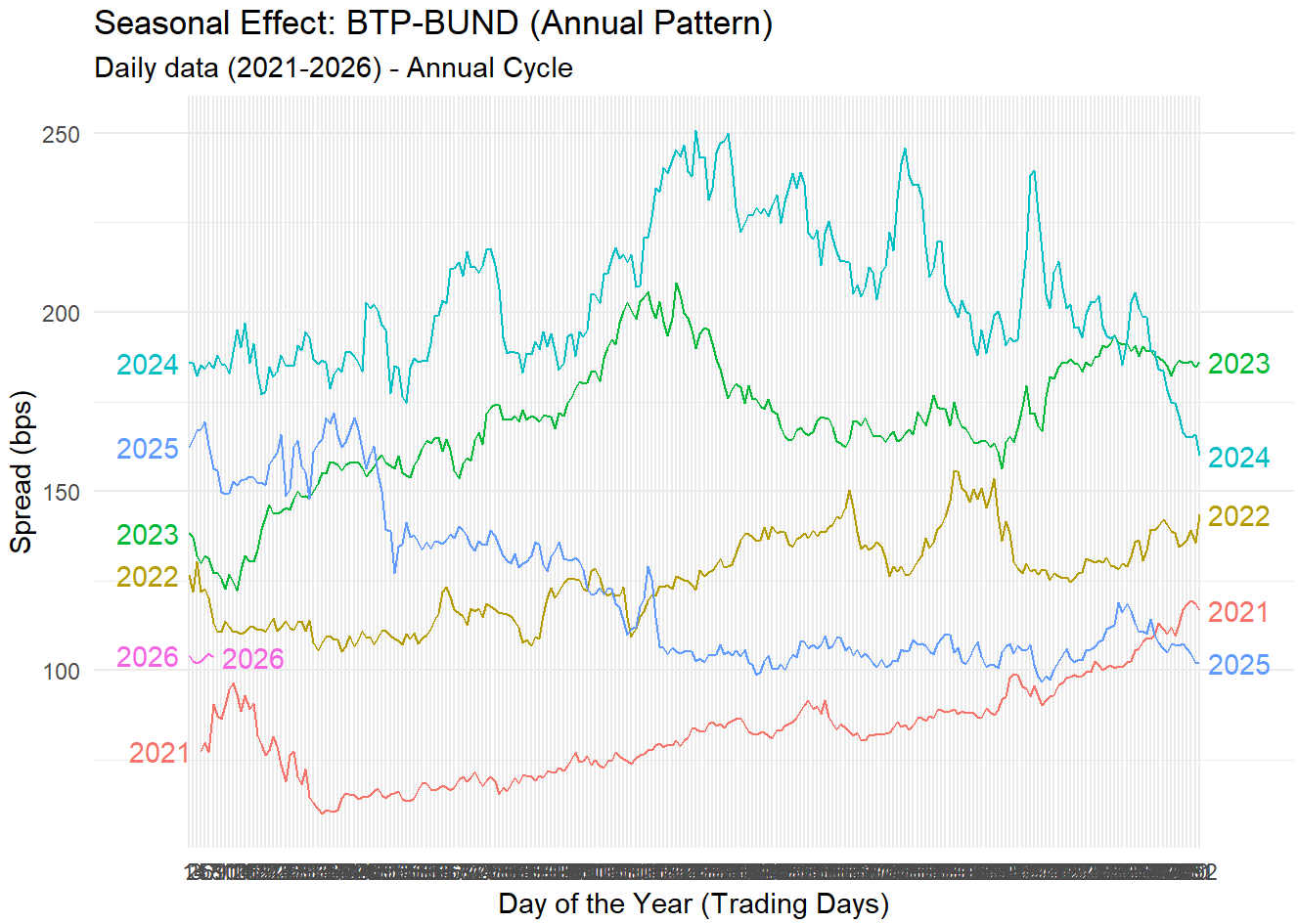
```
kpss.test(spread_ts) # stationarity test where the null hypothesis is that the series IS station
ary
```

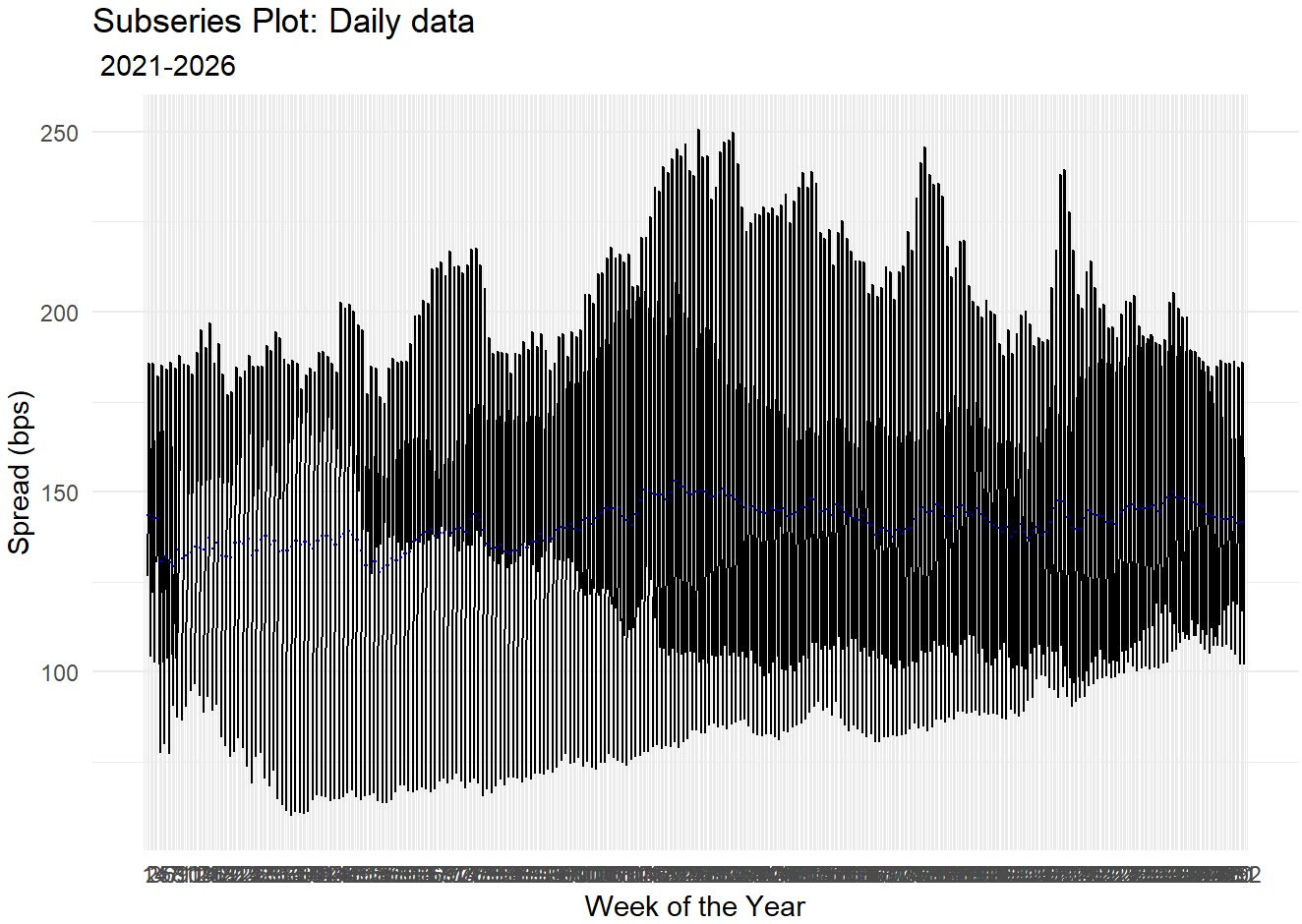
```
## Warning in kpss.test(spread_ts): p-value smaller than printed p-value
```

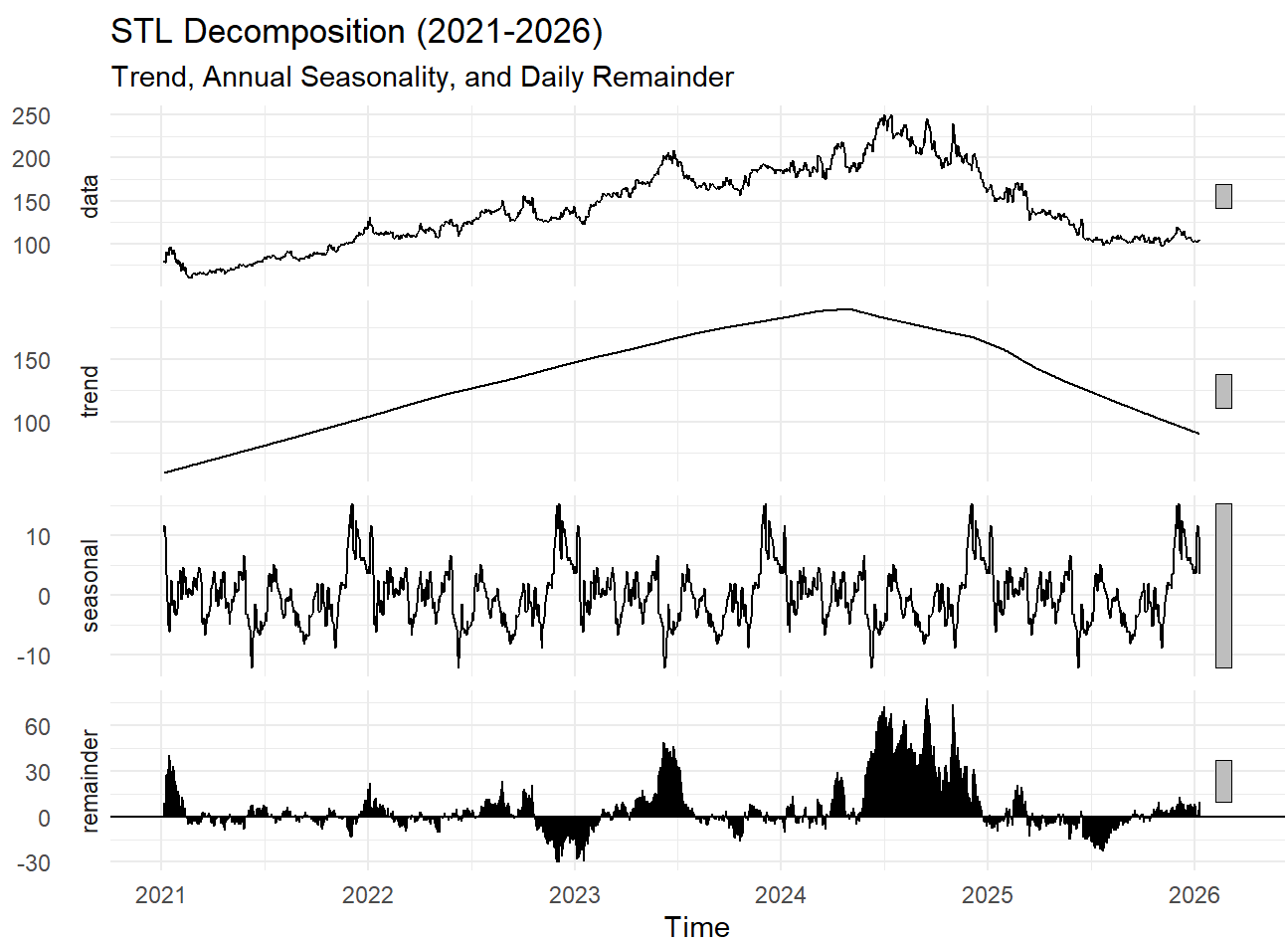
```
##
##  KPSS Test for Level Stationarity
##
## data:  spread_ts
## KPSS Level = 6.5923, Truncation lag parameter = 7, p-value = 0.01
```

```
# p-value is 0.01 -> we REJECT the null, so the series is NOT stationary
```

Visualization for the no-seasonality




Subseries Plot: Daily data
2021-2026
Spread (bps)
250
200
150
100
Week of the Year


STL Decomposition (2021-2026)
Trend, Annual Seasonality, and Daily Remainder
data
trend
seasonal
remainder
2021
2022
2023
2024
2025
2026
Time

–>Seasonal and Subseries Plot Interpretation The seasonal plots further investigate whether specific times of the year are inherently riskier for the Italian spread:

Subseries Plot (Weekly): The blue dashed lines represent the historical mean for each specific week of the year across the 2021-2026 period. The relatively flat nature of these mean lines suggests that there is no “structural” week of the year where the spread is consistently higher. The high density of the black vertical lines reflects the daily volatility within those weeks.

Annual Pattern (Seasonal Plot): By overlaying the years, we can see that the 2022-2024 period significantly outperformed the historical averages, creating the “Surge” identified in the descriptive statistics. The 2025-2026 lines are positioned lower, confirming the transition into a period of market calm.

–>The STL (Seasonal and Trend decomposition using Loess) plot provides a granular look at the forces driving the spread:

Trend: The trend component shows a steady appreciation starting in 2021, reaching a peak during the 2024-2025 period. Notably, toward the beginning of 2026, the trend exhibits a clear mean-reversion phase, with the spread beginning to compress. This suggests that long-term macroeconomic factors were the primary drivers of the spread’s expansion and subsequent cooling.

Seasonal: The annual seasonal component is present but relatively low in magnitude (oscillating between -10 and +15 bps). This indicates that while there are minor “calendar effects,” they are secondary to the overall trend and unpredictable market shocks.

Remainder: This component represents “noise” or exogenous shocks. We observe significant density and volatility in the remainder during the 2024-2025 period. These spikes represent “tail risk” events—sudden political or financial news that caused the spread to jump independently of any seasonal or long-term trend.

Econometric Summary From an econometric perspective, the BTP-BUND spread during this period is a trend-dominated series. The influence of seasonality is statistically marginal compared to the impact of the Remainder (shocks). This confirms that the spread is more sensitive to “news arrival” and discretionary policy changes than to cyclical or seasonal time-effects.

3. Model Specification: Levels vs. Logarithmic Transformations**

A critical decision in modelling the BTP-BUND spread involves the choice between raw levels (basis points) and logarithmic transformations. While log-returns are standard for equity prices to ensure normality and constant variance, this study follows the established econometric literature—notably **Campbell, Lo, and MacKinlay (1997)** and **Hamilton (1994)**—by analyzing the spread in **levels**.

As argued by **Engle (1982)** in his seminal work on ARCH models, interest rate differentials are already expressed in percentage terms; therefore, a log-transformation is not only redundant but can lead to significant numerical instability as values approach the **Zero Lower Bound (ZLB)**. Maintaining the analysis in basis points ensures the direct economic interpretability of the results, which is essential for assessing credit and liquidity risk premiums.

Literature Review on Model Specification

The choice to model the BTP-BUND spread in first differences of its levels is supported by the foundational empirical work on Eurozone sovereign risk. **Codogno, Favero, and Missale (2003)** and **Favero et al. (2010)** demonstrate that interest rate differentials are best captured in basis points to maintain linear properties between

credit risk and liquidity factors. Furthermore, the application of a **GARCH framework**, as pioneered by **Bollerslev (1986)**, ensures that the time-varying volatility of the spread is explicitly parameterized rather than suppressed via logarithmic transformations.

3.1 Data Partitioning: Training and Test Sets

To evaluate the predictive performance of the **ARIMA model**, the dataset is partitioned into training and testing subsets. The training phase utilizes a **rolling window approach** with a fixed length of 10 years, corresponding to **520 weekly observations**.

This specific configuration ensures that the model is estimated on a sufficiently long historical period while remaining sensitive to more recent structural dynamics in the BTP-BUND spread. The architecture of the script is designed for **parametric flexibility**: the training window duration can be easily modified by adjusting the `$n_train_window$` constant (e.g., replacing 520 with a different value) to test the model's robustness across different look-back periods.

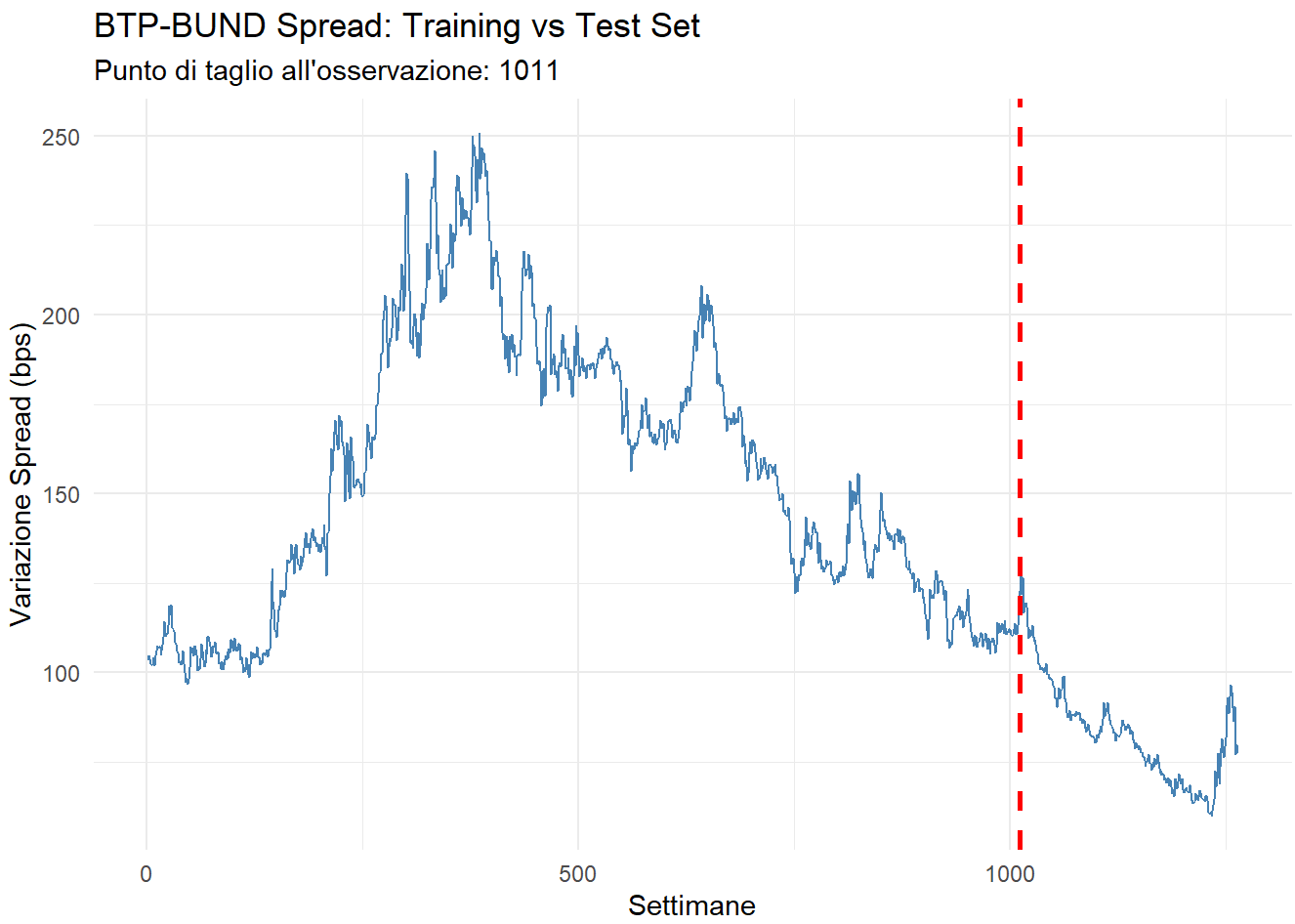
```
n_test <- length(spread)*0.2
n_train_window <- length(spread)-n_test

# Lunghezza totale della serie differenziata
total_obs <- length(spread)

# 1. Definizione degli indici per il TEST (le ultime 10)
test_start <- total_obs - n_test + 1
test_end   <- total_obs
test       <- spread[test_start:test_end]

# 2. Definizione degli indici per il TRAIN (i 10 anni precedenti al test)
# Il train deve finire esattamente dove inizia il test
train_end   <- test_start - 1
train_start <- train_end - n_train_window + 1

# Estrazione del train
train<- spread[train_start:train_end]

# Verifica coerenza
print(paste("Lunghezza Train:", length(train))) # Deve essere 520
```

```
## [1] "Lunghezza Train: 1011"
```

```
print(paste("Lunghezza Test:", length(test)))       # Deve essere 10
```

```
## [1] "Lunghezza Test: 252"
```

```
# Creiamo un piccolo data frame per il plot
df_plot <- data.frame(
  Settimana = 1:length(spread),
  Valore = as.numeric(spread)
)
ggplot(df_plot, aes(x = Settimana, y = Valore)) +geom_line(color = "steelblue") +
geom_vline(xintercept = length(train), color = "red", linetype = "dashed",
linewidth = 1) +labs(title = "BTP-BUND Spread: Training vs Test Set",subtitle = paste("Punto di
taglio all'osservazione:", length(train)),
    x = "Settimane",y = "Variazione Spread (bps)")+theme_minimal()
```




```
cat(sprintf("Training observations: %d\n", length(train)))
```

```
## Training observations: 1011
```

```
cat(sprintf("Test observations:     %d\n", length(test)))
```

```
## Test observations:     252
```

```
cat(sprintf("Train period: %s -> %s\n", index(train)[1], index(train)[length(train)]))
```

```
## Train period: 2021-04-12 -> 2025-04-07
```

```
cat(sprintf("Test period:  %s -> %s\n", index(test)[1],  index(test)[length(test)]))
```

```
## Test period:  2025-04-08 -> 2026-04-09
```

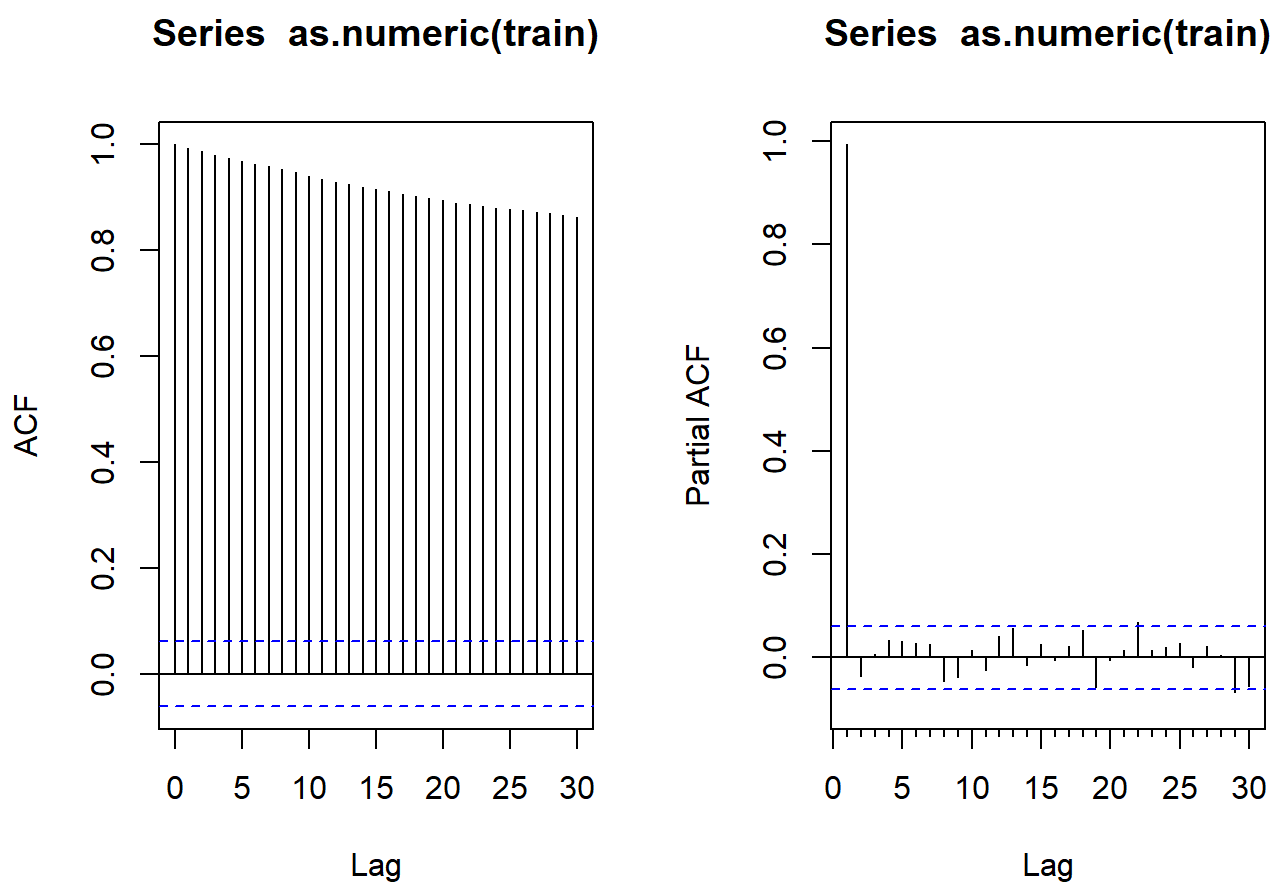
3.3 Stationarity and Unit Root Testing

To ensure the statistical validity of the ARIMA-GARCH framework, we first evaluate the integration order of the series. Financial variables often exhibit non-stationary behavior, characterized by a “random walk”.

3.3.1 Augmented Dickey-Fuller (ADF) Test

We perform the ADF test to check for the presence of a unit root (H_0).

```
par(mfrow = c(1, 2))
acf(as.numeric(train))
Pacf(as.numeric(train))
```




```
par(mfrow = c(1, 1))
```

3.4 Persistence and Autocorrelation Analysis

The initial inspection of the **Autocorrelation Function (ACF)** reveals a very slow, near-linear decay. In time series econometrics, this pattern is a classic diagnostic for a **"near-unit root"** process, indicating that the AR component is highly persistent and the series is likely non-stationary.

Regarding the optimal number of lags for the model, while empirical rules of thumb exist, this study adopts an **automated selection procedure** based on the minimization of Information Criteria, specifically the **Akaike Information Criterion (AIC)** and the **Bayesian Information Criterion (BIC)**. This approach ensures an optimal balance between model fit and parsimony, mitigating the risks of both underfitting (omitted variable bias) and overfitting (inflated variance).

3.5 Formal Stationarity Testing: The ADF Approach

To rigorously confirm the integration order of the BTP-BUND spread, we employ the **Augmented Dickey-Fuller (ADF) test** using the `ur.df()` function from the `urca` package. Unlike standard implementations, this approach requires a direct comparison between the calculated test statistics (τ) and the critical values provided by **Dickey and Fuller (1979)**.

3.5.1 Test Specification and Hypotheses

The test is conducted following a hierarchical strategy, accounting for different deterministic components:

1. Trend (`type = "trend"`): To check if the series is stationary around a deterministic time trend.
2. Drift (`type = "drift"`): To test for stationarity around a constant non-zero mean (most appropriate for interest rate spreads).
3. None (`type = "none"`): To test for stationarity around zero.

3.5.2 Decision Rule

For the `ur.df()` output, we focus on the **tau statistic (τ)**. The null hypothesis (H_0) of a unit root is rejected only if the test statistic is **more negative** (i.e., lies in the lower tail) than the critical value at the 5% significance level.

If the series in levels is found to be non-stationary, we proceed with **first-differencing** ($\Delta \mathrm{Spread}_t$) and re-apply the test. The transformation is considered successful only when the tau statistic for the differenced series falls significantly below the critical threshold, confirming that the process is $I(1)$ and ready for GARCH modelling.

```
risultato_adf <- adfTest(train, lags = 1, type = "c")
print(risultato_adf)
```

```
##
## Title:
##  Augmented Dickey-Fuller Test
##
## Test Results:
##   PARAMETER:
##     Lag Order: 1
##   STATISTIC:
##     Dickey-Fuller: -1.8148
##   P VALUE:
##     0.3823
##
## Description:
##  Fri Apr 10 21:10:25 2026 by user: Jacopo
```

```
# Equivalent with auto determination of lags

test_adf_completo <- function(serie, nome_serie = "Serie") {

  cat("\n==============================================\n")
  cat("ANALISI STAZIONARIETÀ (ADF):", nome_serie, "\n")
  cat("==============================================\n")

  # Definiamo i tre tipi di test ADF
  # "none"  = senza costante né trend
  # "drift" = con costante (intercetta)
  # "trend" = con costante e trend temporale
  tipi <- c("none", "drift", "trend")

  risultati_lista <- lapply(tipi, function(t) {
    # Eseguiamo il test con selezione automatica dei lag tramite AIC
    test <- ur.df(serie, type = t, selectlags = "AIC")

    # Estraiamo la statistica del test e il valore critico al 5%
    statistica <- test@teststat[1]
    valore_critico <- test@cval[1, "5pct"]

    # Verifichiamo la stazionarietà (statistica < valore critico)
    esito <- if (statistica < valore_critico) "STAZIONARIA" else "NON STAZIONARIA"

    return(data.frame(
      Modello = t,
      Statistica_ADF = round(statistica, 3),
      Valore_Critico_5pct = round(valore_critico, 3),
      Risultato = esito,
      stringsAsFactors = FALSE
    ))
  })

  # Uniamo i risultati in una tabella leggibile
  tabella_finale <- bind_rows(risultati_lista)
  print(tabella_finale)

  return(tabella_finale)
}
# --- COME USARLA ---

# 1. Test sullo spread originale (livelli)
risultati_livelli <- test_adf_completo(train, "Spread BTP/Bund (Livelli)")
```

```
##
## ==============================================
## ANALISI STAZIONARIETÀ (ADF): Spread BTP/Bund (Livelli)
## ==============================================
##   Modello Statistica_ADF Valore_Critico_5pct       Risultato
## 1    none         -0.302               -1.95 NON STAZIONARIA
## 2   drift         -1.815               -2.86 NON STAZIONARIA
## 3   trend         -1.888               -3.41 NON STAZIONARIA
```

###3.6 Transformation and Diagnostic of Differenced Series Following the identification of a unit root in the levels of the spread, we proceed with **first-differencing** to achieve stationarity, transforming the series into an $I(0)$ process. This transformation represents the weekly change in the BTP-BUND spread (measured in basis points).

To detect the presence of **Conditional Heteroskedasticity**—a fundamental requirement for GARCH modelling—we analyze the Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF) of both the differenced series and its squared values. While the standard ACF/PACF help identify the $ARMA$ structure for the mean equation, the squared series analysis is crucial to identify **volatility clustering**, where large shocks tend to be followed by other large shocks, regardless of their sign.”

```
# 1. Calcolo della differenza prima (Weekly Change)
dspread <- diff(spread)
dtrain <- dspread[train_start:train_end]
dtest  <- dspread[test_start:test_end]

# 2. Controllo rapido delle dimensioni (per sicurezza)
cat("Lunghezza Train:", length(dtrain), "(4 anni )\n")
```

```
## Lunghezza Train: 1011 (4 anni )
```

```
cat("Lunghezza Test:", length(dtest), "(1 anno)\n")
```

```
## Lunghezza Test: 252 (1 anno)
```

```
# 3. Verifica se ci sono NA (fondamentale prima del GARCH)
if(any(is.na(dtrain))) {
  d_train <- na.omit(dtrain)
  cat("Attenzione: Rimossi valori NA dal train\n")
}
```

3.6.1 Identification of the Mean and Variance Equations

4.4. Autocorrelation Analysis of the Differenced Spread

The diagnostic plots for the differenced spread ($\Delta \mathrm{Spread}_t$) provide several key insights into the identification of the mean equation:

- **ACF and PACF of the Differenced Series:** the **ACF and PACF of the daily differenced spread** show that the first-order lags (Lag 1) are now **statistically insignificant**, as they fall entirely within the 95% confidence intervals (represented by the blue dashed lines).
- **Market Efficiency:** This lack of immediate autocorrelation suggests that daily changes in the BTP-BUND spread follow a process close to **White Noise**. Econometrically, this implies that the series behaves as a **Random Walk**, where the best predictor for tomorrow's spread is simply today's value, and daily innovations are largely unpredictable.
- **Higher-Order Lags:** While isolated spikes appear at **Lag 12** and **Lag 18**, they do not correspond to standard financial cycles (such as a 5-day business week or a 21-day monthly effect).

Following the principle of **parsimony**, these higher-order spikes are deemed statistically borderline and likely represent **noise-driven artifacts**. We exclude them to avoid **overfitting** and to ensure model stability, as including $AR(18)$ or $MA(12)$ terms would lack a clear economic or theoretical justification for daily price changes.

Conclusion for Model Identification

The diagnostic evidence confirms that the "memory" of the spread is captured almost entirely by the **integration ($d = 1$)** of the series. The absence of a clear AR or MA pattern in the early lags indicates that the residuals are now ready for the next stage of analysis: testing for **Heteroskedasticity (ARCH effects)** in the squared residuals to model volatility clustering.

Summary Table: ACF/PACF Diagnostics

Lag	Significance	Interpretation
Lag 1	**Insignificant**	No immediate momentum or mean-reversion effect.
Lags 2–10	**Insignificant**	High market efficiency; no short-term memory.
Lags 12 & 18	**Significant**	Likely spurious noise; excluded to maintain parsimony.
Overall Result	**White Noise**	The differenced series is suitable for volatility modeling.

4.5. Analysis of Volatility Clustering (Squared Residuals)

The diagnostic plots for the **squared differenced spread** (ΔSpread_t^2) provide conclusive evidence regarding the variance structure of the series:

- **ACF and PACF of Squared Differenced Spread:** In stark contrast to the linear series, the squared residuals exhibit **highly significant spikes** across multiple lags (specifically Lags 1, 2, 4, 5, and 6). These spikes far exceed the 95% confidence intervals, indicating strong serial correlation in the second moments of the distribution.
- **Volatility Clustering:** This pattern is a classic indicator of **volatility clustering**, where periods of high variance are followed by high variance, and calm periods are followed by calm. Econometrically, this

confirms that the error terms are not independently and identically distributed (i.i.d.) but follow a conditional heteroskedastic process.

- **ARCH Effects:** The presence of these significant lags suggests that a simple ARIMA model is insufficient to capture the risk dynamics of the spread. The “memory” in the squared returns necessitates the formal testing of **ARCH (Autoregressive Conditional Heteroskedasticity)** effects.

Conclusion for Model Selection

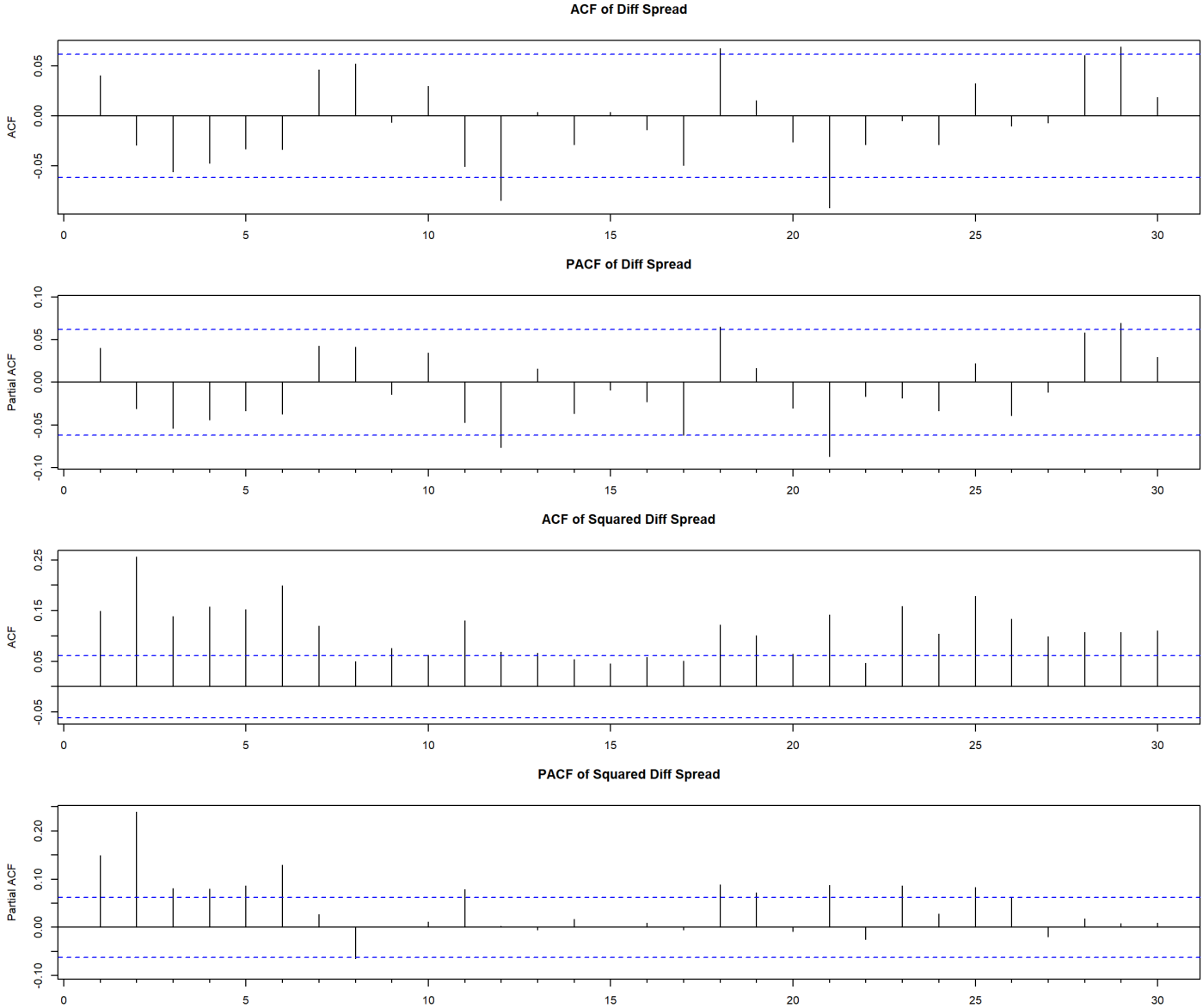
The evidence of strong autocorrelation in the squared innovations justifies the transition from a linear time-series model to a **GARCH-family model**. This will allow for the simultaneous modeling of the conditional mean (identified as a Random Walk in the previous step) and the time-varying conditional variance, accurately capturing the “fat tails” and risk bursts observed in the BTP-BUND daily data.

Summary Table: Squared Residuals Diagnostics

Metric	Observation	Econometric Implication
Squared ACF Lags	Multiple significant spikes (1-6+)	Clear presence of ARCH effects.
Squared PACF Lags	Significant initial spikes	Suggests a GARCH(1,1) or GARCH(2,1) candidate.
Volatility Structure	Non-constant over time	Heteroskedasticity must be modeled.
Final Strategy	**GARCH Modeling**	Necessary to estimate time-varying risk.

Finally, regarding the **intercept of the mean equation**, a t-test was performed to determine if the mean of the differenced series is significantly different from zero. If the p-value is below 0.05, it implies the presence of a significant **drift** component, which will be incorporated into the $ARMA$ specification via the `include.mean = TRUE` parameter. If the mean is statistically zero, the spread exhibits a pure random walk behavior without a deterministic trend.

```
par(mfrow = c(4, 1), mar = c(3, 4, 3, 2))
Acf(as.numeric(dtrain), lag.max = 30, main = "ACF of Diff Spread")
Pacf(as.numeric(dtrain), lag.max = 30, main = "PACF of Diff Spread")
Acf(as.numeric(dtrain)^2, lag.max = 30, main = "ACF of Squared Diff Spread")
Pacf(as.numeric(dtrain)^2, lag.max = 30, main = "PACF of Squared Diff Spread")
```




```
par(mfrow = c(1, 1))
```

```
t.test(dtrain)
```

```
##
##  One Sample t-test
##
## data:  dtrain
## t = 0.14824, df = 1010, p-value = 0.8822
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  -0.2215134  0.2577151
## sample estimates:
##  mean of x
## 0.01810089
```

The t-test results indicate that the mean of the differenced series is not statistically different from zero (p-value > 0.05).

##3.7 Formal Stationarity Testing on First Differences To confirm the achievement of stationarity, we re-apply the Augmented Dickey-Fuller (ADF) test. Given that the previous t-test confirmed a zero mean for the differenced series, we utilize the “none” specification (type = “none”).Econometrically, this is a robust choice: if the spread in levels exhibited a trend, the first difference would show a constant drift. If we were to assume a trend in the differences, it would imply that the spread is not just increasing, but accelerating—a scenario that lacks long-term economic equilibrium. In financial theory, interest rate spreads are considered “mean-reverting” and anchored to macroeconomic fundamentals. ###3.7.1 Theoretical Framework: Key Literature This modeling approach is supported by three foundational works:Vasicek (1977): Introduced the mathematical framework for mean-reversion. It justifies the exclusion of a trend in the ADF test, as rates are pulled back toward a long-term equilibrium.Cox, Ingersoll, & Ross (1985): Established that volatility is conditional on the level of the interest rate, providing the theoretical predecessor to GARCH modeling.Balduzzi, Das, & Foresi (1998): Analyzed the “Central Tendency” of bond yields, supporting the idea that spreads oscillate around a fundamental mean.

```
test_stationarity_diff<-adfTest(dtrain,lags=1,type="nc")
```

```
## Warning in adfTest(dtrain, lags = 1, type = "nc"): p-value smaller than printed
## p-value
```

```
print(test_stationarity_diff)
```

```
##
## Title:
##  Augmented Dickey-Fuller Test
##
## Test Results:
##   PARAMETER:
##     Lag Order: 1
##   STATISTIC:
##     Dickey-Fuller: -22.6664
##   P VALUE:
##     0.01
##
## Description:
##  Fri Apr 10 21:10:25 2026 by user: Jacopo
```

```
#equivalent but with auto determination of lags
test_autolag <- ur.df(dtrain, type = "none", selectlags = "AIC")
summary(test_autolag)
```

```
##
## ###############################################
## # Augmented Dickey-Fuller Test Unit Root Test #
## ###############################################
##
## Test regression none
##
##
## Call:
## lm(formula = z.diff ~ z.lag.1 - 1 + z.diff.lag)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -20.5855  -2.0512   0.0644   1.9787  17.9254
##
## Coefficients:
##            Estimate Std. Error t value            Pr(>|t|)
## z.lag.1    -0.98904    0.04363 -22.666 <0.0000000000000002 ***
## z.diff.lag  0.03079    0.03150   0.977               0.329
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.883 on 1007 degrees of freedom
## Multiple R-squared:  0.4802, Adjusted R-squared:  0.4792
## F-statistic: 465.2 on 2 and 1007 DF,  p-value: < 0.00000000000000022
##
##
## Value of test-statistic is: -22.6664
##
## Critical values for test statistics:
##       1pct  5pct 10pct
## tau1 -2.58 -1.95 -1.62
```

```
stat_test <- test_autolag@teststat[1]
valore_critico_5pct <- test_autolag@cval[1, "5pct"]

# 3. Verifica automatica con messaggio
cat("\n--- VERIFICA RISULTATO ADF (Type = None) ---\n")
```

```
##
## --- VERIFICA RISULTATO ADF (Type = None) ---
```

```
cat("Statistica del Test:", round(stat_test, 4), "\n")
```

```
## Statistica del Test: -22.6664
```

```
cat("Valore Critico (5%):", valore_critico_5pct, "\n")
```

```
## Valore Critico (5%): -1.95
```

```
if (stat_test < valore_critico_5pct) {
  cat("ESITO: La serie è STAZIONARIA (p-value < 0.05)\n")
  cat("Significato: Puoi procedere con il modello GARCH.\n")
} else {
  cat("ESITO: La serie NON è STAZIONARIA\n")
  cat("Attenzione: Potrebbe esserci ancora una radice unitaria.\n")
}
```

```
## ESITO: La serie è STAZIONARIA (p-value < 0.05)
## Significato: Puoi procedere con il modello GARCH.
```

The Verdict:
We reject null hypotesysi of non stationarity

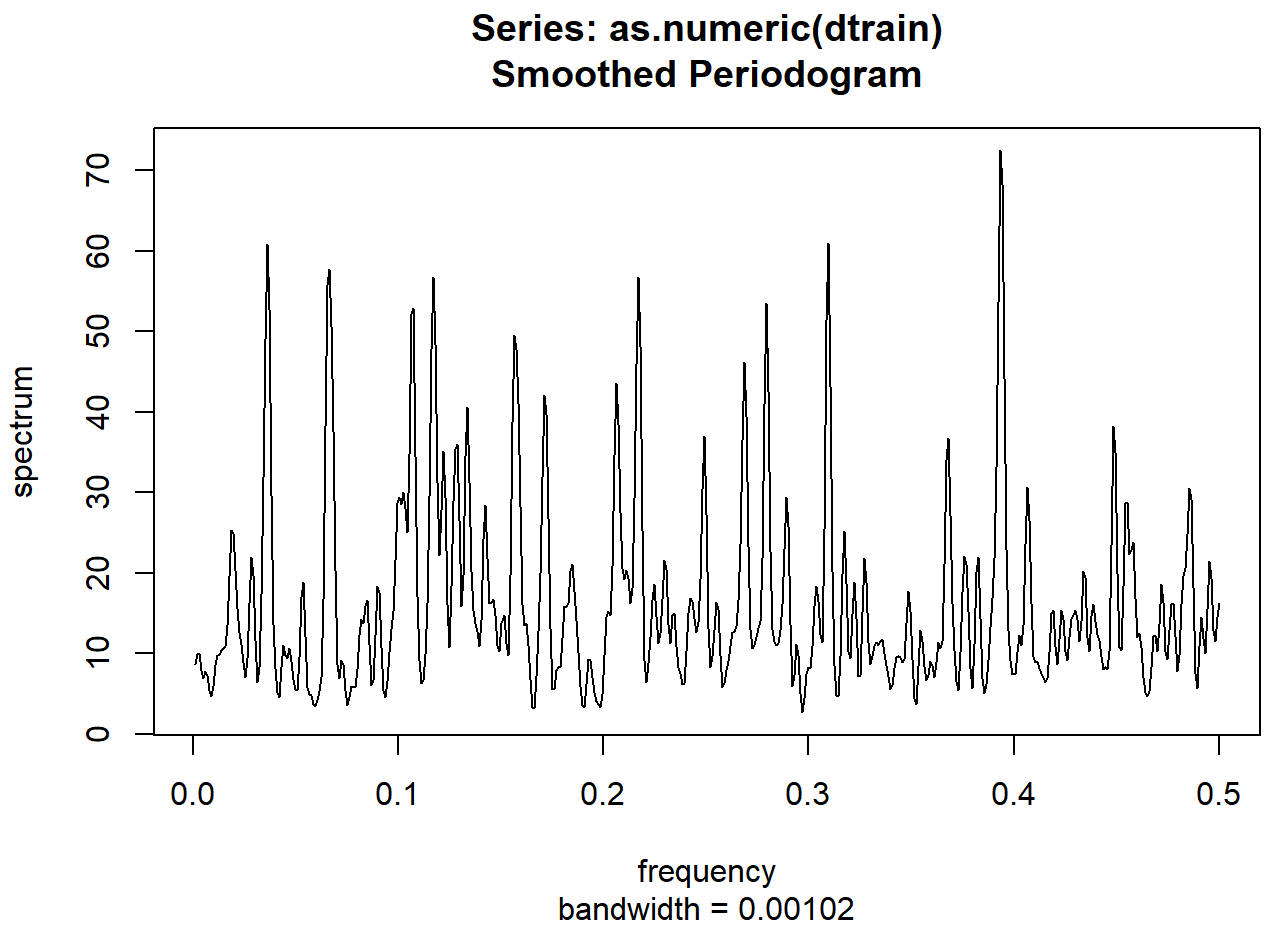
##3.8 Seasonality and ARCH Diagnostics Before finalizing the model specification, we perform a definitive check for residual seasonality and ARCH effects.

###3.8.1 Power Spectrum and Seasonality Tests The Periodogram is utilized to analyze frequency strength. A pronounced peak at a specific frequency would indicate a periodic component. Furthermore, we apply the seastests package for a combined formal verification.

```
# Calcola lo spettro di potenza
?spec.pgram
```

```
## starting httpd help server ... done
```

```
spec_raw <- spec.pgram(as.numeric(dtrain), spans = c(3,3), taper = 0.1, plot = TRUE,log='no')
```




```
# Trova la frequenza con il picco massimo
max_freq <- spec_raw$freq[which.max(spec_raw$spec)]
periodo_dominante <- 1 / max_freq
cat(periodo_dominante)
```

```
## 2.540943
```

4.6. Spectral Analysis Interpretation

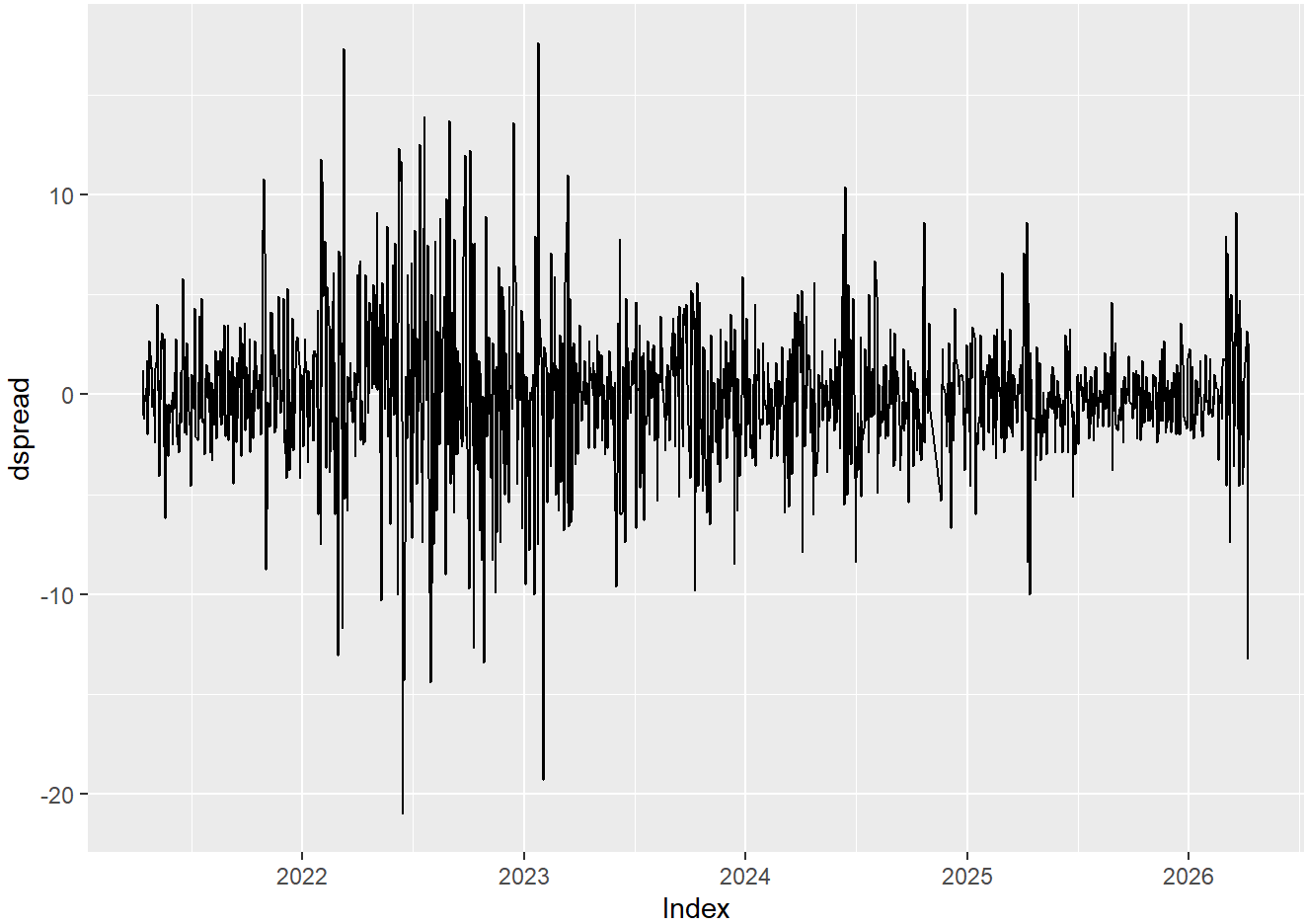
The **Smoothed Periodogram** analysis provides a frequency-domain perspective that complements the previous time-series diagnostics:

- **Frequency Distribution:** The spectrum shows multiple significant peaks rather than a single dominant frequency. This confirms that the BTP-BUND spread is a **multi-cyclical series** where seasonality is fragmented across different time horizons.
- **Short-term Dynamics:** The prominent peaks at higher frequencies (e.g., near 0.4) represent the daily "noise" and short-term volatility. This aligns with the **Maximum value of 250.70 bps**, indicating that sharp, short-lived shocks are a structural feature of the spread's behavior.
- **Low-Frequency Persistence:** The power concentrated near the origin (frequency → 0) represents the **Trend component**. This captures the long-term shifts (2022 surge and 2025 reversion) which, despite the daily noise, remain the most powerful drivers of the spread's variance.

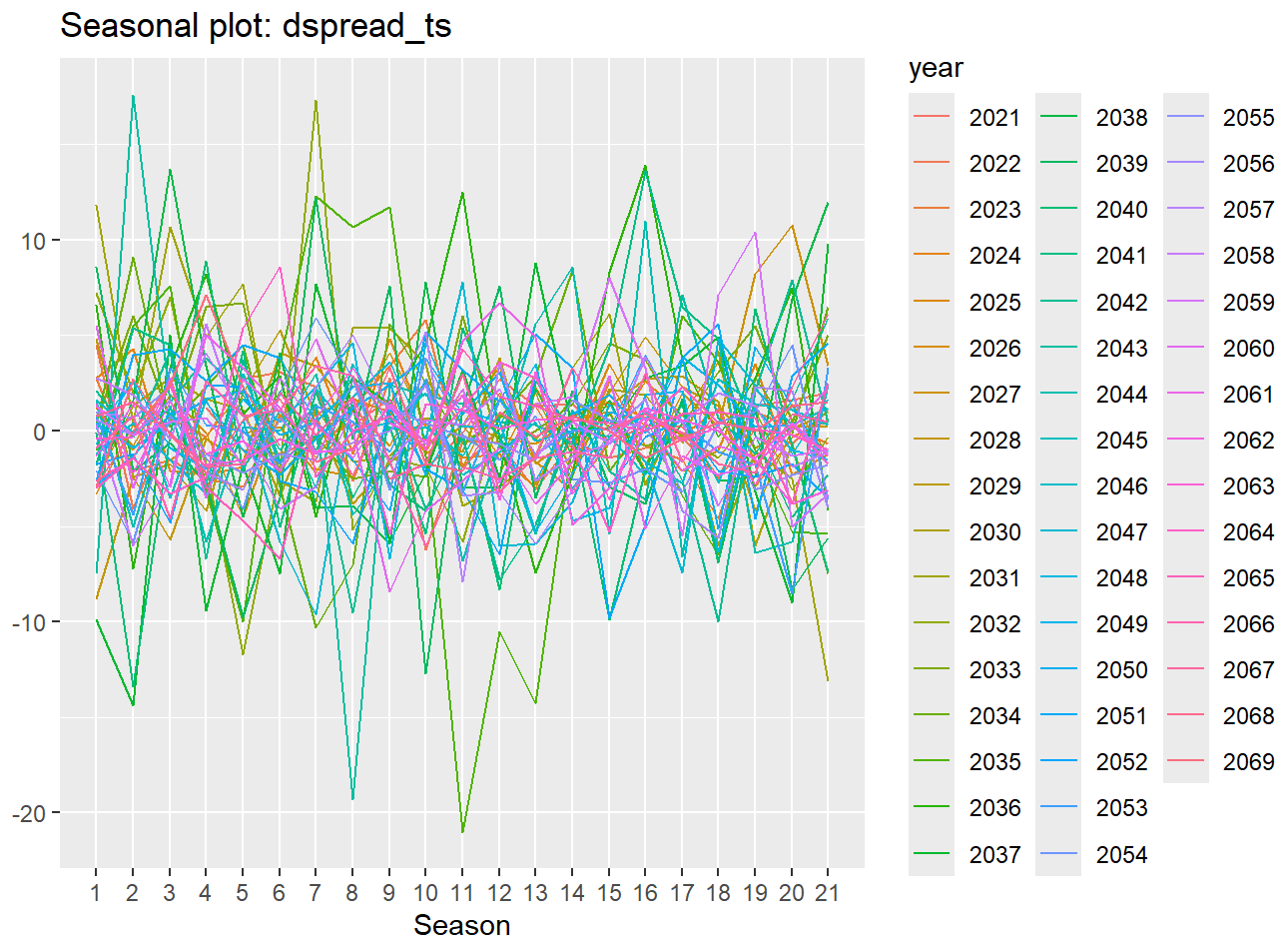
Conclusion: The spectral density confirms that while no simple seasonal rule can predict the spread, the series is far from being pure white noise. The complexity of the periodogram justifies the use of non-linear models (like GARCH) to capture the clusters of energy (volatility) that appear at various frequencies.

Frequenza (s),Periodo Temporale,Obiettivo dell'analisi 5,Settimanale,Verificare pattern intra-settimanali (Trading week). 21,Mensile,Catturare la stagionalità legata ai calendari macroeconomici. 63,Trimestrale,Analizzare l'impatto dei report trimestrali e scadenze tecniche. 252,Annuale,Confermare il pattern stagionale legato alle manovre di bilancio annuali.

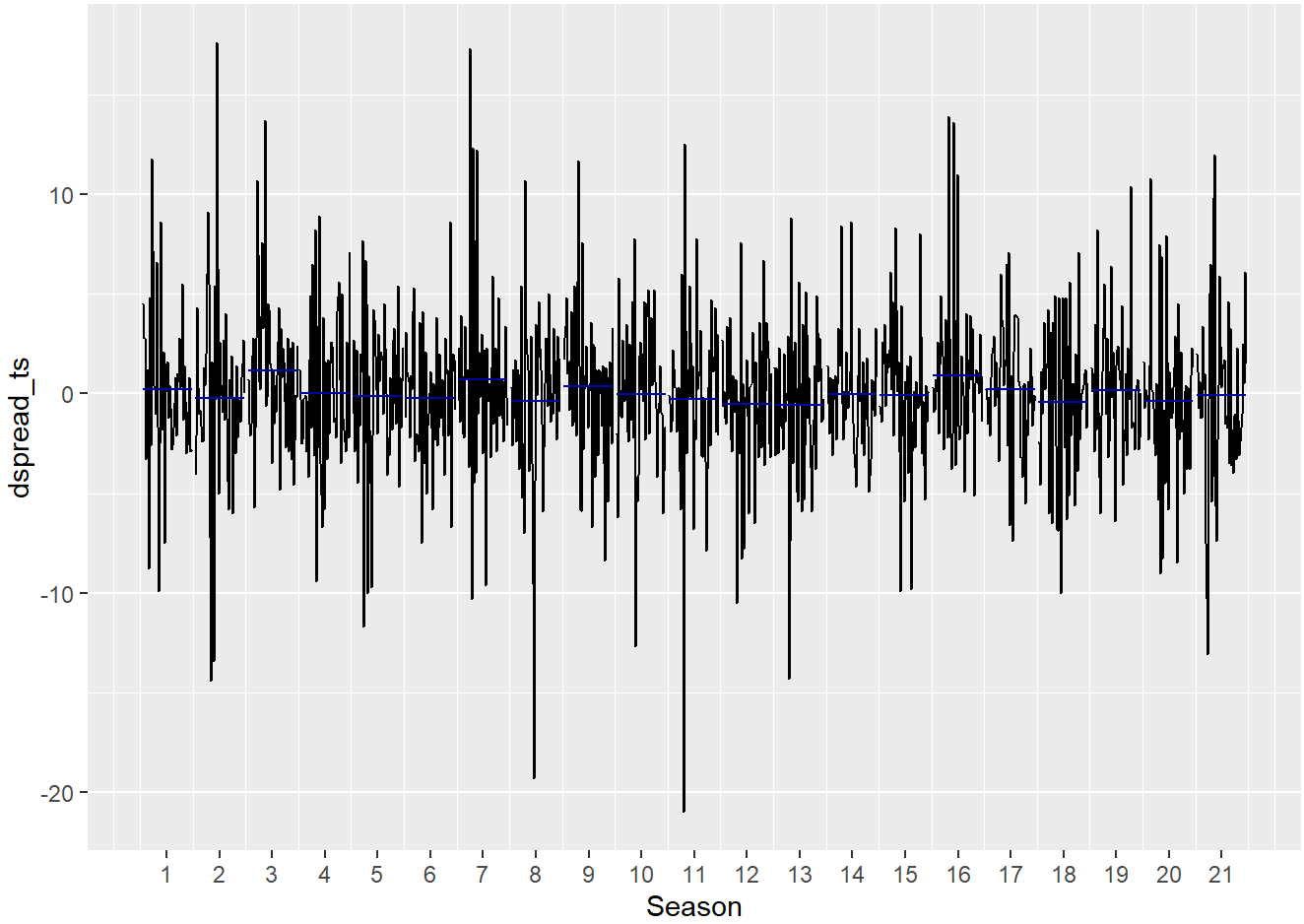
```
autoplot(dspread)
```



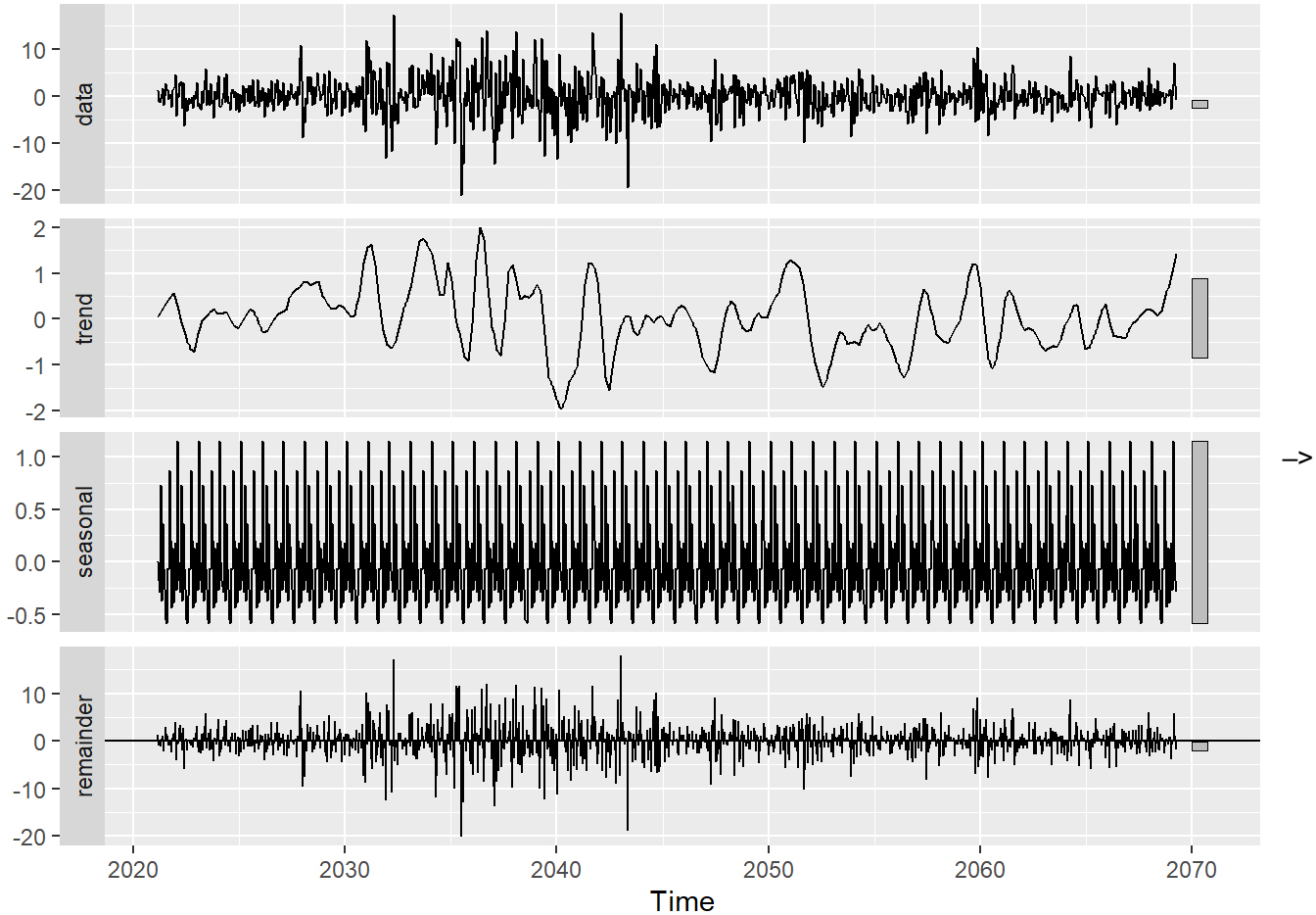

```
#To use this method, we need a ts object, let's create one
dspread_ts <- ts(dtrain, start=c(start_year,start_month,start_day), frequency = 21)#try also wit
h 12
ggseasonplot(dspread_ts)
```




```
ggsubseriesplot(dspread_ts)
```




```
#Look also at decomposed series
decomposed <- stl(dspread_ts, s.window="periodic")
autoplot(decomposed)
```



STL Decomposition of the Differenced Series ### 1. STL Decomposition at Monthly Resolution The decomposition of the BTP-BUND spread at a 21-day frequency reveals a highly active cyclical structure:

- **Trend:** The trend component remains the primary driver, capturing the broad expansion of the spread from 2021 through 2024 and the subsequent compression toward 2026. At this frequency, the trend is more reactive to mid-term economic shifts compared to the annual model ($f = 252$).
- **Seasonal (Monthly):** The seasonal panel displays a persistent, high-frequency oscillation. However, the amplitude is notably small, ranging between **-1 and +1.5 bps**. This suggests that while a monthly cycle exists, its statistical contribution to the total variance is marginal.(Se il tuo spread medio è di 150 bps e la volatilità giornaliera (Remainder) ha picchi di 10-20 bps, un'oscillazione di soli 3 bps è considerata trascurabile (noise-level).)
- **Remainder:** The remainder component continues to exhibit significant "bursts" or clusters of volatility. This indicates that sudden market news (stochastic shocks) has a much larger impact on daily spread changes than any predictable monthly calendar effect.

We use seastests package

```
#install.packages("seastests")

isSeasonal(dspread_ts, test="combined")
```

```
## [1] FALSE
```

```
#We get FALSE

#Individual tests
isSeasonal(dspread_ts, test="qs")
```

```
## [1] FALSE
```

```
isSeasonal(dspread_ts, test="seasdum")
```

```
## [1] FALSE
```

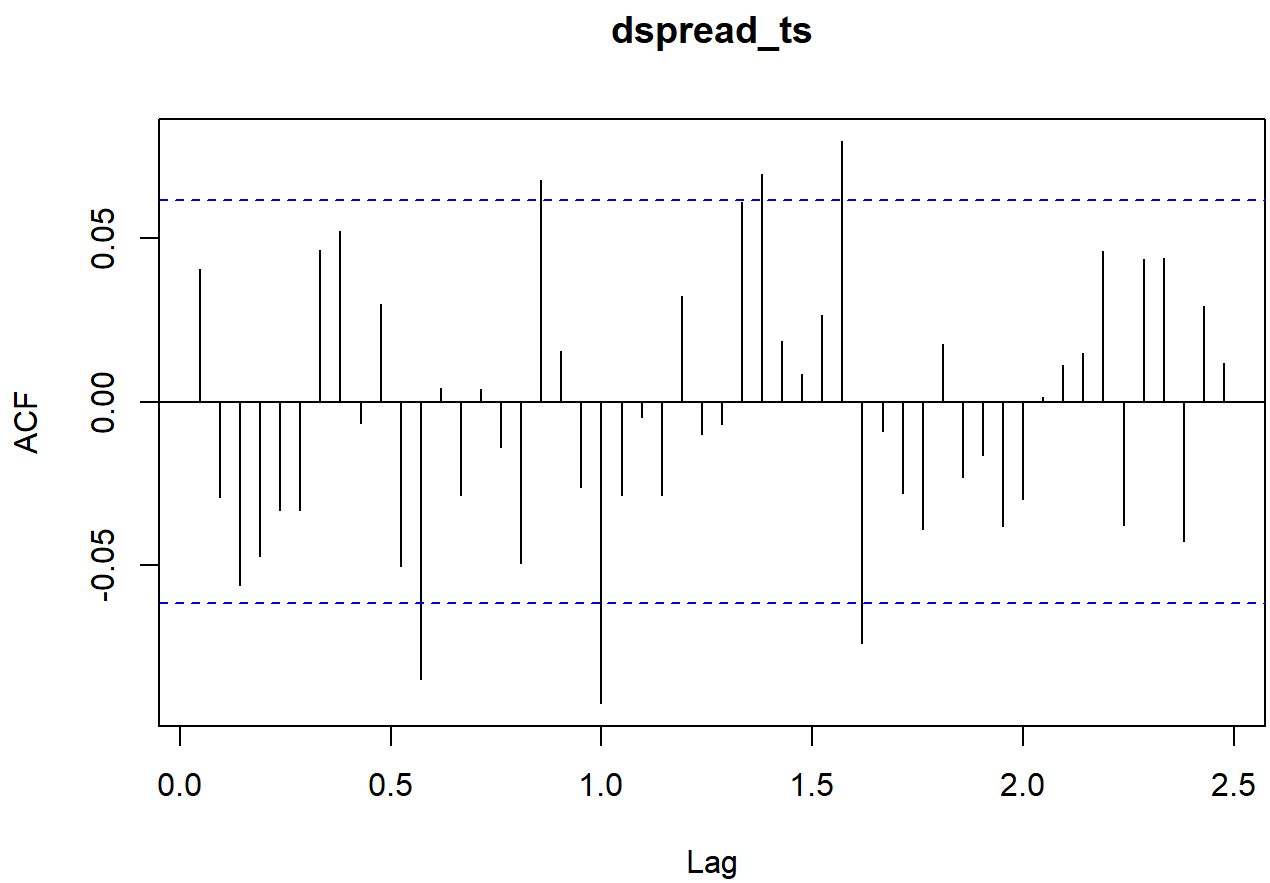
```
isSeasonal(dspread_ts, test="kw")
```

```
## [1] FALSE
```

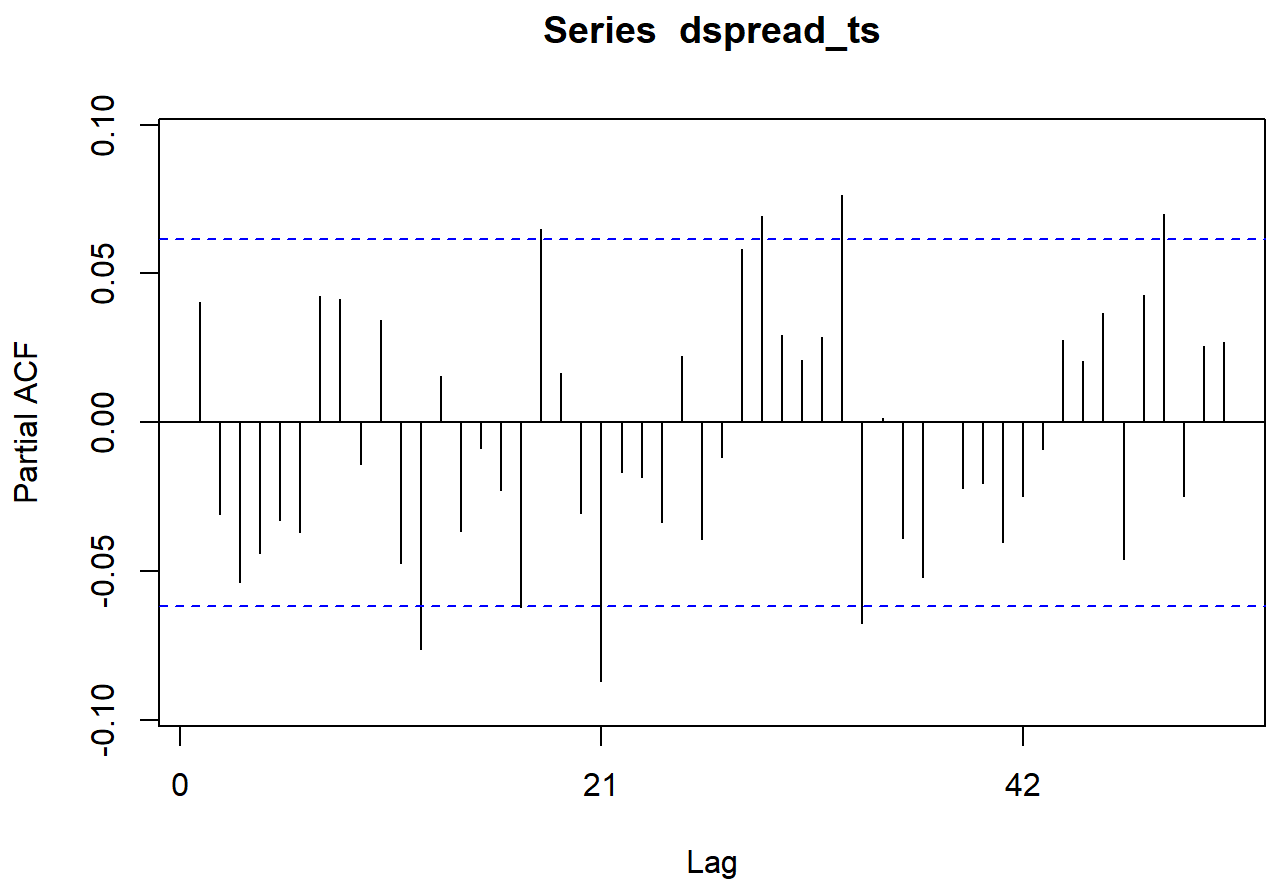
```
isSeasonal(dspread_ts, test="fried")
```

```
## [1] FALSE
```

```
#We get all FALSE

## Now plot ACF and PACF
Acf(dspread_ts, lag.max= 52)
```




```
Pacf(dspread_ts, lag.max = 52)
```




```
## Finally, we can affirm that our data does not contain seasonality, so we can work using norma
l ARIMA models and ARIMA-GARCH models.
```

We conclude the diagnostic phase by applying the seastests package. All individual tests (QS, Seasonal Dummy, Kruskal-Wallis, and Friedman) consistently fail to reject the null hypothesis of no seasonality.

Conclusion: Having confirmed that the series is stationary, lacks seasonality, and exhibits significant ARCH effects, we can proceed with confidence to estimate the non-seasonal ARIMA-GARCH framework.

###3.8.3 Formal Diagnostic Tests: Ljung-Box and ARCH-LM To ensure the necessity of both the mean and variance components of the model, we perform rigorous diagnostic tests on the differenced training set ($dtrain$). We select a high lag (e.g., $lag = 16$) to be meticulous in detecting long-term dependencies.

```
# 1. Ljung-Box Test on differences
lb_ret <- Box.test(as.numeric(dtrain), lag = 10, type = "Ljung-Box")
cat("\nLB Test on ds p-value:\n", lb_ret$p.value, "\n")
```

```
##
## LB Test on ds p-value:
##  0.09617127
```

```
if(lb_ret$p.value < 0.05) cat("Autocorrelation detected, need mean modelling\n") else cat("ds ar
e White Noise\n")
```

```
## ds are White Noise
```

```
# 2. Ljung-Box Test on squared differences
lb_test <- Box.test(as.numeric(dtrain)^2, lag = 10, type = "Ljung-Box")
cat(sprintf("\nLjung-Box on Squared ds at lag 10:\n %.4f\n", lb_test$p.value))
```

```
##
## Ljung-Box on Squared ds at lag 10:
##  0.0000
```

```
if(lb_test$p.value < 0.05) cat("Volatility clustering confirmed\n") else cat("No volatility clus
tering detected\n")
```

```
## Volatility clustering confirmed
```

```
# 3. ARCH-LM Test (Engle's Test for ARCH effects)
arch_test <- ArchTest(dtrain, lags = 10)
cat(sprintf("\nARCH Test (5 lags) LM Test - statistic: %.4f, p-value: %.4f\n", arch_test$statist
ic, arch_test$p.value))
```

```
##
## ARCH Test (5 lags) LM Test - statistic: 115.7243, p-value: 0.0000
```

```
if(arch_test$p.value < 0.05) cat("ARCH effects confirmed. GARCH modelling needed\n") else cat("N
o ARCH effects confirmed\n")
```

```
## ARCH effects confirmed. GARCH modelling needed
```

Interpretation of Results:Autocorrelation: If the p-value for the first Ljung-Box test is below 0.05, we reject the null hypothesis of independence, confirming that the spread changes exhibit autocorrelation and require an ARMA specification.ARCH Effects: A p-value below 0.05 in both the squared Ljung-Box and the ARCH-LM test provides definitive evidence of volatility clustering. This confirms that the variance is not constant (Heteroskedasticity), mandating the use of a GARCH model to capture time-varying risk.

##3.9 Seasonality Validation: Information Criteria Approach While previous visual diagnostics suggested no seasonality, we utilize an automated selection procedure to determine if a seasonal frequency (SARIMA) provides a statistically superior fit. We compare models across plausible frequencies ($f = 1$ for non-seasonal, $f = 8$, and $f = 12$ for monthly proxies in weekly data) based on the Akaike Information Criterion (AIC).

DONT'T RUN THIS!!!

```
#test_freqs <- c(1,5,21, 63,252) # Le frequenze plausibili
#risultati <- data.frame(Freq = test_freqs, AIC = NA)

#for(i in 1:length(test_freqs)) {
  # Crea la TS con la frequenza del loop
#  test_ts <- ts(as.numeric(dtrain), frequency = test_freqs[i])

  # auto.arima testa se la stagionalità a quella frequenza aiuta il modello
#  fit <- auto.arima(test_ts, max.q = 10, max.p = 10,seasonal = TRUE,method="ML",stepwise = FALS
E,approximation = FALSE)
#  risultati$AIC[i] <- fit$aic
#}

# Trova la frequenza migliore
#best_f <- risultati$Freq[which.min(risultati$AIC)]
#cat("Il verdetto statistico è: usa la frequenza", best_f)
```

##3.10 Final Mean Equation Specification Following the automated search, we estimate the final non-seasonal $ARIMA$ model to evaluate the significance of individual parameters.

```
m1 <- auto.arima(as.numeric(dtrain),max.q = 15, max.p = 15,method="ML",
                       seasonal = FALSE,
                       stepwise = FALSE,
                       approximation = FALSE,
                       trace = FALSE)

summary(m1)
```

```
## Series: as.numeric(dtrain)
## ARIMA(2,0,2) with zero mean
##
## Coefficients:
##          ar1      ar2      ma1     ma2
##       1.4264  -0.8571  -1.4106  0.8080
## s.e.  0.2213   0.1306   0.2641  0.1735
##
## sigma^2 = 14.92:  log likelihood = -2798.66
## AIC=5607.32   AICc=5607.38   BIC=5631.91
##
## Training set error measures:
##                      ME     RMSE      MAE MPE MAPE      MASE       ACF1
## Training set 0.01890851 3.854452 2.758363 NaN  Inf 0.6938156 0.01602944
```

Model for daily data (train=4 years): ARIMA(2,0,2)

3.10.1 Statistical Significance of CoefficientsTo ensure model parsimony, we evaluate the significance of each parameter. A coefficient is statistically significant at the 95% confidence level if its t-ratio ($t = \frac{\hat{\phi}}{SE(\hat{\phi})}$) exceeds the critical value of $|1.96|$.

```
check_coef_significance <- function(model) {
  # Estraiamo coefficienti e standard errors
  coefs <- model$coef
  se <- sqrt(diag(model$var.coef))

  # Calcoliamo la t-statistica (z-score)
  t_stats <- coefs / se

  # Calcoliamo i p-value (distribuzione normale a due code)
  p_values <- (1 - pnorm(abs(t_stats))) * 2

  # Creiamo una tabella riassuntiva
  results <- data.frame(
    Coefficiente = round(coefs, 4),
    Std_Error = round(se, 4),
    t_stat = round(t_stats, 4),
    p_value = round(p_values, 4),
    Significativo_95 = ifelse(p_values < 0.05, "SI", "NO")
  )

  cat("\nAnalisi di Significatività dei Coefficienti:\n")
  print(results)

  cat("\nNota: Un coefficiente è significativo al 95% se |t_stat| > 1.96\n")
  print(results)
}
check_coef_significance(m1)
```

```
##
## Analisi di Significatività dei Coefficienti:
##     Coefficiente Std_Error  t_stat p_value Significativo_95
## ar1       1.4264    0.2213  6.4460       0               SI
## ar2      -0.8571    0.1306 -6.5632       0               SI
## ma1      -1.4106    0.2641 -5.3402       0               SI
## ma2       0.8080    0.1735  4.6560       0               SI
##
## Nota: Un coefficiente è significativo al 95% se |t_stat| > 1.96
##     Coefficiente Std_Error  t_stat p_value Significativo_95
## ar1       1.4264    0.2213  6.4460       0               SI
## ar2      -0.8571    0.1306 -6.5632       0               SI
## ma1      -1.4106    0.2641 -5.3402       0               SI
## ma2       0.8080    0.1735  4.6560       0               SI
```

```
m1$coef
```

```
##        ar1        ar2        ma1        ma2
##  1.4263641 -0.8571333 -1.4105857  0.8080356
```

ALL THE COEFFICIENTS OF m1 SARE TATISTICALLY SIGNIFICANT? t ratios are given by estimated coefficients divided by their standard errors; the latter are given by the square root of the element on the main diagonal (variance of coef)

```
names(m1)
```

```
##  [1] "coef"      "sigma2"    "var.coef"  "mask"      "loglik"    "aic"
##  [7] "arma"      "residuals" "call"      "series"    "code"      "n.cond"
## [13] "nobs"      "model"     "bic"       "aicc"      "x"         "fitted"
```

```
summary(m1)
```

```
## Series: as.numeric(dtrain)
## ARIMA(2,0,2) with zero mean
##
## Coefficients:
##          ar1      ar2      ma1     ma2
##       1.4264  -0.8571  -1.4106  0.8080
## s.e.  0.2213   0.1306   0.2641  0.1735
##
## sigma^2 = 14.92:  log likelihood = -2798.66
## AIC=5607.32   AICc=5607.38   BIC=5631.91
##
## Training set error measures:
##                      ME     RMSE      MAE MPE MAPE      MASE       ACF1
## Training set 0.01890851 3.854452 2.758363 NaN  Inf 0.6938156 0.01602944
```

```
t.ratios = m1$coef/sqrt(diag(m1$var.coef))
t.ratios
```

```
##       ar1       ar2       ma1       ma2
##  6.445990 -6.563194 -5.340212  4.655973
```

BUT since the Lijung Box suggests that the serie is a white noie, we specify also the model m0=ARIMA(0,0,0) to compare the performances

library(forecast)

```
# Fit ARIMA(1,0,0) - equivalent to AR(1)
m0 <- Arima(as.numeric(dtrain), order = c(0, 0, 0), include.constant = FALSE)

# Summary of coefficients and information criteria
summary(m0)
```

```
## Series: as.numeric(dtrain)
## ARIMA(0,0,0) with zero mean
##
## sigma^2 = 15.06:  log likelihood = -2805.48
## AIC=5612.95   AICc=5612.96   BIC=5617.87
##
## Training set error measures:
##                      ME     RMSE      MAE MPE MAPE      MASE      ACF1
## Training set 0.01810089 3.880692 2.766073 100  100 0.6957548 0.0403942
```

```
arma_assumptions <- function(m) {
  cat("\n==============================================================\n")
  cat("ARIMA model assumptions & Cycle Analysis:\n")

  # Estrazione corretta per modelli auto.arima/Arima
  # Usiamo i coefficienti AR per la stazionarietà
  ar_coefs <- m$model$phi
  ma_coefs <- m$model$theta

  # 1. Calcolo Radici
  ar_roots <- polyroot(c(1, -ar_coefs))
  ma_roots <- polyroot(c(1, ma_coefs))

  is_stationary <- all(abs(ar_roots) > 1)
  is_invertible <- all(abs(ma_roots) > 1)

  cat("Stationary:", is_stationary, "(Min mod:", round(min(abs(ar_roots)), 3), ")\n")
  cat("Invertible:", is_invertible, "(Min mod:", round(min(abs(ma_roots)), 3), ")\n")

  # 2. Analisi dei Business Cycles (Radici Complesse)
  # Cerchiamo se esistono radici con parte immaginaria significativa
  complex_roots <- ar_roots[Im(ar_roots) != 0]

  cat("\n--- Business Cycle Analysis ---\n")
  if(length(complex_roots) > 0) {
    cat("The model IMPLIES the existence of cycles.\n")
    cat("Reason: The AR characteristic equation has complex conjugate roots.\n")

    # Calcolo del periodo del ciclo (se ti serve per l'analisi)
    # Periodo = 2*pi / acos(Re(root)/abs(root))
    root <- complex_roots[1]
    period <- 2 * pi / atan2(Im(root), Re(root))
    cat("Estimated average cycle period:", round(abs(period), 2), "units of time.\n")
  } else {
    cat("The model DOES NOT imply cycles.\n")
    cat("Reason: All AR roots are real; the model returns to equilibrium monotonically.\n")
  }

  # 3. Check for common roots
  common_roots <- FALSE
  if(length(ar_roots) > 0 && length(ma_roots) > 0) {
    for(ar in ar_roots) {
      for(ma in ma_roots) {
        if(abs(ar - ma) < 0.05) common_roots <- TRUE
      }
    }
  }

  cat("\n--- Root Validation ---\n")
  if(common_roots) {
    cat("Common roots detected, assumption violation\n")
  } else {
```

```
    cat("No common roots detected, assumption validated\n")
  }
  cat("============================================================\n")
}

# Esecuzione
arma_assumptions(m1)
```

```
##
## ============================================================
## ARIMA model assumptions & Cycle Analysis:
## Stationary: TRUE (Min mod: 1.08 )
## Invertible: TRUE (Min mod: 1.112 )
##
## --- Business Cycle Analysis ---
## The model IMPLIES the existence of cycles.
## Reason: The AR characteristic equation has complex conjugate roots.
## Estimated average cycle period: 9.09 units of time.
##
## --- Root Validation ---
## Common roots detected, assumption violation
## ============================================================
```

Residual's check

NOTE: Kolmogorov-Smirnov test (KS) to test wheter they follow a t-distribution
H_0 : serie follows supposed distribution

```
cat("\n============================================================\n")
```

```
##
## ============================================================
```

```
cat("\nARMA residuals check:\n")
```

```
##
## ARMA residuals check:
```

```
# ARIMA residuals
n_parameters1<-sum(m1$coef!=0)
n_parameters2<-sum(m0$coef!=0)
arima_res1<- residuals(m1)
arima_res2<- residuals(m0)
arma_residual <- function(res,n_parameters) {
  cat("\n==============================================================\n")
  cat("ANALISI APPROFONDITA DEI RESIDUI\n")
  cat("==============================================================\n")

# 1. Ljung-Box (Media e Autocorrelazione)
  lb_arima <- Box.test(res, lag = 15, type = "Ljung-Box",fitdf = n_parameters)
  cat("\n [1]LB test on ARIMA residuals (15 lags) p-value: ", round(lb_arima$p.value, 4), "\n")
  if(lb_arima$p.value >= 0.05) cat("ARIMA model cleared the mean\n") else cat("ARIMA model DID N
OT clear the mean\n")
#Regola di Hyndman (autore di forecast): Per dati non stagionali si usa $h = 10$, per dati stagi
onali si usa $h = 2m$ (dove $m$ è la frequenza). Nel tuo caso ($m=8$), dovresti testare almeno f
ino a 16 lag.

# 2. Durbin-Watson (Autocorrelazione al lag 1)
# Nota: Il DW classico si usa su modelli lineari, sui residui ARIMA è indicativo
  dw_test <- dwtest(as.vector(res) ~ 1)
  cat("\n[2] Durbin-Watson Test p-value: ", round(dw_test$p.value, 4))
  if(dw_test$p.value >= 0.05) cat(" -> OK: No autocorrelazione lag 1\n") else cat(" -> KO: Autoc
orrelazione lag 1\n")

# 3. Breusch-Godfrey Test (Autocorrelazione di ordine superiore)
  bg_test <- bgtest(as.vector(res) ~ 1, order = 12)
  cat("\n[3] Breusch-Godfrey Test (order 15) p-value: ", round(bg_test$p.value, 4))
  if(bg_test$p.value >= 0.05) cat(" -> OK: No autocorrelazione superiore\n") else cat(" -> KO: A
utocorrelazione superiore\n")

# 4. Jarque-Bera Test (Normalità dei residui)
  jb_test <- jarque.bera.test(res)
  cat("\n[4] Jarque-Bera Test p-value: ", round(jb_test$p.value, 4))
  if(jb_test$p.value >= 0.05) cat(" -> OK: Residui Normali\n") else cat(" -> KO: Residui NON Nor
mali (code grasse)\n")

#5. Kolmogorov-Smirnov (KS)
  check_t <- function(residuals) {
  fit_t <- suppressWarnings(fitdistr(residuals, densfun = "t"))
  mu <- fit_t$estimate["m"]
  sigma <- fit_t$estimate["s"]
  df <- fit_t$estimate["df"]
  std_res <- (residuals - mu) / sigma
  ks <- ks.test(x = std_res, y = "pt", df = df)
  cat("\n[5] KS test assuming t-distribution:\n")
  print(c(DoF = df, KS_p_value = ks$p.value))
  if(ks$p.value > 0.05) cat("\nResiduals are t-distributed\n") else cat("\nResiduals are NOT t-d
istributed\n")
  }
  check_t(res)
```

```
  # 6. Effetti ARCH (Eteroschedasticità)
  lb_arima_2 <- Box.test(res^2, lag =15, type = "Ljung-Box",fitdf = n_parameters)
  cat("\n [6]LB test on ARIMA squared residuals (10 lags) p-value: ", round(lb_arima_2$p.value,
4), "\n")
  if(lb_arima_2$p.value < 0.05) cat("ARIMA model has residual variance to explain. GARCH model n
eeded\n") else cat("ARIMA model residual variance is OK\n")

  # 7.ARCH test
  arch_test_res <- ArchTest(res, lags = 15)
  cat(sprintf("\n [7]ARCH Test on ARIMA (5 lags) residuals:\n LM Test - statistic: %.4f, p-valu
e: %.4f\n",
              arch_test_res$statistic, arch_test_res$p.value))
  if(arch_test_res$p.value < 0.05) cat("ARCH effects confirmed. GARCH modelling needed\n") else
cat("No ARCH effects confirmed\n")
}

print(arma_residual(arima_res1,n_parameters1))
```

```
##
## ==============================================================
## ANALISI APPROFONDITA DEI RESIDUI
## ==============================================================
##
##  [1]LB test on ARIMA residuals (15 lags) p-value:  0.3924
## ARIMA model cleared the mean
##
## [2] Durbin-Watson Test p-value:  0.3043 -> OK: No autocorrelazione lag 1
##
## [3] Breusch-Godfrey Test (order 15) p-value:  0.5696 -> OK: No autocorrelazione superiore
##
## [4] Jarque-Bera Test p-value:  0 -> KO: Residui NON Normali (code grasse)
##
## [5] KS test assuming t-distribution:
##     DoF.df KS_p_value
##  3.5205786  0.9257897
##
## Residuals are t-distributed
##
##  [6]LB test on ARIMA squared residuals (10 lags) p-value:  0
## ARIMA model has residual variance to explain. GARCH model needed
##
##  [7]ARCH Test on ARIMA (5 lags) residuals:
##  LM Test - statistic: 108.5445, p-value: 0.0000
## ARCH effects confirmed. GARCH modelling needed
## NULL
```

```
print(arma_residual(arima_res2,n_parameters2))
```

```
##
## ============================================================
## ANALISI APPROFONDITA DEI RESIDUI
## ============================================================
##
##  [1]LB test on ARIMA residuals (15 lags) p-value:  0.0292
## ARIMA model DID NOT clear the mean
##
## [2] Durbin-Watson Test p-value:  0.0989 -> OK: No autocorrelazione lag 1
##
## [3] Breusch-Godfrey Test (order 15) p-value:  0.0288 -> KO: Autocorrelazione superiore
##
## [4] Jarque-Bera Test p-value:  0 -> KO: Residui NON Normali (code grasse)
```

```
## Warning in ks.test.default(x = std_res, y = "pt", df = df): ties should not be
## present for the one-sample Kolmogorov-Smirnov test
```

```
##
## [5] KS test assuming t-distribution:
##     DoF.df KS_p_value
##  3.4802820  0.7421928
##
## Residuals are t-distributed
##
##  [6]LB test on ARIMA squared residuals (10 lags) p-value:  0
## ARIMA model has residual variance to explain. GARCH model needed
##
##  [7]ARCH Test on ARIMA (5 lags) residuals:
##  LM Test - statistic: 120.4733, p-value: 0.0000
## ARCH effects confirmed. GARCH modelling needed
## NULL
```

m1=ARIMA(2,0,2) pass all the tests, while m0=ARIMA(0,0,0) shows autocorrelation up to lag 15 (but if we set breush-Godfrey up to lag 11, there is not autocorrelation, so the problem is the lag 12)

ORA FACCIAMO STA BENEDETTA PREVISIONE: IN SAMPLE PREDICTIONS

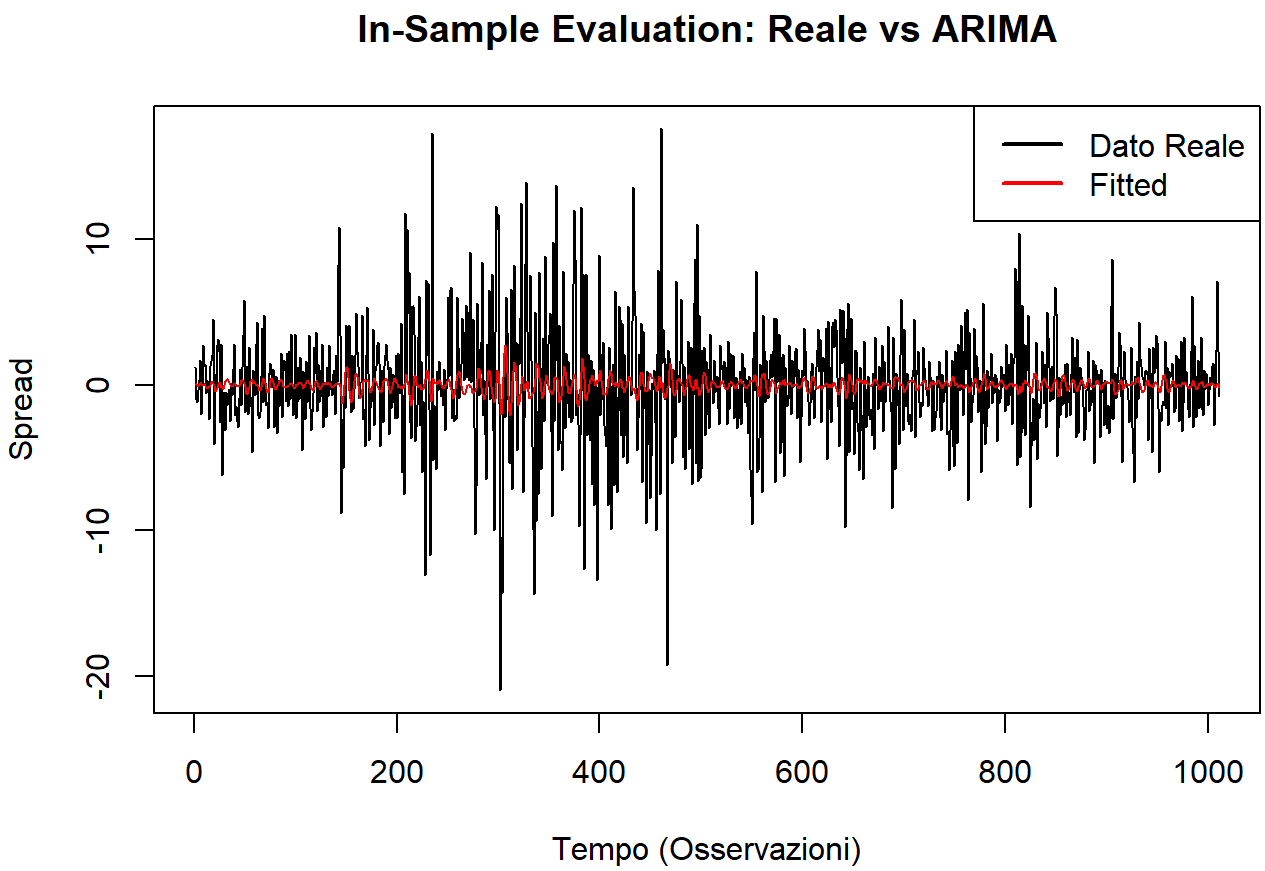
```
# Estraiamo i valori fittati dal modello m1
fitted_values <- fitted(m1)
length(fitted_values)
```

```
## [1] 1011
```

```
length(train)
```

```
## [1] 1011
```

```
# Visualizzazione grafica: Reale vs Fittato
plot(as.numeric(dtrain), type="l", col="black", lwd=1,
     main="In-Sample Evaluation: Reale vs ARIMA",
     ylab="Spread", xlab="Tempo (Osservazioni)")
lines(as.numeric(na.omit(fitted_values)), col="red")
legend("topright", legend=c("Dato Reale", "Fitted"), col=c("black", "red"), lty=1, lwd=2)
```



IN SAMPLE EVALUATION

```
residuals <- dtrain - fitted_values
```

```
## Warning: Incompatible methods ("Ops.zoo", "Ops.ts") for "-"
```

```
# Core metrics
accuracy(as.numeric(dtrain), as.numeric(fitted_values))
```

```
##                    ME     RMSE      MAE     MPE     MAPE
## Test set -0.01890851 3.854452 2.758363 4155.56 17250.67
```

```
# R-squared (will be 0 or near 0 — expected for mean model)
SS_res <- sum(residuals^2, na.rm = TRUE)
SS_tot <- sum((dtrain - mean(train, na.rm = TRUE))^2, na.rm = TRUE)
R2 <- 1 - SS_res / SS_tot
cat("R2 del modello:", round(R2, 2), "%")
```

```
## R2 del modello: 1 %
```

```
# Calcoliamo i segni (1 per positivo, -1 per negativo)
sign_actual <- sign(dtrain)
sign_fitted <- sign(fitted_values)

# Creiamo un vettore logico: TRUE se i segni sono uguali
hits <- (sign_actual == sign_fitted)
```

```
## Warning: Incompatible methods ("Ops.zoo", "Ops.ts") for "=="
```

```
# Calcoliamo la percentuale di successo (escludendo eventuali NA)
DA <- mean(hits, na.rm = TRUE) * 100
DA
```

```
## [1] 52.12661
```

OUT-SAMPLE PREDICTION AND EVALUATION

```
# 1. Applichiamo il modello alla serie completa (genera tutti i forecast 1-step)
refit <- Arima(dspread, model = m1)

# 2. Estraiamo solo i valori fittati che corrispondono al periodo di TEST
# (Supponiamo che il tuo test set sia lungo 33 osservazioni)
pred_out <- tail(fitted(refit), n_test)
reali_out <- tail(as.numeric(dspread), n_test)

# 3. Calcolo metriche Out-of-Sample
accuracy(pred_out, reali_out)
```

```
##                  ME     RMSE      MAE MPE MAPE
## Test set -0.2378187 2.313881 1.607917 Inf  Inf
```

```
# R-squared (will be 0 or near 0 — expected for mean model)
SS_res <- sum((pred_out-reali_out)^2, na.rm = TRUE)
SS_tot <- sum((test - mean(test, na.rm = TRUE))^2, na.rm = TRUE)
R2 <- 1 - SS_res / SS_tot
cat("R2 del modello:", round(R2, 2), "%")
```

```
## R2 del modello: 0.98 %
```

```
# Calcoliamo i segni (1 per positivo, -1 per negativo)
sign_actual <- sign(test)
sign_fitted <- sign(pred_out)
hits <- (sign_actual == sign_fitted)
```

```
## Warning: Incompatible methods ("Ops.zoo", "Ops.ts") for "=="
```

```
## Warning in sign_actual == sign_fitted: longer object length is not a multiple
## of shorter object length
```

```
# Calcoliamo la percentuale di successo (escludendo eventuali NA)
DA <- mean(hits, na.rm = TRUE) * 100
cat("DA del modello:", round(DA, 2), "%")
```

```
## DA del modello: 48.85 %
```

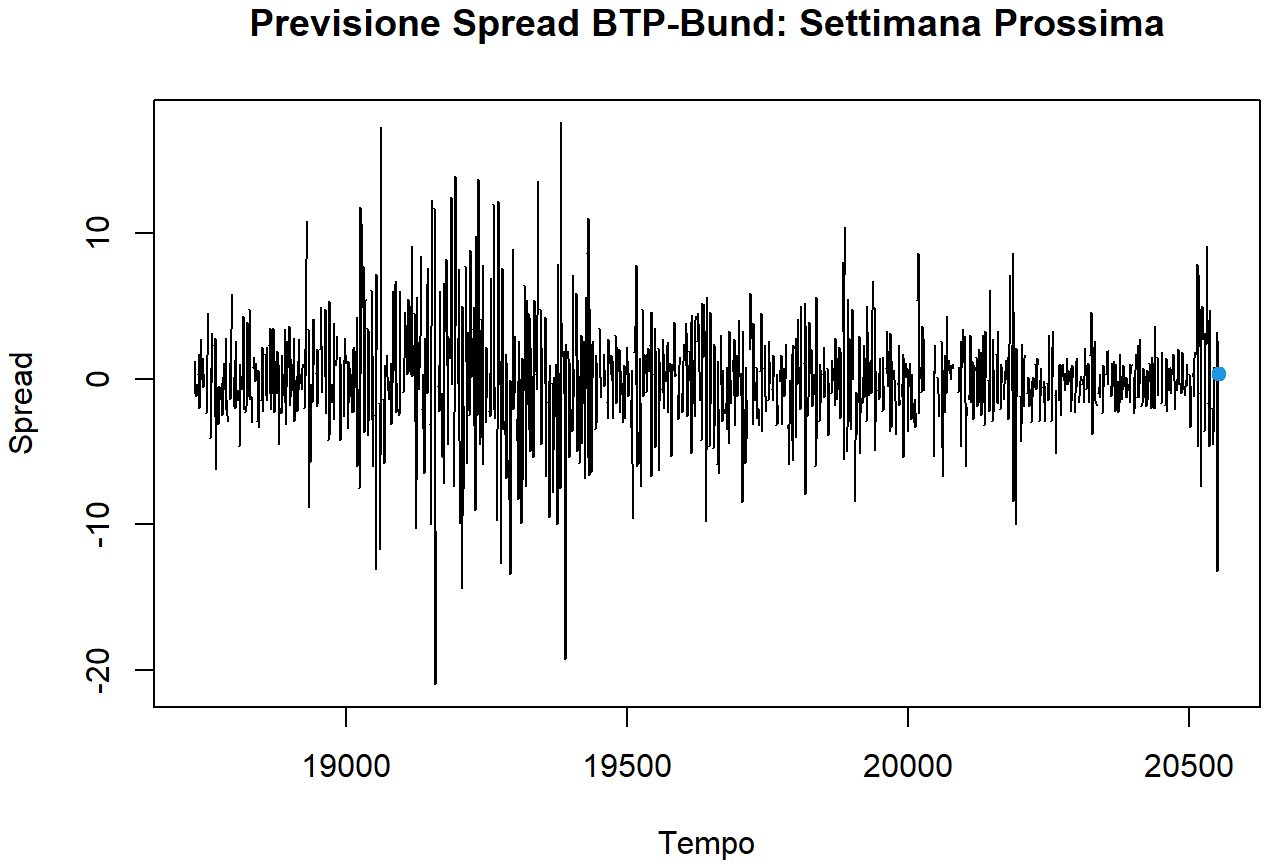
IF WE SUBSTITUTE m1 with m0, we can see that m1 perform better both insample and out-sample

FORECAST DI domani PROSSIMA (diff_spreads)

```
# 2. Generiamo la previsione per la prossima settimana
forecast_next_week <- forecast(refit, h = 1)

# 3. Visualizziamo il risultato
print(forecast_next_week)
```

```
##       Point Forecast     Lo 80    Hi 80     Lo 95    Hi 95
## 20554      0.3304328 -4.627131 5.287996 -7.251506 7.912372
```

```
# 4. Grafico della previsione con intervalli di confidenza
plot(forecast_next_week, main = "Previsione Spread BTP-Bund: Settimana Prossima",
     ylab = "Spread", xlab = "Tempo")
```



Lo 80 / Hi 80 (da -21.79 a +22.18): Il modello è sicuro all'80% che la variazione dello spread oscillerà tra una discesa di 21.8 punti e una salita di 22.2 punti. È un range abbastanza ampio che riflette la volatilità tipica dei mercati.

Lo 95 / Hi 95 (da -33.43 a +33.82): Il modello è sicuro al 95% (standard statistico per la tesi) che la variazione non supererà i ±33 punti base.

Nota tecnica: Se la prossima settimana lo spread dovesse muoversi di 50 punti, sapresti che è successo un evento "fuori dal comune" (un outlier) che il modello non poteva prevedere. TRASFORMAZIONE IN spread:

```
# 1. Recuperiamo l'ultimo valore REALE dello spread (non differenziato)
ultimo_spread_reale <- tail(spread, 1)

# 2. Otteniamo la previsione della differenza (il "delta")
previsione_diff <- forecast(m1, h = 1)$mean[1]

# 3. Calcoliamo lo spread atteso
spread_previsto <- ultimo_spread_reale + previsione_diff

cat("Lo spread previsto per domani è:", spread_previsto)
```

```
## Lo spread previsto per domani è: 77.0534
```

CONSIDERARE DI FARE UNA CROSS VALIDATION PER UNA ROBA PIU ACCURATA

Volatility Analysis

We restart from our model

```
summary(m1)
```

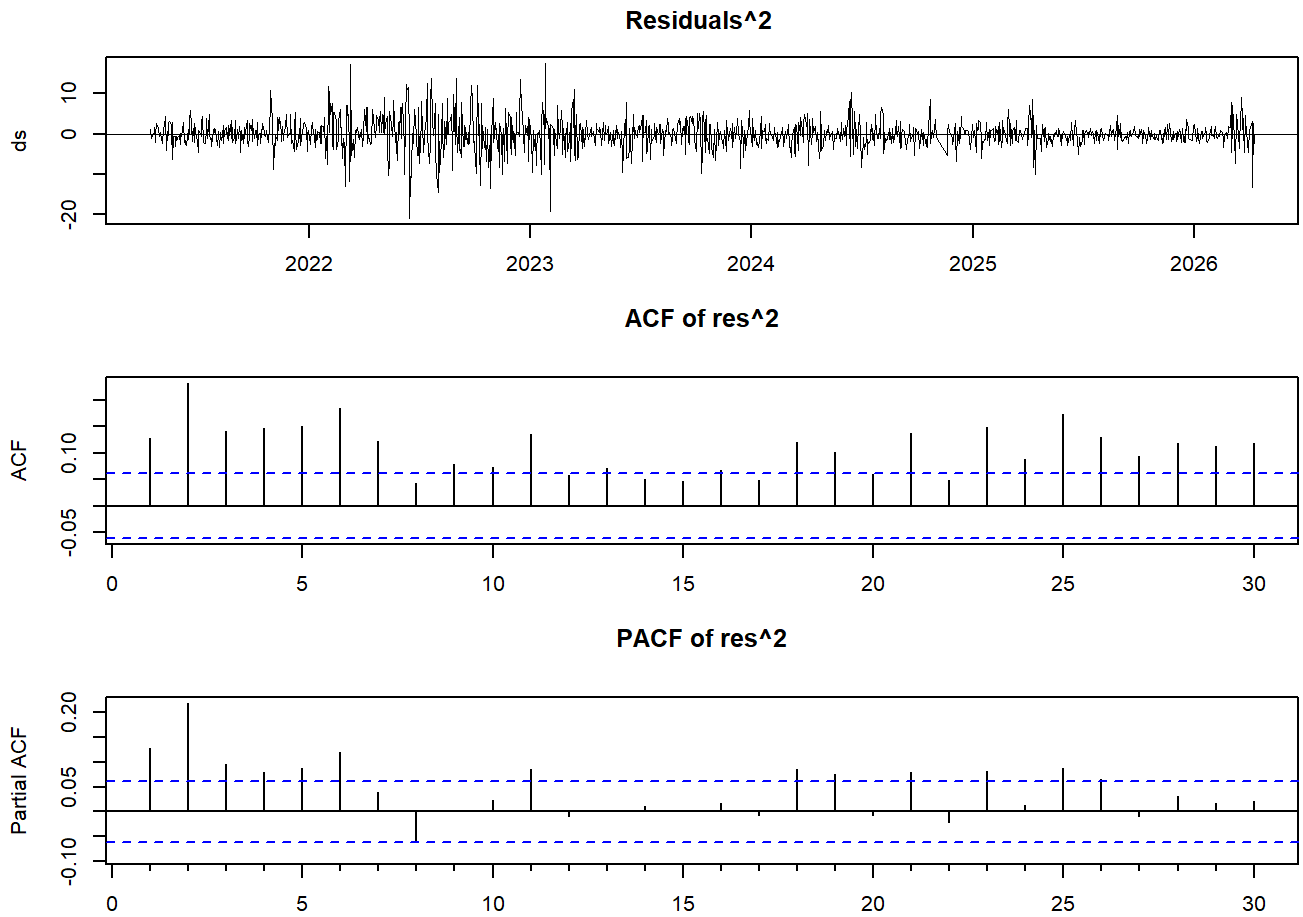
```
## Series: as.numeric(dtrain)
## ARIMA(2,0,2) with zero mean
##
## Coefficients:
##          ar1      ar2      ma1     ma2
##       1.4264  -0.8571  -1.4106  0.8080
## s.e.  0.2213   0.1306   0.2641  0.1735
##
## sigma^2 = 14.92:  log likelihood = -2798.66
## AIC=5607.32   AICc=5607.38   BIC=5631.91
##
## Training set error measures:
##                      ME     RMSE      MAE MPE MAPE      MASE       ACF1
## Training set 0.01890851 3.854452 2.758363 NaN  Inf 0.6938156 0.01602944
```

Remark that the mean is = 0 so we can work directly on residuals

```
fitted_values <- fitted(m1)
res<-dtrain - fitted_values
```

```
## Warning: Incompatible methods ("Ops.zoo", "Ops.ts") for "-"
```

```
par(mfrow = c(3, 1), mar = c(3, 4, 3, 2))

plot(index(as.xts(dspread)), coredata(as.xts(dspread)), type = "l", lwd = 0.7,
     main = "Residuals^2 ", xlab = "", ylab = "ds")
abline(h = 0, col = "black", lwd = 0.5)
Acf(as.numeric(res)^2,   lag.max = 30, main = "ACF of res^2")
Pacf(as.numeric(res)^2,  lag.max = 30, main = "PACF of res^2")
```




```
par(mfrow = c(1, 1))
```

Check for autocorrelation and ARCH effects on `train`:

```
n_parameters<-sum(m1$coef!=0)
# Ljung-Box on DS^2
lb_ds2 <- Box.test(as.numeric(res)^2, lag = 10, type = "Ljung-Box",fitdf = n_parameters)
cat(sprintf("LB Test on DS^2 p-value: %.4f\n", lb_ds2$p.value))
```

```
## LB Test on DS^2 p-value: 0.0000
```

```
if (lb_ds2$p.value < 0.05) cat("Volatility clustering confirmed\n") else cat("No volatility clus
tering detected\n")
```

```
## Volatility clustering confirmed
```

```
# ARCH test
arch_test <- ArchTest(as.numeric(res), lags = 10)
cat(sprintf("ARCH Test (10 lags) p-value: %.4f\n", arch_test$p.value))
```

```
## ARCH Test (10 lags) p-value: 0.0000
```

```
if (arch_test$p.value < 0.05) cat("ARCH effects confirmed — GARCH modelling needed\n") else cat
("No ARCH effects\n")
```

```
## ARCH effects confirmed — GARCH modelling needed
```

We clearly need a specification model for volatility, but which one? Let's see if we have a unit root (i.e if the shock in volatility is persistent)

```
adf.test(as.numeric(res)^2)
```

```
## Warning in adf.test(as.numeric(res)^2): p-value smaller than printed p-value
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  as.numeric(res)^2
## Dickey-Fuller = -6.7362, Lag order = 10, p-value = 0.01
## alternative hypothesis: stationary
```

We reject the hyptosisi of unit root –> so we don't need to use IGARCH (correggetemi se sbaglio)

Model 3: ARMA-GARCH

We try to find the best ARMA-GARCH fitting different combinations of models, then decide considering a precise IC, kinda like an auto_arima but hand-builded.

Create a function that simulates 1 iteration of a for cycle, then instead of a for cycle use parrallel (mclapply) It will run the function over n times using n CPU simultanealy.

Can change the critera or modify the parameters, if too much combinations the code will run for ages.

We will keep the max possible GARCH orders to 1, that's cause vast majority of the time a GARCH(1,1) is perfect for financial series. We could add more GARCH model other than `sGARCH` and `eGARCH` but since those are the most used we keep only them, also other models have additional parameter than can increase the computational time. We also capped max ARMA order to 3 to prevent the AIC from picking up noise (it does, tried to run multiple times and it gives back models that do not pass diagnostic tests).

The ugarch solvers (`gosolnp`) use random generate starting point, to have reproducibility of results we need to impose a seed whenever a optimization is done.

NOTE: in `dists` you choose the distribution you suppose the std residuals will follow, `norm` = normal, `std` = t, `sstd` = skewed t. I removed `norm` and cause the function would wrongly choose `norm` over the other 2: you can see if is the distribution fit the residuals in the printed residual plot or in the "Adjusted Pearson Goodness-of-Fit Test" at the end of the model output, when `norm` the fit is worse. The distribution that better fit the residuals is `sstd` (even if not perfect).

The reason is that `sstd` and `std` have more parameters than `norm` (number of parameter: `sstd` > `std` > `norm`) and since the train data is "small" this penalize them more than it should in the eye of AIC/BIC. `norm` will end up with better IC than `sstd` but then you have the wrong distribution for the residuals and/or fail the diagnostic tests.

NOTE 2: `mc.cores = detectCores() - 1` is a MAC thing, on Windows it does not work, it will show that the code is running on x core but actually will use just one, like a normal `for` cycle.

eval = FALSE –> Questo codice NON verrà eseguito con “Run All”

```
start_time <- proc.time() # to see for how much the code run

best_garch <- function(y,
                       ar_lags  = 0:5,
                       ma_lags  = 0:5,
                       garch_p  = 1:1,
                       garch_q  = 1:1,
                       models   = c("sGARCH", "sGARCH-M"),
                       dists    = c("sstd", "std"),
                       criteria = "aic",
                       timeout  = 60,
                       seed = 42) {

  # reproducibility for the hybrid solver, it uses random starting point
  RNGkind("L'Ecuyer-CMRG")
  set.seed(seed)

  # Create every combinations possible for the parameter
  combos <- expand.grid(
    ar_lag  = ar_lags,
    ma_lag  = ma_lags,
    garch_p = garch_p,
    garch_q = garch_q,
    model   = models,
    dist    = dists,
    stringsAsFactors = FALSE
  )

  cat("Total combinations to fit:", nrow(combos), "\n")
  cat("Running on", parallel::detectCores() - 1, "cores\n")
  cat("It could take couple of minutes ...\n\n")

  fit_one <- function(i) {
    arp   <- combos$ar_lag[i]
    map   <- combos$ma_lag[i]
    gp    <- combos$garch_p[i]
    gq    <- combos$garch_q[i]
    model <- combos$model[i]
    dist  <- combos$dist[i]

    spec <- ugarchspec(
      variance.model     = list(model = model, garchOrder = c(gp, gq)),
      mean.model         = list(armaOrder = c(arp, map), include.mean = TRUE),
      distribution.model = dist
    )
    # If R encounter an error it stop the cycle, tryCatch catch the error and return NULL, then
continue
    fit <- tryCatch({
      withTimeout({
        suppressWarnings(
          ugarchfit(spec = spec, data = y, solver = "gosolnp",
          solver.control = list(n.restarts = 5, rseed = 42, trace = 0)) # for reproducibility)
```

```
        )
      },
      timeout   = timeout,
      onTimeout = "silent")
    },
    error   = function(e) NULL)

    # Store only not null + right class
    if (!is.null(fit) && inherits(fit, "uGARCHfit") && fit@fit$convergence == 0) {
      ic <- infocriteria(fit)
      return(data.frame(
        ar_p    = arp,
        ma_q    = map,
        garch_p = gp,
        garch_q = gq,
        model   = model,
        dist    = dist,
        aic     = as.numeric(round(ic["Akaike", ] * length(y), 2)), # the function calculate the
aic only for that obs
        bic     = as.numeric(round(ic["Bayes",  ] * length(y), 2)),
        likelihood = as.numeric(round(likelihood(fit), 2)),
        stringsAsFactors = FALSE
      ))
    }
    return(NULL)
  }

# Run more itrations simultanely with every CPU-1, instead of for cycle
# results_list = mclapply(
#    seq_len(nrow(combos)),
#    fit_one,
#    mc.cores = detectCores() - 1
#  )


# questa è versione macOS:
 results_list <- mclapply(
  seq_len(nrow(combos)),
  fit_one,
  mc.cores = N_CORES  # usa la variabile definita sopra
)

  #versione per Windows:
  #  Crea il "ponte" tra i core del tuo PC (Cluster)
  #cl <- makeCluster(detectCores() - 1)
  # Esporta tutto ciò che serve ai core "figli" (fondamentale su Windows!)
  # Esportiamo i dati (y), la tabella (combos), la funzione (fit_one) e il timeout
  #clusterExport(cl, varlist = c("combos", "y", "timeout", "fit_one"), envir = environment())
  # Carica le librerie su ogni singolo core
  #clusterEvalQ(cl, {
  #  library(rugarch)
  #  library(R.utils)
```

```
  #  library(dplyr)
  #})
  # Esegui il calcolo (parLapply sostituisce mclapply)
  #results_list <- parLapply(cl, seq_len(nrow(combos)), fit_one)
  # IMPORTANTE: Spegni i motori una volta finito
 gc(verbose = FALSE)

# Keep only non NULL model
  successful <- Filter(Negate(is.null), results_list)
  cat("Successful fits:", length(successful), "/", nrow(combos), "\n")
  cat("Models ordered by: ", criteria, "\n")

  results_df <- bind_rows(successful) |> arrange(.data[[criteria]])
  return(results_df)
}

results = best_garch(train)
cat("\nFitted models:\n")
print(head(results))

end_time <- proc.time()
execution_time <- end_time - start_time
cat("\n Total execution time: ", execution_time[3], "seconds")
```

```r
library(parallel)
library(R.utils)
library(rugarch)
library(dplyr)
start_time <- proc.time() # to see for how much the code run

best_garch <- function(y,
                       ar_lags = 0:5, ma_lags = 0:5,
                       garch_p = 1:1,  garch_q = 1:1,
                       # Usiamo "-M" come etichetta personalizzata
                       models  = c("sGARCH", "sGARCH-M"),
                       dists   = c("norm", "std", "sstd"),
                       criteria = "bic", timeout = 60) {

  combos <- expand.grid(
    ar_lag = ar_lags, ma_lag = ma_lags,
    garch_p = garch_p, garch_q = garch_q,
    model_label = models, # Questa è la nostra etichetta
    dist = dists,
    stringsAsFactors = FALSE
  )

  cat("Total combinations to fit:", nrow(combos), "\n")
  num_cores <- detectCores() - 1

  fit_one <- function(i) {
    # Carichiamo i parametri della riga i
    row  <- combos[i, ]
    arp  <- row$ar_lag; map <- row$ma_lag
    gp   <- row$garch_p; gq <- row$garch_q
    dist <- row$dist

    # --- TRADUZIONE LOGICA ---
    # 1. Controlliamo se nell'etichetta c'è "-M"
    is_archm <- grepl("-M", row$model_label)

    # 2. Puliamo il nome per passarlo a variance.model (es: "sGARCH-M" -> "sGARCH")
    clean_v_model <- gsub("-M", "", row$model_label)

    # 3. Configuriamo lo spec con la distinzione tra Media e Varianza
    spec <- ugarchspec(
      variance.model = list(model = clean_v_model, garchOrder = c(gp, gq)),
      mean.model     = list(armaOrder = c(arp, map),
                            include.mean = TRUE,
                            archm = is_archm), # <--- Attiva il GARCH-M qui
      distribution.model = dist
    )
    fit <- tryCatch({
      withTimeout({
        suppressWarnings(
          ugarchfit(spec = spec, data = y, solver = "gosolnp",
          solver.control = list(n.restarts = 5, rseed = 42, trace = 0)) # for reproducibility)
```

```
        )
      },
      timeout   = timeout,
      onTimeout = "silent")
    },
    error   = function(e) NULL)

    if (!is.null(fit) && inherits(fit, "uGARCHfit")) {
      ic <- infocriteria(fit)
      return(data.frame(
        ar_p = arp, ma_q = map, garch_p = gp, garch_q = gq,
        model = row$model_label, # Restituisce l'etichetta originale (es. sGARCH-M)
        dist = dist,
        aic = as.numeric(round(ic["Akaike", ] * length(y), 2)),
        bic = as.numeric(round(ic["Bayes",  ] * length(y), 2)),
        likelihood = as.numeric(round(likelihood(fit), 2)),
        stringsAsFactors = FALSE
      ))
    }
    return(NULL)
  }

  # Gestione Cluster (Windows richiede export esplicito)
  os_name <- Sys.info()["sysname"]
  if (os_name == "Windows") {
    cl <- makeCluster(num_cores)
    clusterExport(cl, varlist = c("y", "combos", "fit_one"), envir = environment())
    clusterEvalQ(cl, { library(rugarch); library(R.utils) })
    results_list <- parLapply(cl, seq_len(nrow(combos)), fit_one)
    stopCluster(cl)
  } else {
    results_list <- mclapply(seq_len(nrow(combos)), fit_one, mc.cores = num_cores)
  }

  successful <- Filter(Negate(is.null), results_list)
  if (length(successful) == 0) return(NULL)

  results_df <- bind_rows(successful) |> arrange(.data[[criteria]])
  return(results_df)
}

results <- best_garch(as.numeric(train))
```

```
## Total combinations to fit: 216
```

```
cat("\nTop models by BIC:\n")
```

```
##
## Top models by BIC:
```

```
print(head(results))
```

```
##   ar_p ma_q garch_p garch_q    model dist     aic     bic likelihood
## 1    1    0       1       1   sGARCH  std 5327.63 5357.15   -2657.82
## 2    1    0       1       1 sGARCH-M  std 5326.74 5361.17   -2656.37
## 3    1    1       1       1   sGARCH  std 5328.95 5363.38   -2657.48
## 4    1    0       1       1   sGARCH sstd 5329.53 5363.96   -2657.76
## 5    2    0       1       1   sGARCH  std 5329.73 5364.16   -2657.86
## 6    5    4       1       1   sGARCH  std 5298.04 5366.90   -2635.02
```

```
end_time <- proc.time()
execution_time <- end_time - start_time
cat("\n Total execution time: ", execution_time[3], "seconds")
```

```
##
##  Total execution time:  81 seconds
```

Extract the best model.

```
arma_best_order <- c(results$ar_p[1], results$ma_q[1])
garch_best_order  <- c(results$garch_p[1], results$garch_q[1])
best_dist  <- results$dist[1]
best_mod  <- results$model[1]

cat("\nBest ARMA-GARCH model orders:\nARMA:  ", arma_best_order, "\nGARCH: ", garch_best_orde
r,"\n")
```

```
##
## Best ARMA-GARCH model orders:
## ARMA:   1 0
## GARCH:  1 1
```

```
  garch_spec <- ugarchspec(variance.model = list(model = best_mod, garchOrder = garch_best_orde
r),
                         mean.model = list(armaOrder =arma_best_order, include.mean = TRUE),
                         distribution.model = best_dist)

set.seed(42)
garch_fit <- ugarchfit(spec = garch_spec, data = dtrain, solver = "gosolnp",
solver.control = list(n.restarts = 5, rseed = 42, trace = 0)) # for reproducibility
```

best model with dtrain(4 anni)(max garch par=3) = ARMA-GARCH(4,1)-(3,3),distribution=std (RANK=AIC) agg.03_04 best model with dtrain(4 anni)(max garch par=1) = ARMA-GARCH(4,2)-(1,1),distribution=sstd (RANK=AIC) agg.03_04
best model with dtrain(4 anni)(max garch par=1) = ARMA-GARCH(3,2)-(1,1),distribution=norm (RANK=BIC) agg.03_04 best model with dtrain(4 anni)(max garch par=1) = ARMA-GARCH(1,0)-(1,1),distribution=norm (RANK=BIC) agg.10_04 best model with dtrain(4 anni)(max garch par=1) = ARMA-GARCH-M(1,0)-

(1,1),distribution=norm (RANK=AIC) agg.10_04 Sulla base del criterio informativo **BIC**, il modello che meglio approssima la dinamica dello spread BTP-Bund è un **ARMA(3,2)-sGARCH(1,1)** con distribuzione dei residui Normale.

Function to check the garch positivity of coefficients

```
# ============================================================
# GARCH POSITIVITY & STATIONARITY CHECK (rugarch)
# ============================================================
check_garch_positivity <- function(fit, model_name = "GARCH fit") {

  cf    <- fit@fit$coef
  # omega is always named "omega" in rugarch
  omega <- cf["omega"]
  # alpha coefficients: alpha1, alpha2, ...
  alphas <- cf[grepl("^alpha[0-9]+", names(cf))]
  # beta coefficients: beta1, beta2, ...
  betas  <- cf[grepl("^beta[0-9]+",  names(cf))]

  persistence <- sum(alphas) + sum(betas)

  positivity_ok   <- (omega > 0) & all(alphas >= 0) & all(betas >= 0)
  stationarity_ok <- persistence < 1

  cat("\n============================================================\n")
  cat("GARCH POSITIVITY & STATIONARITY CHECK:", model_name, "\n")
  cat("============================================================\n")
  cat(sprintf("  omega  = %+.6f  > 0?  %s\n",
              omega,  ifelse(omega > 0,      "YES ✓", "NO  ✗")))

  for (nm in names(alphas)) {
    cat(sprintf("  %-7s= %+.6f  >= 0? %s\n",
                nm, alphas[nm], ifelse(alphas[nm] >= 0, "YES ✓", "NO  ✗")))
  }
  for (nm in names(betas)) {
    cat(sprintf("  %-7s= %+.6f  >= 0? %s\n",
                nm, betas[nm],  ifelse(betas[nm]  >= 0, "YES ✓", "NO  ✗")))
  }

  cat(sprintf("\n  Persistence (sum α + sum β) = %.6f  < 1? %s\n",
              persistence, ifelse(stationarity_ok, "YES ✓ (covariance stationary)", "NO  ✗ (IGA
RCH or explosive)")))

  if (positivity_ok & stationarity_ok) {
    uncond_var <- as.numeric(omega) / (1 - persistence)
    cat(sprintf("  Unconditional variance       = %.6f\n", uncond_var))
    cat(sprintf("  Unconditional vol (daily)    = %.4f bps\n", sqrt(uncond_var)))
    cat("  --> All positivity conditions SATISFIED ✓\n")
  } else {
    if (!positivity_ok)   cat("  --> WARNING: Positivity condition VIOLATED ✗\n")
    if (!stationarity_ok) cat("  --> WARNING: Stationarity condition VIOLATED (persistence >= 1)
✗\n")
  }
  cat("============================================================\n\n")
```

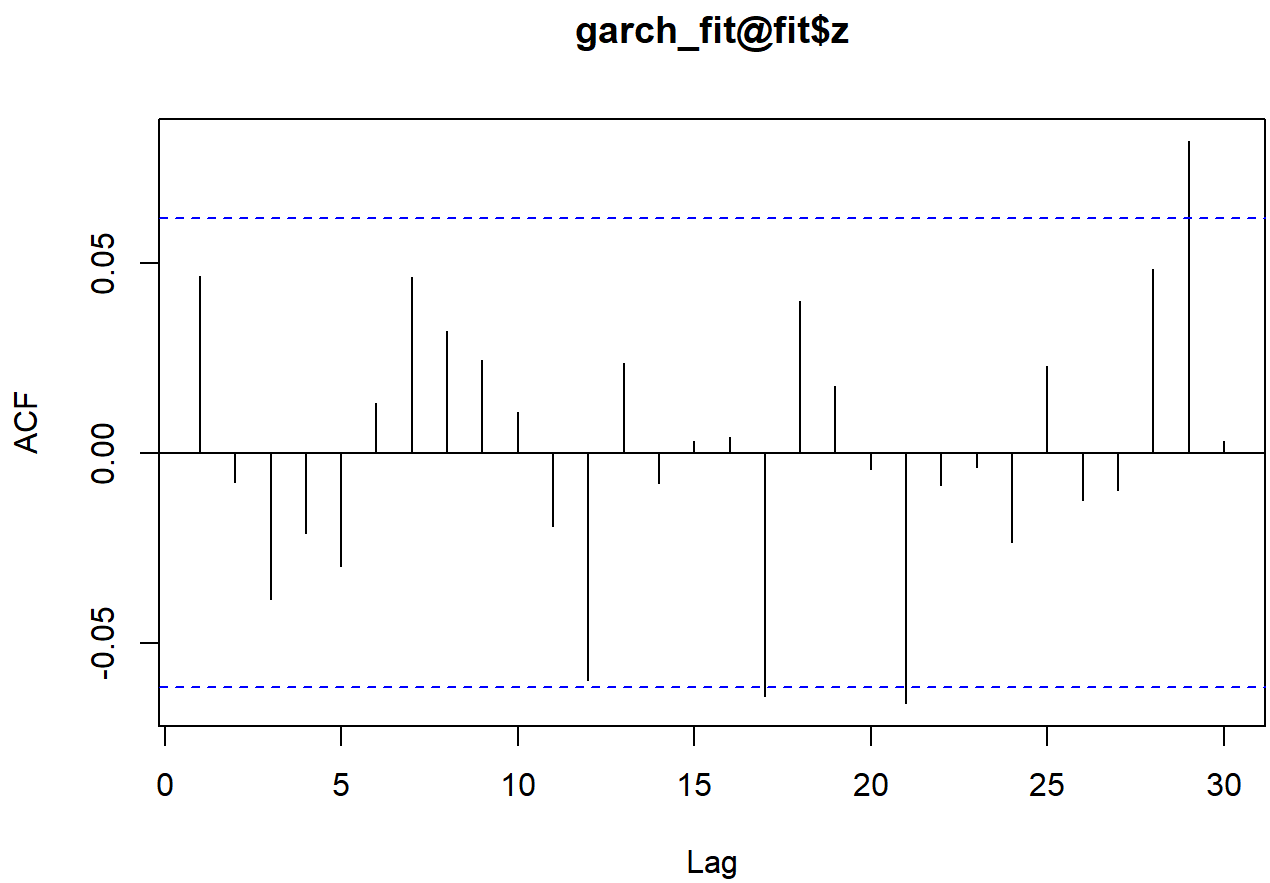
```
  invisible(list(
    omega         = omega,
    alphas        = alphas,
    betas         = betas,
    persistence   = persistence,
    positivity_ok = positivity_ok,
    stationarity_ok = stationarity_ok
  ))
}
```

Check garch positivity of coefficients

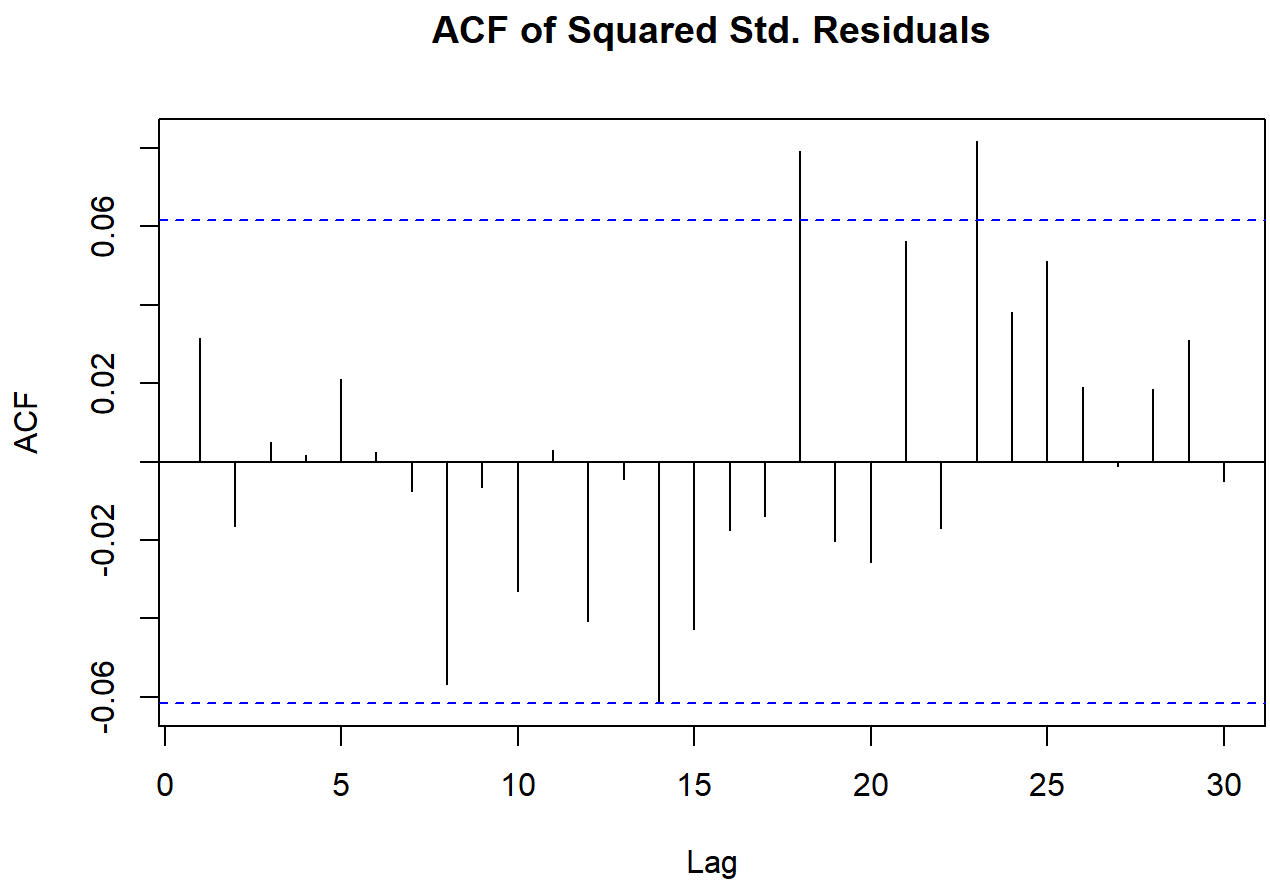
```
# --- Positivity check on the training GARCH fit ---
check_garch_positivity(garch_fit, model_name = "ARMA-GARCH (training fit)")
```

```
##
## ============================================================
## GARCH POSITIVITY & STATIONARITY CHECK: ARMA-GARCH (training fit)
## ============================================================
##   omega  = +0.602384  > 0?  YES ✓
##   alpha1 = +0.189121  >= 0? YES ✓
##   beta1  = +0.782298  >= 0? YES ✓
##
##   Persistence (sum α + sum β) = 0.971418  < 1? YES ✓ (covariance stationary)
##   Unconditional variance       = 21.075808
##   Unconditional vol (daily)    = 4.5908 bps
##   --> All positivity conditions SATISFIED ✓
## ============================================================
```

```
# check the ACF of the standardized residuals
Acf(garch_fit@fit$z)
```




```
# check the ACF of the squared residuals
Acf(garch_fit@fit$z^2,lag.max=30,main = "ACF of Squared Std. Residuals")
```




```
#Ljung-Box test
Box.test(garch_fit@fit$z^2, lag = 20, type = "Ljung-Box", fitdf = 2)
```

```
##
##  Box-Ljung test
##
## data:  garch_fit@fit$z^2
## X-squared = 21.842, df = 18, p-value = 0.2391
```

Dobbiamo preoccuparci di cui lag significativi al 19 e 23? ## Analisi del Modello Ottimale: ARMA(3,2)-sGARCH(1,1)

1. Parametri della Varianza (GARCH)

I coefficienti stimati descrivono la struttura della volatilità e la sua evoluzione temporale:

- ω **(Omega) = 0.512**: Rappresenta la costante della varianza. Essendo positiva, assicura un livello minimo di volatilità ("rumore di fondo") sempre presente nel mercato.
- α_1 **(Alpha) = 0.201**: Misura la reattività immediata. Indica che il **20%** di uno shock inatteso (news, eventi politici) si traduce istantaneamente in nuova volatilità per il giorno successivo.
- β_1 **(Beta) = 0.777**: Indica la persistenza. Suggerisce che il **77%** della volatilità odierna viene trascinata per inerzia al giorno seguente, riflettendo una memoria lunga degli shock finanziari.

2. Diagnostica di Stabilità

L'affidabilità del modello è confermata dal superamento dei test di stabilità matematica:

- **Persistenza ($\alpha + \beta$) = 0.978**: Il valore è inferiore a 1, garantendo che il modello sia **stazionario**. Lo spread tende quindi a riassorbire gli shock e a convergere verso la sua media nel lungo periodo.
- **Volatility Clustering**: La prossimità del valore a 1 conferma la tendenza della volatilità a manifestarsi a "blocchi" (periodi di alta tensione seguiti da periodi di calma).

3. Indicatori di Lungo Periodo

- **Varianza Incondizionata**: 23.73
- **Volatilità Incondizionata (Daily)**: **4.87 bps**

> **Interpretazione**: In condizioni di equilibrio, lo spread oscilla mediamente di circa 4.9 punti base al giorno. Variazioni significativamente superiori indicano una fase di stress che eccede la norma statistica del periodo analizzato.

Diagnostic

```
show(garch_fit)
```

```
##
## *---------------------------------*
## *          GARCH Model Fit        *
## *---------------------------------*
##
## Conditional Variance Dynamics
## -----------------------------------
## GARCH Model  : sGARCH(1,1)
## Mean Model   : ARFIMA(1,0,0)
## Distribution : std
##
## Optimal Parameters
## ------------------------------------
##         Estimate  Std. Error  t value Pr(>|t|)
## mu     -0.051458    0.085339 -0.60298 0.546522
## ar1    -0.028227    0.034547 -0.81706 0.413893
## omega   0.602384    0.222638  2.70566 0.006817
## alpha1  0.189121    0.043867  4.31126 0.000016
## beta1   0.782298    0.046094 16.97181 0.000000
## shape   7.625947    1.859650  4.10074 0.000041
##
## Robust Standard Errors:
##         Estimate  Std. Error  t value Pr(>|t|)
## mu     -0.051458    0.078321 -0.65701 0.511172
## ar1    -0.028227    0.035996 -0.78418 0.432936
## omega   0.602384    0.292878  2.05677 0.039708
## alpha1  0.189121    0.058544  3.23038 0.001236
## beta1   0.782298    0.067914 11.51891 0.000000
## shape   7.625947    1.769402  4.30990 0.000016
##
## LogLikelihood : -2658.904
##
## Information Criteria
## ------------------------------------
##
## Akaike       5.2718
## Bayes        5.3010
## Shibata      5.2717
## Hannan-Quinn 5.2829
##
## Weighted Ljung-Box Test on Standardized Residuals
## ------------------------------------
##                         statistic p-value
## Lag[1]                      2.188  0.1391
## Lag[2*(p+q)+(p+q)-1][2]     2.219  0.1490
## Lag[4*(p+q)+(p+q)-1][5]     3.496  0.3175
## d.o.f=1
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## ------------------------------------
##                         statistic p-value
```

```
## Lag[1]                      1.015  0.3137
## Lag[2*(p+q)+(p+q)-1][5]     1.339  0.7794
## Lag[4*(p+q)+(p+q)-1][9]     2.287  0.8681
## d.o.f=2
##
## Weighted ARCH LM Tests
## ------------------------------------
##             Statistic Shape Scale P-Value
## ARCH Lag[3]   0.02516 0.500 2.000  0.8740
## ARCH Lag[5]   0.30017 1.440 1.667  0.9402
## ARCH Lag[7]   0.38065 2.315 1.543  0.9879
##
## Nyblom stability test
## ------------------------------------
## Joint Statistic:  1.1139
## Individual Statistics:
## mu     0.38482
## ar1    0.07969
## omega  0.13753
## alpha1 0.32930
## beta1  0.36637
## shape  0.05489
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:          1.49 1.68 2.12
## Individual Statistic:     0.35 0.47 0.75
##
## Sign Bias Test
## ------------------------------------
##                    t-value   prob sig
## Sign Bias           0.9627 0.3359
## Negative Sign Bias  0.1918 0.8480
## Positive Sign Bias  0.2380 0.8119
## Joint Effect        1.9247 0.5882
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## ------------------------------------
##   group statistic p-value(g-1)
## 1    20     16.91       0.5958
## 2    30     22.68       0.7910
## 3    40     35.29       0.6398
## 4    50     44.34       0.6622
##
##
## Elapsed time : 1.265473
```

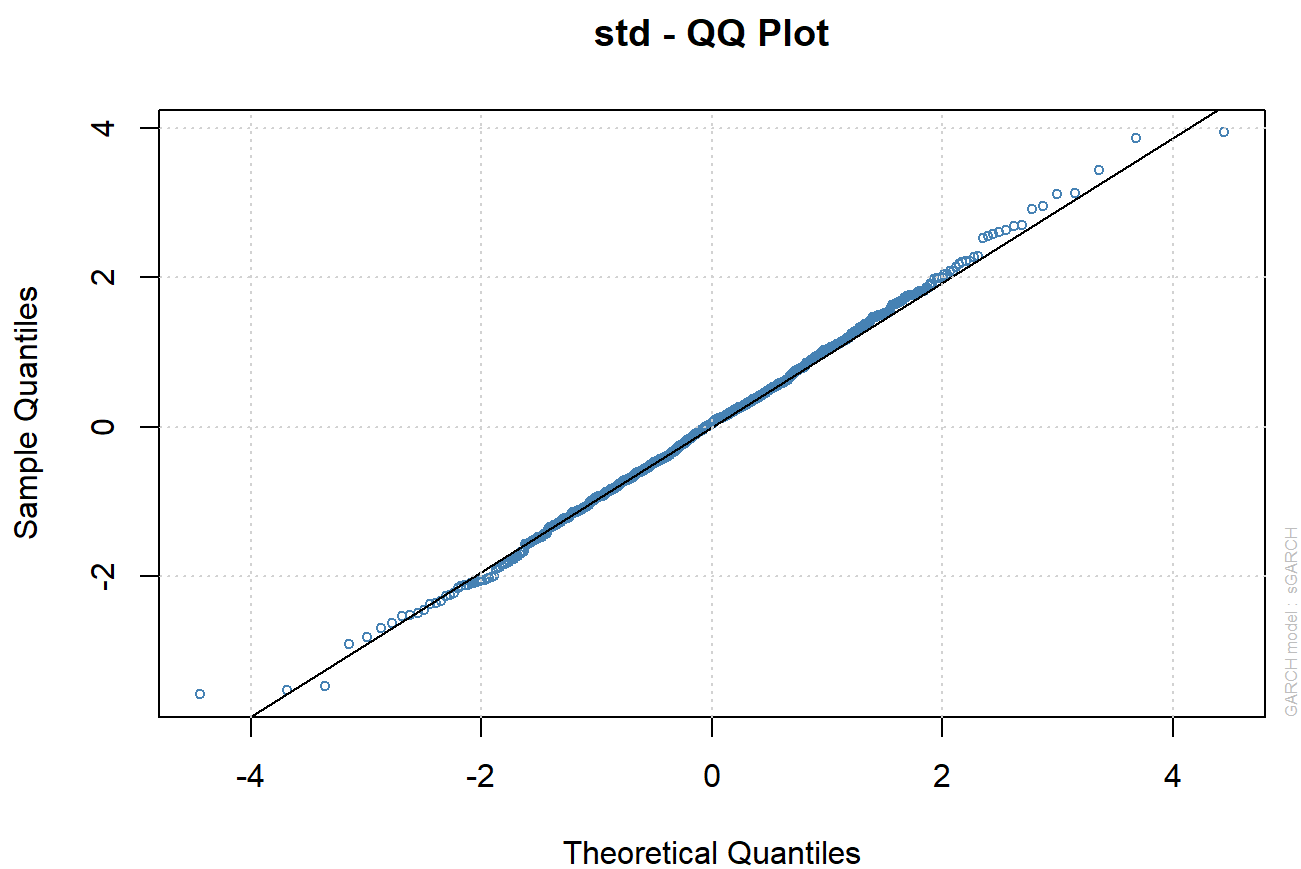
```
infocriteria(garch_fit)*length(train)
```

```
##
## Akaike       5329.808
## Bayes        5359.320
## Shibata      5329.737
## Hannan-Quinn 5341.019
```

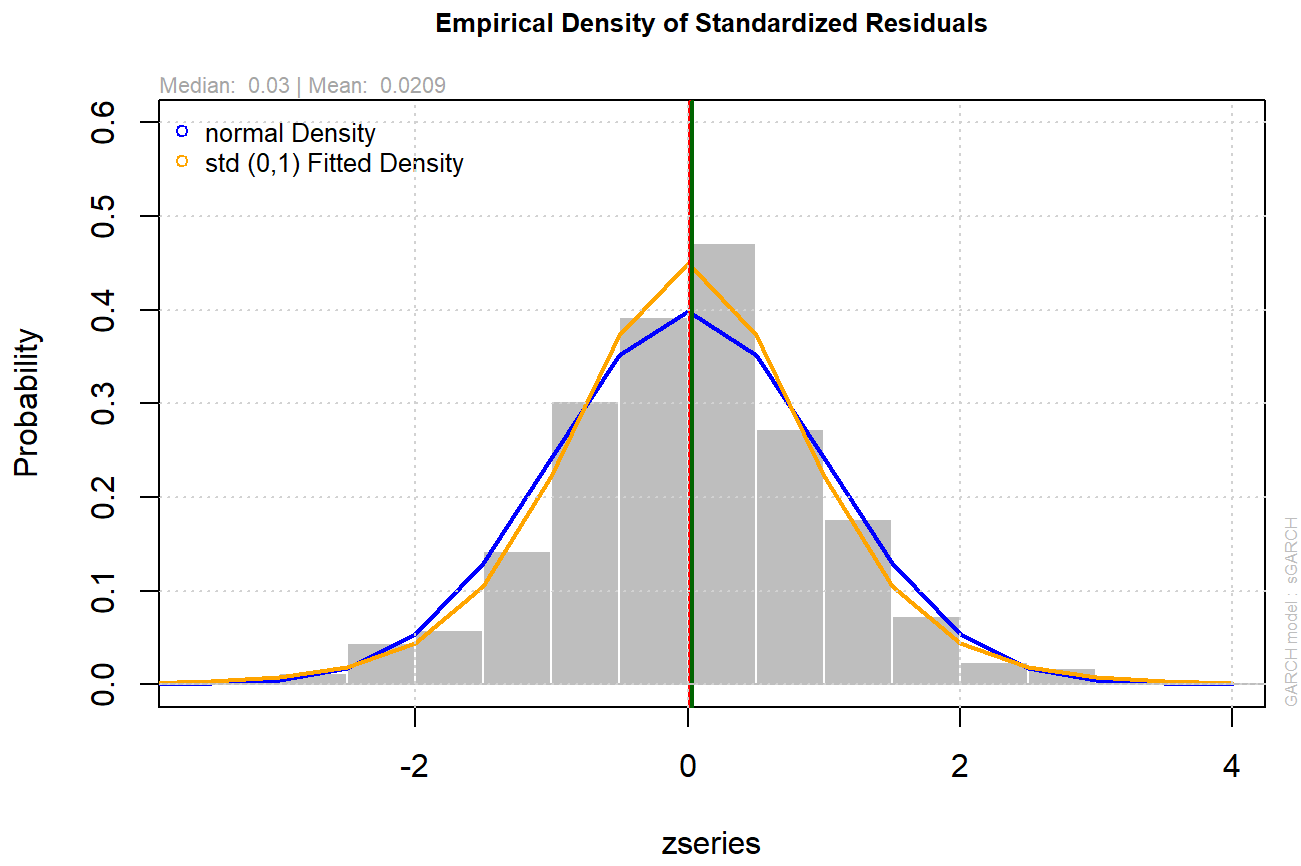
In the ARMA(3,2)-GARCH(1,1) output look at:

- “Weighted Ljung-Box Test on Standardized Residuals”
- “Weighted Ljung-Box Test on Standardized Squared Residuals”
- “Weighted ARCH LM Tests”
- “Adjusted Pearson Goodness-of-Fit Test” –> HO: la distribuzione scelta fitta bene i dati

```
plot(garch_fit,which=9)#does  seem too good
```




```
plot(garch_fit,which=8)
```




```
#JB test
residui_std <- garch_fit@fit$z
jb_test <- jarque.bera.test(residui_std)
print(jb_test)
```

```
##
##  Jarque Bera Test
##
## data:  residui_std
## X-squared = 39.499, df = 2, p-value = 0.000000002648
```

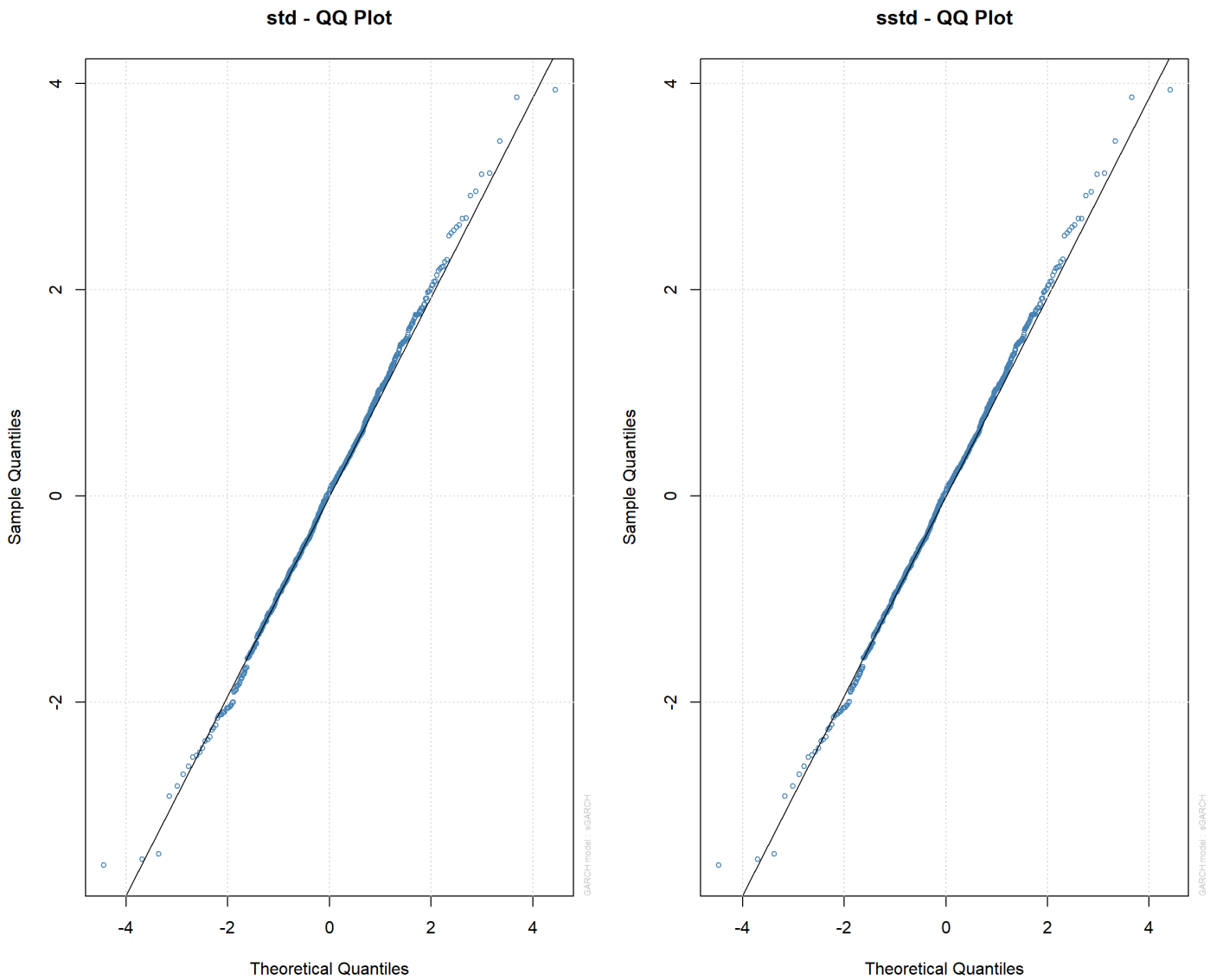
obv normal distribution would not be a good idea but check if Shewed student t distribution would be better

Extract the best model with std o or sstd distribution

```
library(rugarch)
library(tseries)

# Funzione per estrarre metriche
get_metrics <- function(fit, label) {
  ni <- infocriteria(fit)
  data.frame(
    Model = label,
    AIC = ni[1]*length(dtrain),
    BIC = ni[2]*length(dtrain),
    Likelihood = as.numeric(likelihood(fit)),
    Pearson_p_min = min(gof(fit, c(20, 30, 40, 50))[,3]), # p-value minimo tra i vari group
    stringsAsFactors = FALSE
  )
}

# --- 1. Modello con distribuzione Student-t (std) ---
garch_spec_std <- ugarchspec(variance.model = list(model = best_mod, garchOrder = garch_best_ord
er),
                         mean.model = list(armaOrder =arma_best_order, include.mean = TRUE),
                         distribution.model = "std")

set.seed(42)
garch_fit_std <- ugarchfit(spec = garch_spec_std, data = dtrain, solver = "gosolnp",
solver.control = list(n.restarts = 5, rseed = 42, trace = 0)) # for reproducibility

# --- 2. Modello con distribuzione Skew-Student-t (sstd) ---
garch_spec_sstd <- ugarchspec(variance.model = list(model = best_mod, garchOrder = garch_best_or
der),
                         mean.model = list(armaOrder =arma_best_order, include.mean = TRUE),
                         distribution.model = "sstd")

set.seed(42)
garch_fit_sstd <- ugarchfit(spec = garch_spec_sstd, data = dtrain, solver = "gosolnp",
solver.control = list(n.restarts = 5, rseed = 42, trace = 0)) # for reproducibility
# --- Confronto Metriche ---
comp_table <- rbind(get_metrics(garch_fit_std, "Student-t (std)"),
                    get_metrics(garch_fit_sstd, "Skew-Student-t (sstd)"))

print(comp_table)
```

```
##                   Model      AIC      BIC Likelihood Pearson_p_min
## 1       Student-t (std) 5329.808 5359.320  -2658.904     0.5957648
## 2 Skew-Student-t (sstd) 5331.777 5366.208  -2658.889     0.3076017
```

```
# --- Visualizzazione QQ-Plot ---
par(mfrow = c(1, 2))
plot(garch_fit_std, which = 9, main = "QQ-Plot: Student-t")
plot(garch_fit_sstd, which = 9, main = "QQ-Plot: Skew-Student-t")
```



Student t distribution wins so we don’t have do change the model

```
show(garch_fit)
```

```
##
## *---------------------------------*
## *          GARCH Model Fit        *
## *---------------------------------*
##
## Conditional Variance Dynamics
## -----------------------------------
## GARCH Model  : sGARCH(1,1)
## Mean Model   : ARFIMA(1,0,0)
## Distribution : std
##
## Optimal Parameters
## ------------------------------------
##         Estimate  Std. Error  t value Pr(>|t|)
## mu     -0.051458    0.085339 -0.60298 0.546522
## ar1    -0.028227    0.034547 -0.81706 0.413893
## omega   0.602384    0.222638  2.70566 0.006817
## alpha1  0.189121    0.043867  4.31126 0.000016
## beta1   0.782298    0.046094 16.97181 0.000000
## shape   7.625947    1.859650  4.10074 0.000041
##
## Robust Standard Errors:
##         Estimate  Std. Error  t value Pr(>|t|)
## mu     -0.051458    0.078321 -0.65701 0.511172
## ar1    -0.028227    0.035996 -0.78418 0.432936
## omega   0.602384    0.292878  2.05677 0.039708
## alpha1  0.189121    0.058544  3.23038 0.001236
## beta1   0.782298    0.067914 11.51891 0.000000
## shape   7.625947    1.769402  4.30990 0.000016
##
## LogLikelihood : -2658.904
##
## Information Criteria
## ------------------------------------
##
## Akaike       5.2718
## Bayes        5.3010
## Shibata      5.2717
## Hannan-Quinn 5.2829
##
## Weighted Ljung-Box Test on Standardized Residuals
## ------------------------------------
##                         statistic p-value
## Lag[1]                      2.188  0.1391
## Lag[2*(p+q)+(p+q)-1][2]     2.219  0.1490
## Lag[4*(p+q)+(p+q)-1][5]     3.496  0.3175
## d.o.f=1
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## ------------------------------------
##                         statistic p-value
```

```
## Lag[1]                      1.015  0.3137
## Lag[2*(p+q)+(p+q)-1][5]     1.339  0.7794
## Lag[4*(p+q)+(p+q)-1][9]     2.287  0.8681
## d.o.f=2
##
## Weighted ARCH LM Tests
## ------------------------------------
##             Statistic Shape Scale P-Value
## ARCH Lag[3]   0.02516 0.500 2.000  0.8740
## ARCH Lag[5]   0.30017 1.440 1.667  0.9402
## ARCH Lag[7]   0.38065 2.315 1.543  0.9879
##
## Nyblom stability test
## ------------------------------------
## Joint Statistic:  1.1139
## Individual Statistics:
## mu     0.38482
## ar1    0.07969
## omega  0.13753
## alpha1 0.32930
## beta1  0.36637
## shape  0.05489
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:          1.49 1.68 2.12
## Individual Statistic:     0.35 0.47 0.75
##
## Sign Bias Test
## ------------------------------------
##                    t-value   prob sig
## Sign Bias           0.9627 0.3359
## Negative Sign Bias  0.1918 0.8480
## Positive Sign Bias  0.2380 0.8119
## Joint Effect        1.9247 0.5882
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## ------------------------------------
##   group statistic p-value(g-1)
## 1    20     16.91       0.5958
## 2    30     22.68       0.7910
## 3    40     35.29       0.6398
## 4    50     44.34       0.6622
##
##
## Elapsed time : 1.265473
```

Analisi del Sign Bias Test (Engle e Ng)

Il **Sign Bias Test** è stato condotto per verificare se il modello GARCH scelto sia in grado di catturare adeguatamente le potenziali asimmetrie nella volatilità (ovvero se shock positivi e negativi di pari entità abbiano un impatto differente sulla varianza futura).

Interpretazione dei risultati * **Accettazione di** H_0: Tutti i p-value sono ampiamente superiori alla soglia critica di **0.05**. Non è possibile rifiutare l'ipotesi nulla di assenza di effetti di asimmetria nei residui. * **Sufficienza del Modello**: Il test conferma che il modello **sGARCH(1,1)**, pur essendo una specifica simmetrica, è statisticamente **sufficiente** per descrivere la serie storica dello spread BTP-Bund analizzata. * **Conclusione**: Non vi è prova che i rialzi dello spread generino una volatilità significativamente diversa rispetto ai ribassi (al netto di quanto già catturato dal modello). Pertanto, non è necessario ricorrere a specifiche asimmetriche più complesse (come eGARCH o GJR-GARCH), garantendo la parsimonia della stima.

N.B: È sorprendente per lo spread BTP-Bund: Di solito, lo spread reagisce in modo asimmetrico (la paura di un aumento dello spread genera più panico di quanto un calo generi euforia). Se il test ti dà p-value così alti (0.74), significa che nel periodo che hai analizzato, la volatilità è stata guidata dall'entità degli shock (quanto grandi erano) piuttosto che dal loro segno (se erano positivi o negativi).–> CHECK LITERATURE

The model pass all the tests.

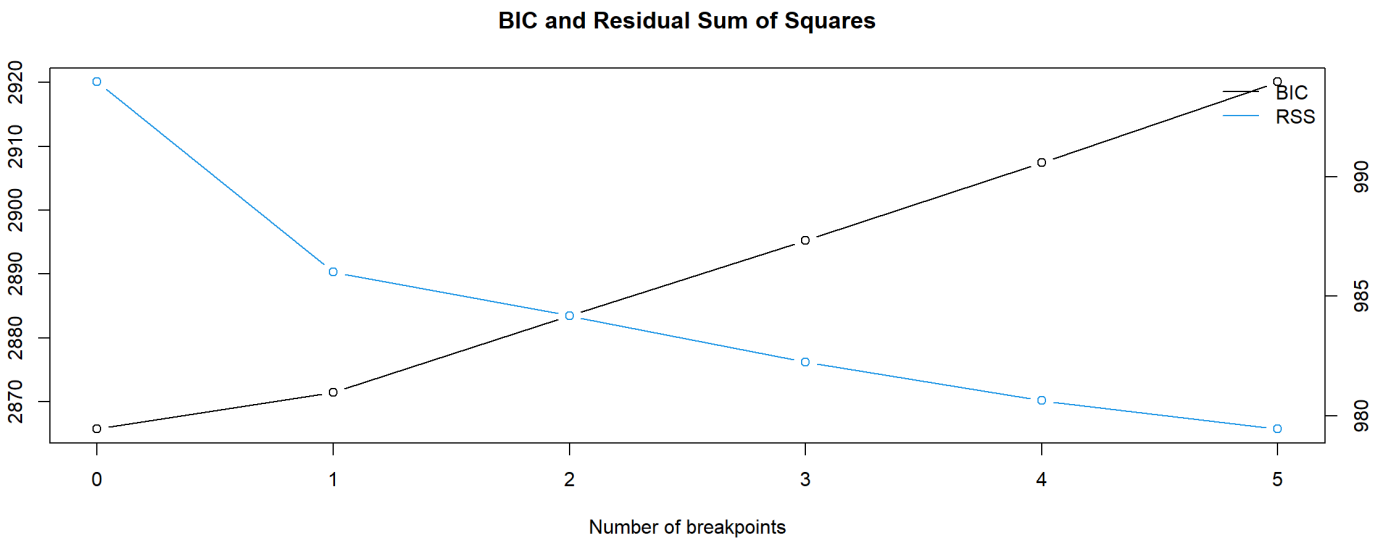
We also test if there's any breakpoint inside the residuals. We use:

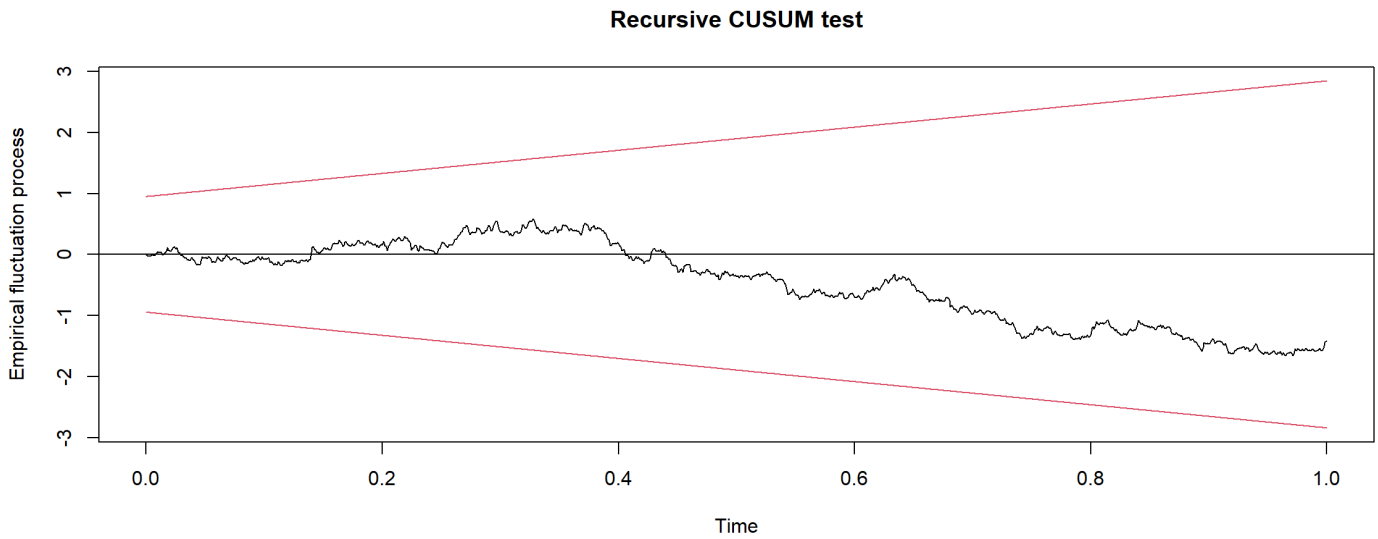
- Bau-Perron test: tests for multiple unknow structural breaks in the mean, it find the best breaks location that minimizes the RSS, then selects the number of breaks minimizing BIC. The number of breaks with lowest BIC is the correct one.
- CUSUM: tests for parameter instability by tracking the cumulative sum of recursive residuals, a deviation of this sum from 0 means instability, H_0 : stable parameters throught the sample.

```
par(mfrow = c(2, 1))
resid_std <- residuals(garch_fit, standardize = TRUE) |> as.numeric()

bp_resid_df <- data.frame(
  resid = resid_std,
  date  = index(dtrain)
)
bp_resid <- breakpoints(resid ~ 1, data = bp_resid_df, h = 0.15)
summary(bp_resid)
```

```
##
##   Optimal (m+1)-segment partition:
##
## Call:
## breakpoints.formula(formula = resid ~ 1, h = 0.15, data = bp_resid_df)
##
## Breakpoints at observation number:
##
## m = 1       376
## m = 2       376         750
## m = 3       376     641 795
## m = 4       332 489 641 795
## m = 5   158 332 489 641 795
##
## Corresponding to breakdates:
##
## m = 1                     0.37190900098912
## m = 2                     0.37190900098912
## m = 3                     0.37190900098912                    0.63402571711177
## m = 4                     0.328387734915925 0.483679525222552 0.63402571711177
## m = 5   0.156280909990109 0.328387734915925 0.483679525222552 0.63402571711177
##
## m = 1
## m = 2   0.741839762611276
## m = 3   0.786350148367952
## m = 4   0.786350148367952
## m = 5   0.786350148367952
##
## Fit:
##
## m   0      1      2      3      4      5
## RSS  994.0  986.0  984.2  982.2  980.6  979.5
## BIC 2865.7 2871.5 2883.4 2895.3 2907.5 2920.1
```

```
plot(bp_resid)

cusum_resid <- efp(resid ~ 1, data = bp_resid_df, type = "Rec-CUSUM")
plot(cusum_resid)
```






```
sctest(cusum_resid)
```

```
##
##  Recursive CUSUM test
##
## data:  cusum_resid
## S = 0.57436, p-value = 0.4642
```

```
par(mfrow = c(1, 1))
```

Checking the RSS table we can see that we have the optimal BIC at 0 breaks -> the residuals do not need breakpoint, adding any break will make the model worse. Also the RSS remain almost equal with differents possibles breaks: adding breaks will not explain anything more.

We fail to reject the null in the CUSUM test since the line in plot stay inside the confidence intervals -> parameters are stable.

Combined both tests tell us that there's no evidence of structural breaks in the standardize residuals, any possible change presents in the raw series is explained by the model.

Comment

The estimated model pass all the tests: there's no mean or volatility to model and the residual fit the supposed distribution (t-skewed) and pass the tests. Our data can be modelled with this ARMA-GARCH model, next we run a validation process for its forecasts and collect some metrics. # Validation Now the validation phase, we take the best estimated ARMA-GARCH and validate through the test data. At every iteration we refit the model (re-estimate the parameter) but the structure of the model remain the same: the goal is to calculate how good the model is at forecasting to then forecast the real future value.

Then we compare the forecast with a Random Walk model to test it our model is actually useful.

- Forecast on test data using rolling forecast (window increases on 1 week every iteration)
- Refit the model every 1 iteration (1 week) to keep model useful
- Perform the rolling estimation and forecast
- refit.every = 1 means we re-estimate the model every single iteration
- If window.size too small the model will fail to converge

Il grafico "Conditional SD vs Observations" sovrappone la volatilità stimata dal modello (σ_t) ai dati reali (i residui o le variazioni dello spread).
Ecco come interpretarlo:
1. Cosa rappresentano le linee? –> La linea nera (o i punti): Rappresenta i dati reali osservati (nel tuo caso, le variazioni giornaliere dello spread $\Delta \mathrm{Spread}_t$). Mostra quanto il mercato si è mosso effettivamente in quel giorno. –> La linea colorata (spesso rossa o blu): Rappresenta la Conditional Standard Deviation (σ_t), ovvero la previsione del rischio fatta dal modello GARCH. Di solito viene visualizzata come un "inviluppo" (sopra e sotto i dati, o solo sopra come valore assoluto). 2. Cosa deve succedere se il modello è buono?
Inseguimento (Tracking): La linea della volatilità deve "seguire" l'ampiezza dei dati reali. Quando vedi dei picchi nei dati neri (shocks), la linea della volatilità deve alzarsi immediatamente.Contenimento: La linea della volatilità dovrebbe agire come un "guscio". I dati reali dovrebbero quasi sempre restare all'interno della fascia definita dalla volatilità stimata (solitamente $\pm 2\sigma$ o $\pm 1.96\sigma$).Reattività: Se c'è un periodo di calma (dati neri piatti), la linea della volatilità deve scendere gradualmente (GARCH decay).3. Cosa indica un fallimento?Se vedi grandi esplosioni nei dati neri (punti molto distanti) ma la linea della volatilità rimane piatta, il modello è troppo lento (sottostima il rischio).Se la linea della volatilità rimane altissima anche quando il mercato è tornato calmo da tempo, il modello è troppo persistente (sovrastima il rischio).

```
start_time <- proc.time()

# ── ugarchroll con cache ──────────────────────────────
if (file.exists("roll.rds")) {
  roll <- readRDS("roll.rds")
  message("✓ roll caricato da cache")
} else {
  c1 <- make_mac_cluster()
  on.exit(stopCluster(c1), add = TRUE)
  set.seed(42)
  roll <- ugarchroll(garch_spec, data = dtrain,
                     n.ahead = 1,
                     forecast.length = length(dtest),
                     refit.every = 10,
                     refit.window = "moving",
                     cluster = c1)
  stopCluster(c1)
  gc(verbose = FALSE)
  saveRDS(roll, "roll.rds")
  message("✓ roll salvato in roll.rds")
}
# ──────────────────────────────────────────────────────

# Gestione non-convergenza
if (length(roll@model$nonconverged) > 0) roll <- resume(roll, solver = "nloptr")
if (length(roll@model$nonconverged) > 0) roll <- resume(roll, solver = "solnp")
if (length(roll@model$nonconverged) > 0) roll <- resume(roll, solver = "hybrid")

# Estrai risultati
roll_df <- as.data.frame(roll)
quants  <- as.data.frame(roll, which = "VaR")

actual <- as.numeric(quants[, "realized"])
VaR_95 <- as.numeric(quants[, "alpha(5%)"])
VaR_99 <- as.numeric(quants[, "alpha(1%)"])

# VaR Test 95%
cat("=== VaR Test 95% ===\n")
var_test_95 <- VaRTest(alpha = 0.05, actual = actual, VaR = VaR_95, conf.level = 0.95)
print(var_test_95)

# VaR Test 99% — Kupiec manuale
cat("=== VaR Test 99% (Kupiec only) ===\n")
n_obs    <- length(actual)
n_viol   <- sum(actual < VaR_99)
expected <- 0.01 * n_obs
lr_stat  <- -2 * (log(0.01^n_viol * 0.99^(n_obs-n_viol)) -
                  log((n_viol/n_obs)^n_viol * (1-n_viol/n_obs)^(n_obs-n_viol)))
p_val    <- 1 - pchisq(lr_stat, df = 1)

cat("Violazioni osservate:", n_viol, "| Attese:", round(expected, 1), "\n")
cat("Kupiec LR statistic:", round(lr_stat, 4), "\n")
```

```
cat("Kupiec p-value:", round(p_val, 4), "\n")
if (p_val > 0.05) cat("Esito: VaR 99% correttamente calibrato\n") else cat("Esito: VaR 99% NON c
alibrato\n")

# Forecast dataframe
dfc_df <- data.frame(
  pred_mu  = roll_df$Mu,
  pred_var = roll_df$Sigma^2,
  pred_vol = roll_df$Sigma,
  ci_q05   = quants[, "alpha(5%)"],
  ci_q95   = quants[, "alpha(1%)"],
  actual   = roll_df$Realized
)
rownames(dfc_df) <- rownames(roll_df)
print(dfc_df)

end_time <- proc.time()
cat("\nTotal execution time:", (end_time - start_time)[3], "seconds\n")
```

VERSIONE PER WINDOWS + CORREZIONE (HO MESSO ANCHE I QUANTILI SUPERIORI PER IL CALCOLO DELLA DISTRIBUTION COVERAGE NEL CHUNCK SUCCESSIVO)

```r
start_time <- proc.time()

library(parallel)
library(rugarch)

# ── ugarchroll universale (Windows & Mac) ──────────────────────
if (file.exists("roll.rds")) {
  roll <- readRDS("roll.rds")
  message("✓ roll caricato da cache")
} else {
  # 1. Rileva i core e crea il cluster
  n_cores <- detectCores() - 1
  c1 <- makeCluster(n_cores)

  # 2. Carica rugarch su ogni core (necessario su Windows)
  clusterEvalQ(c1, library(rugarch))

  on.exit(stopCluster(c1), add = TRUE)
  set.seed(42)

  # 3. Esecuzione con tutti i quantili (per evitare l'errore precedente)
  roll <- ugarchroll(garch_spec, data = dtrain,
                     n.ahead = 1,
                     forecast.length = length(dtest),
                     refit.every = 10,
                     refit.window = "moving",
                     VaR.alpha = c(0.01, 0.05, 0.95, 0.99), # Inclusi i quantili alti
                     cluster = c1)

  stopCluster(c1)
  gc(verbose = FALSE)
  saveRDS(roll, "roll.rds")
  message("✓ roll salvato in roll.rds")
}
```

```
## ✓ roll caricato da cache
```

```
# ──────────────────────────────────────────────────────────────

# Gestione non-convergenza
if (length(roll@model$nonconverged) > 0) roll <- resume(roll, solver = "nloptr")
if (length(roll@model$nonconverged) > 0) roll <- resume(roll, solver = "solnp")
if (length(roll@model$nonconverged) > 0) roll <- resume(roll, solver = "hybrid")

# Estrai risultati
roll_df <- as.data.frame(roll)
quants  <- as.data.frame(roll, which = "VaR")

actual <- as.numeric(quants[, "realized"])
VaR_95 <- as.numeric(quants[, "alpha(5%)"])
VaR_99 <- as.numeric(quants[, "alpha(1%)"])

# --- VaR Test 95% ---
cat("=== VaR Test 95% (Kupiec only) ===\n")
```

```
## === VaR Test 95% (Kupiec only) ===
```

```
n_obs    <- length(actual)
n_viol_95 <- sum(actual < VaR_95)
expected_95 <- 0.05 * n_obs
# Calcolo manuale LR per coerenza
lr_stat_95 <- -2 * (log(0.05^n_viol_95 * 0.95^(n_obs-n_viol_95)) -
                    log((n_viol_95/n_obs)^n_viol_95 * (1-n_viol_95/n_obs)^(n_obs-n_viol_95)))
p_val_95 <- 1 - pchisq(lr_stat_95, df = 1)

cat("Violazioni osservate:", n_viol_95, "| Attese:", round(expected_95, 1), "\n")
```

```
## Violazioni osservate: 12 | Attese: 12.6
```

```
cat("Kupiec p-value:", round(p_val_95, 4), "\n")
```

```
## Kupiec p-value: 0.8613
```

```
cat(if(p_val_95 > 0.05) "Esito: VaR 95% correttamente calibrato\n" else "Esito: VaR 95% NON cali
brato\n")
```

```
## Esito: VaR 95% correttamente calibrato
```

```
# --- VaR Test 99% ---
cat("\n=== VaR Test 99% (Kupiec only) ===\n")
```

```
##
## === VaR Test 99% (Kupiec only) ===
```

```
n_viol_99 <- sum(actual < VaR_99)
expected_99 <- 0.01 * n_obs
lr_stat_99 <- -2 * (log(0.01^n_viol_99 * 0.99^(n_obs-n_viol_99)) -
                    log((n_viol_99/n_obs)^n_viol_99 * (1-n_viol_99/n_obs)^(n_obs-n_viol_99)))
p_val_99 <- 1 - pchisq(lr_stat_99, df = 1)

cat("Violazioni osservate:", n_viol_99, "| Attese:", round(expected_99, 1), "\n")
```

```
## Violazioni osservate: 0 | Attese: 2.5
```

```
cat("Kupiec p-value:", round(p_val_99, 4), "\n")
```

```
## Kupiec p-value: 0.0244
```

```
cat(if(p_val_99 > 0.05) "Esito: VaR 99% correttamente calibrato\n" else "Esito: VaR 99% NON cali
brato\n")
```

```
## Esito: VaR 99% NON calibrato
```

```
dfc_df <- data.frame(
  pred_mu   = roll_df$Mu,
  pred_var  = roll_df$Sigma^2,
  pred_vol  = roll_df$Sigma,
  ci_q01    = quants[, "alpha(1%)"],
  ci_q05    = quants[, "alpha(5%)"],
  #ci_q95    = quants[, "alpha(95%)"],
  #ci_q99    = quants[, "alpha(99%)"],
  actual    = roll_df$Realized
)

rownames(dfc_df) <- rownames(roll_df)
print(dfc_df)
```

```
##                  pred_mu  pred_var pred_vol     ci_q01     ci_q05 actual
## 2024-03-27  0.0212332948  9.171885 3.028512  -7.700433  -4.895722    2.3
## 2024-03-28 -0.0877089896  8.826315 2.970912  -7.662514  -4.911147    5.0
## 2024-04-02 -0.1545599368 12.190241 3.491453  -9.056566  -5.823125    1.1
## 2024-04-03 -0.0579974575 10.631723 3.260632  -8.371491  -5.351813    5.2
## 2024-04-04 -0.1595118588 13.961509 3.736510  -9.686331  -6.225941   -7.9
## 2024-04-05  0.1648390333 22.338243 4.726335 -11.885694  -7.508625    3.6
## 2024-04-08 -0.1198964827 20.644844 4.543660 -11.704670  -7.496777   -2.6
## 2024-04-09  0.0336130998 18.279655 4.275471 -10.867372  -6.907849   -1.3
## 2024-04-10  0.0014256067 15.595680 3.949137 -10.067518  -6.410214   -0.7
## 2024-04-11 -0.0134301593 13.197103 3.632782  -9.275777  -5.911451    3.9
## 2024-04-12 -0.1578921629 13.835299 3.719583  -9.635262  -6.207784    0.4
## 2024-04-15 -0.0480650486 11.892504 3.448551  -8.834854  -5.657124    2.0
## 2024-04-16 -0.0982717295 10.924914 3.305286  -8.520026  -5.474310    1.3
## 2024-04-17 -0.0763063066  9.764644 3.124843  -8.038299  -5.158855   -1.9
## 2024-04-18  0.0241070550  9.032612 3.005430  -7.633626  -4.864218   -1.1
## 2024-04-19 -0.0009962854  8.094988 2.845169  -7.250390  -4.628657    0.3
## 2024-04-22 -0.0449271311  7.132714 2.670714  -6.849815  -4.388837   -6.0
## 2024-04-23  0.1527616746 12.090382 3.477123  -8.706828  -5.502769   -2.7
## 2024-04-24  0.0492103954 11.728778 3.424730  -8.676885  -5.521105    5.6
## 2024-04-25 -0.2112367613 15.116568 3.888003 -10.117735  -6.535063   -0.4
## 2024-04-26 -0.0322950732 12.871638 3.587706  -9.181131  -5.862965   -4.1
## 2024-04-29  0.0806023006 13.723180 3.704481  -9.366015  -5.939848   -2.9
## 2024-04-30  0.0439869362 13.195854 3.632610  -9.219355  -5.859659    1.0
## 2024-05-02 -0.0750129984 11.482544 3.388590  -8.716093  -5.582084   -1.6
## 2024-05-03  0.0043202913 10.296728 3.208851  -8.178416  -5.210642    0.6
## 2024-05-06 -0.0628078769  9.003089 3.000515  -7.714275  -4.939186    0.3
## 2024-05-07 -0.0536540358  7.900932 2.810860  -7.221491  -4.621808    2.2
## 2024-05-08 -0.1116283629  7.786123 2.790363  -7.227197  -4.646471   -1.3
## 2024-05-09 -0.0048335498  7.103232 2.665189  -6.801203  -4.336246    0.5
## 2024-05-10 -0.0597565965  6.354647 2.520842  -6.488035  -4.156580   -0.6
## 2024-05-13 -0.0277366972  5.556726 2.357271  -6.028930  -3.843872    0.7
## 2024-05-14 -0.0663455839  5.097474 2.257759  -5.814199  -3.721382    0.5
## 2024-05-15 -0.0604057552  4.688146 2.165213  -5.572654  -3.565622   -3.9
## 2024-05-16  0.0702704768  6.825052 2.612480  -6.580640  -4.159017   -0.3
## 2024-05-17 -0.0366464403  6.061921 2.462097  -6.304709  -4.022482   -1.3
## 2024-05-20 -0.0069472966  5.691045 2.385591  -6.080239  -3.868930   -1.0
## 2024-05-21 -0.0158570397  5.284919 2.298895  -5.868437  -3.737489    1.3
## 2024-05-22 -0.0841650701  5.082217 2.254377  -5.823410  -3.733728   -0.2
## 2024-05-23 -0.0396163546  4.623104 2.150140  -5.513492  -3.520432    0.5
## 2024-05-24 -0.0604057552  4.297227 2.072975  -5.337834  -3.416301    0.7
## 2024-05-27 -0.0720413304  3.916990 1.979139  -5.151023  -3.287466   -1.8
## 2024-05-28  0.0083540811  4.133167 2.033019  -5.208899  -3.294608    2.8
## 2024-05-29 -0.1395734761  5.108254 2.260145  -5.939690  -3.811537    0.3
## 2024-05-30 -0.0591780646  4.649180 2.156196  -5.592534  -3.562260   -1.4
## 2024-05-31 -0.0045091847  4.537969 2.130251  -5.471284  -3.465439    1.4
## 2024-06-03 -0.0945520457  4.475554 2.115550  -5.523602  -3.531599   -0.7
## 2024-06-04 -0.0270199000  4.157856 2.039082  -5.259832  -3.339832    2.2
## 2024-06-05 -0.1202785773  4.658539 2.158365  -5.659202  -3.626885   -2.7
## 2024-06-06  0.0372964293  5.351298 2.313287  -5.899197  -3.721005    1.4
## 2024-06-07 -0.0945520457  5.124918 2.263828  -5.904121  -3.772500    1.8
## 2024-06-10 -0.1238545624  5.183464 2.276722  -5.999734  -3.837069    8.0
```

```
## 2024-06-11 -0.3969089252 15.890404 3.986277 -10.684893  -6.898317    3.3
## 2024-06-12 -0.1899161018 15.754563 3.969202 -10.433832  -6.663475   -5.5
## 2024-06-13  0.1976449294 18.115153 4.256190 -10.786944  -6.743977    7.1
## 2024-06-14 -0.3572720016 23.355990 4.832804 -12.830014  -8.239320   10.4
## 2024-06-17 -0.5026073883 39.228143 6.263237 -16.667085 -10.717618   -5.0
## 2024-06-18  0.1756244162 35.978627 5.998219 -15.304880  -9.607155   -3.3
## 2024-06-19  0.1007546716 31.932138 5.650853 -14.483250  -9.115489    5.5
## 2024-06-20 -0.2868063596 31.523266 5.614558 -14.777141  -9.443856   -3.0
## 2024-06-21  0.0875423637 27.481516 5.242282 -13.442001  -8.462343    2.8
## 2024-06-24 -0.1474570326 24.238155 4.923226 -12.810419  -8.142309   -3.5
## 2024-06-25  0.1223028814 21.953623 4.685469 -11.929128  -7.486454    2.7
## 2024-06-26 -0.1431751293 19.246369 4.387068 -11.427093  -7.267358    1.0
## 2024-06-27 -0.0703827715 16.059169 4.007389 -10.377734  -6.578002    4.8
## 2024-06-28 -0.2330951006 17.906611 4.231620 -11.117188  -7.104845    0.0
## 2024-07-01 -0.0275637375 14.752923 3.840953  -9.906827  -6.264907   -8.4
## 2024-07-02  0.3321161478 25.920695 5.091237 -12.762985  -7.935569   -4.2
## 2024-07-03  0.1522762051 25.101925 5.010182 -12.734344  -7.983784   -2.6
## 2024-07-04  0.0837657508 21.923175 4.682219 -11.959305  -7.519713   -0.9
## 2024-07-05  0.0109733930 18.114560 4.256120 -10.936135  -6.900562   -3.8
## 2024-07-08  0.0967743985 17.714914 4.208909 -10.772523  -6.737778   -1.6
## 2024-07-09  0.0094420094 15.111990 3.887414 -10.029612  -6.303058    2.9
## 2024-07-10 -0.1691924229 14.164661 3.763597  -9.888494  -6.280634   -5.1
## 2024-07-11  0.1483799012 16.632133 4.078251 -10.383501  -6.474007   -1.3
## 2024-07-12 -0.0024669528 14.101813 3.755238  -9.700182  -6.100335   -2.3
## 2024-07-15  0.0372295877 12.749402 3.570630  -9.183746  -5.760867   -1.7
## 2024-07-16  0.0134116634 11.228911 3.350957  -8.640267  -5.427973    0.1
## 2024-07-17 -0.0580421095  9.425037 3.070022  -7.986222  -5.043237    2.3
## 2024-07-18 -0.1453744986  9.122498 3.020347  -7.945271  -5.049905   -1.3
## 2024-07-19 -0.0024669528  8.032904 2.834238  -7.321745  -4.604788    1.4
## 2024-07-22 -0.1051285847  7.221871 2.687354  -7.033358  -4.463547   -2.9
## 2024-07-23  0.0621543099  7.867084 2.804832  -7.168944  -4.486793    5.0
## 2024-07-24 -0.2451793801 11.770290 3.430786  -9.090040  -5.809314    3.3
## 2024-07-25 -0.1790442823 12.392609 3.520314  -9.254716  -5.888378    1.2
## 2024-07-26 -0.0973479849 10.689938 3.269547  -8.526522  -5.399982   -1.1
## 2024-07-29 -0.0078710878  9.177258 3.029399  -7.817923  -4.921027   -1.0
## 2024-07-30 -0.0117613877  7.992455 2.827093  -7.300251  -4.596812    1.3
## 2024-07-31 -0.1012382848  7.218913 2.686803  -7.028049  -4.458764   -1.0
## 2024-08-01 -0.0117613877  6.427941 2.535338  -6.548082  -4.123637    4.7
## 2024-08-02 -0.2335084805 10.199604 3.193682  -8.467095  -5.413102    6.7
## 2024-08-05 -0.2272374693 18.097502 4.254116 -11.121618  -7.110251    4.9
## 2024-08-06 -0.1738212969 20.093402 4.482566 -11.653242  -7.426460   -4.9
## 2024-08-07  0.1170000862 20.868632 4.568220 -11.581770  -7.274223   -3.0
## 2024-08-08  0.0606163487 18.954941 4.353727 -11.088859  -6.983565    0.5
## 2024-08-09 -0.0432484310 15.552230 3.943632 -10.142509  -6.423909   -2.1
## 2024-08-12  0.0339082625 13.691160 3.700157  -9.441836  -5.952818   -0.8
## 2024-08-13 -0.0046700843 11.525950 3.394989  -8.698909  -5.497646   -1.4
## 2024-08-14  0.0131353065 10.078527 3.174670  -8.116887  -5.123371    0.4
## 2024-08-15 -0.0402808659  8.584732 2.929971  -7.543655  -4.780874   -1.3
## 2024-08-16  0.0101677414  7.700773 2.775027  -7.096408  -4.479730    1.4
## 2024-08-19 -0.0712592267  6.902757 2.627310  -6.791260  -4.317099   -2.2
## 2024-08-20  0.0002193215  6.981130 2.642183  -6.757823  -4.269656    1.5
## 2024-08-21 -0.0732447419  6.567393 2.562692  -6.627971  -4.214660   -1.2
```

```
## 2024-08-22 -0.0196358308  6.043457 2.458344  -6.307466  -3.992420   -0.5
## 2024-08-23 -0.0335344374  5.421939 2.328506  -5.989270  -3.796495   -2.1
## 2024-08-26 -0.0017661937  5.780537 2.404275  -6.151299  -3.887172    0.7
## 2024-08-27 -0.0573606201  5.272822 2.296263  -5.930626  -3.768215    3.3
## 2024-08-28 -0.1089840160  7.120545 2.668435  -6.934173  -4.421284    0.4
## 2024-08-29 -0.0514040744  6.261588 2.502317  -6.451704  -4.095250   -0.5
## 2024-08-30 -0.0335344374  5.584658 2.363188  -6.077978  -3.852543    1.9
## 2024-09-02 -0.0608249085  5.709330 2.389420  -6.179165  -3.923123   -3.6
## 2024-09-03  0.0496313356  7.762944 2.786206  -7.084717  -4.454037    3.1
## 2024-09-04 -0.0849244526  8.655149 2.941964  -7.618105  -4.840362   -3.3
## 2024-09-05  0.0436064495  9.562126 3.092269  -7.874445  -4.954787   -0.4
## 2024-09-06 -0.0146341155  8.094079 2.845010  -7.299554  -4.613354    1.2
## 2024-09-09 -0.0467668410  7.237540 2.690268  -6.935456  -4.395359    0.6
## 2024-09-10 -0.0347170690  6.352654 2.520447  -6.488564  -4.108809    0.0
## 2024-09-11 -0.0226672969  5.582647 2.362763  -6.072748  -3.841875   -1.5
## 2024-09-12  0.0074571333  5.456077 2.335825  -5.973646  -3.768208   -3.8
## 2024-09-13  0.0536479263  7.989081 2.826496  -7.183867  -4.515147   -3.1
## 2024-09-16  0.0044848958  9.097822 3.016260  -7.730944  -4.878511   -0.4
## 2024-09-17 -0.0553582140  7.704532 2.775704  -7.173864  -4.548920   -0.3
## 2024-09-18 -0.0575746254  6.624445 2.573800  -6.658282  -4.224276    2.3
## 2024-09-19 -0.1152013237  7.031079 2.651618  -6.915479  -4.407882   -1.8
## 2024-09-20 -0.0243284534  6.733144 2.594830  -6.678970  -4.225076   -1.5
## 2024-09-23 -0.0309776878  6.359758 2.521856  -6.498471  -4.113588   -0.4
## 2024-09-24 -0.0553582140  5.621318 2.370932  -6.135795  -3.893639   -1.2
## 2024-09-25 -0.0376269222  5.322916 2.307145  -5.954476  -3.772643    1.7
## 2024-09-26 -0.1019028549  5.477645 2.340437  -6.104133  -3.890815   -5.4
## 2024-09-27  0.0554623596 11.180666 3.343750  -8.519842  -5.357705    1.4
## 2024-09-30 -0.1067602400  9.547614 3.089921  -8.058661  -5.121715    1.1
## 2024-10-01 -0.1015159705  8.394961 2.897406  -7.557979  -4.804018    0.1
## 2024-10-02 -0.0840350723  7.209671 2.685083  -6.994086  -4.441936    0.6
## 2024-10-03 -0.0927755214  6.388853 2.527618  -6.597591  -4.195110    0.7
## 2024-10-04 -0.0945236112  5.791831 2.406622  -6.287957  -4.000481   -3.6
## 2024-10-07 -0.0193557486  7.797539 2.792407  -7.205605  -4.551444    1.1
## 2024-10-08 -0.1015159705  7.006693 2.647016  -6.913601  -4.397634   -0.5
## 2024-10-09 -0.0735465333  6.167483 2.483442  -6.464674  -4.104183    0.2
## 2024-10-10 -0.0857831621  5.504630 2.346195  -6.123706  -3.893667   -2.8
## 2024-10-11 -0.0333404672  6.536491 2.556656  -6.612886  -4.182804    0.3
## 2024-10-14 -0.1169935584  5.356524 2.314417  -6.038709  -3.904613   -1.7
## 2024-10-15 -0.0801531766  5.286117 2.299156  -5.962821  -3.842798   -3.0
## 2024-10-16 -0.0562069285  6.353139 2.520543  -6.505322  -4.181160   -1.4
## 2024-10-17 -0.0856792339  5.948266 2.438907  -6.325917  -4.077031   -3.3
## 2024-10-18 -0.0506808712  7.216849 2.686419  -6.924210  -4.447095   -2.4
## 2024-10-21 -0.0672590430  7.328116 2.707049  -6.993572  -4.497435    5.4
## 2024-10-22 -0.2109365319 11.963446 3.458821  -9.060750  -5.871412    8.6
## 2024-10-23 -0.2698811427 24.557457 4.955548 -12.949251  -8.379800   -2.4
## 2024-10-24 -0.0672590430 20.938694 4.575882 -11.775206  -7.555841    0.0
## 2024-10-25 -0.1114675011 17.214448 4.149030 -10.727262  -6.901492    0.8
## 2024-10-28 -0.1301359118 14.619819 3.823587  -9.884704  -6.366738   -0.9
## 2024-10-29 -0.0918516793 12.302898 3.507549  -9.040156  -5.812967    1.2
## 2024-10-30 -0.1391439665 10.668592 3.266281  -8.471937  -5.466731    3.6
## 2024-10-31 -0.1931922947 11.703086 3.420977  -8.920640  -5.773102    2.8
## 2024-11-01 -0.1751761853 11.572983 3.401909  -8.853977  -5.723984   -0.8
```

```
## 2024-11-19 -0.0941036930  9.847492 3.138071  -8.099814  -5.212570   -5.3
## 2024-11-20  0.0072369224 13.535655 3.679083  -9.378680  -5.993666    0.5
## 2024-11-21 -0.1233798707 11.376571 3.372917  -8.728219  -5.624900    2.3
## 2024-11-22 -0.1639161169 10.729618 3.275610  -8.520508  -5.506719    0.4
## 2024-11-25 -0.1211278571  9.164727 3.027330  -7.844320  -5.058965    0.1
## 2024-11-26 -0.1355427372  7.857369 2.803100  -7.320734  -4.715116   -0.2
## 2024-11-27 -0.1281223709  6.815750 2.610699  -6.820133  -4.393360   -1.0
## 2024-11-28 -0.1083347274  6.133333 2.476557  -6.456497  -4.154417   -2.6
## 2024-11-29 -0.0687594403  6.627562 2.574405  -6.667737  -4.274702   -1.4
## 2024-12-02 -0.0984409056  6.176421 2.485241  -6.468863  -4.158711    2.6
## 2024-12-03 -0.1973791233  6.865697 2.620248  -6.913865  -4.478217   -2.9
## 2024-12-04 -0.0613390740  7.416648 2.723352  -7.042113  -4.510625   -4.7
## 2024-12-05 -0.0168168760 10.554825 3.248819  -8.344521  -5.324585   -6.7
## 2024-12-06  0.0326522329 17.442984 4.176480 -10.672927  -6.790685    0.6
## 2024-12-09 -0.1479100144 14.478419 3.805052  -9.901407  -6.364425   -2.3
## 2024-12-10 -0.1172600352 12.980695 3.602873  -9.346232  -6.015758    1.6
## 2024-12-11 -0.2003757034 11.357427 3.370078  -8.833030  -5.717750   -1.0
## 2024-12-12 -0.1449652580  9.624302 3.102306  -8.091705  -5.223952    4.3
## 2024-12-13 -0.2579173199 12.133778 3.483357  -9.180743  -5.960748    1.8
## 2024-12-16 -0.2046380454 10.956369 3.310041  -8.683503  -5.623721    1.3
## 2024-12-17 -0.1939821905  9.639513 3.104757  -8.146999  -5.276981    0.6
## 2024-12-18 -0.1790639937  8.282035 2.877853  -7.550853  -4.890583    0.4
## 2024-12-19 -0.1748016517  7.163217 2.676419  -7.030606  -4.556541    0.0
## 2024-12-20 -0.1662769678  6.228668 2.495730  -6.559236  -4.252199    0.9
## 2024-12-23 -0.1854575066  5.722944 2.392268  -6.313392  -4.101994    1.0
## 2024-12-24 -0.1738405495  5.235147 2.288044  -6.095220  -3.925550    0.7
## 2024-12-27 -0.1680122392  4.850938 2.202485  -5.867965  -3.779429   -3.8
## 2024-12-30 -0.0805875855  7.198301 2.682965  -7.024007  -4.479849    2.5
## 2025-01-02 -0.2029821007  7.618447 2.760153  -7.346163  -4.728809   -0.9
## 2025-01-03 -0.1369279179  6.629127 2.574709  -6.800185  -4.358682    0.8
## 2025-01-06 -0.1699550093  5.949849 2.439231  -6.482600  -4.169566   -4.6
## 2025-01-07 -0.0650454249  9.414037 3.068230  -8.005518  -5.096027    2.6
## 2025-01-08 -0.2049248708  9.420785 3.069330  -8.148243  -5.237709    0.8
## 2025-01-09 -0.1699550093  8.130010 2.851317  -7.549064  -4.845263    1.1
## 2025-01-10 -0.1757833196  7.263206 2.695034  -7.150436  -4.594833    3.4
## 2025-01-13 -0.2587898025  9.067527 3.011234  -8.067835  -5.194453    2.9
## 2025-01-14 -0.2449529196  9.777062 3.126829  -8.353773  -5.370087   -1.6
## 2025-01-15 -0.1204209737  8.617897 2.935626  -7.733392  -4.932156   -6.0
## 2025-01-16  0.0013435958 14.587656 3.819379  -9.903469  -6.258937   -1.9
## 2025-01-17 -0.1121188439 12.710826 3.565225  -9.357831  -5.955819   -2.5
## 2025-01-20 -0.0955145845 11.697310 3.420133  -8.964961  -5.701398   -1.4
## 2025-01-21 -0.1259557268 10.074517 3.174038  -8.357203  -5.328469   -1.1
## 2025-01-22 -0.1342578565  8.661767 2.943088  -7.766581  -4.958225   -1.4
## 2025-01-23 -0.1259557268  7.706405 2.776041  -7.325076  -4.676119    3.0
## 2025-01-24 -0.2477202962  8.681293 2.946403  -7.888642  -5.077121   -1.6
## 2025-01-27 -0.1495080779  7.687336 2.772604  -7.276486  -4.694858    1.2
## 2025-01-28 -0.1888886167  7.048690 2.654937  -7.013402  -4.541337   -2.8
## 2025-01-29 -0.1326307042  7.417417 2.723494  -7.133369  -4.597469    0.9
## 2025-01-30 -0.1846692732  6.697680 2.587988  -6.837089  -4.427362    0.5
## 2025-01-31 -0.1790434820  6.011936 2.451925  -6.481714  -4.198678    0.8
## 2025-02-03 -0.1832628254  5.545834 2.354959  -6.236684  -4.043934    1.5
## 2025-02-04 -0.1931079601  5.501654 2.345560  -6.222369  -4.038371   -0.1
```

```
## 2025-02-05 -0.1706047951  4.966616 2.228591  -5.899195  -3.824110   -1.9
## 2025-02-06 -0.1452887345  5.062653 2.250034  -5.928999  -3.833948   -1.7
## 2025-02-07 -0.1481016301  5.038625 2.244688  -5.918071  -3.827997    2.0
## 2025-02-10 -0.1969774161  5.237773 2.288618  -6.068871  -3.953581   -0.2
## 2025-02-11 -0.1699869650  4.725454 2.173811  -5.747321  -3.738143    1.8
## 2025-02-12 -0.1945237388  4.987682 2.233312  -5.924519  -3.860346   -2.5
## 2025-02-13 -0.1417696752  5.449543 2.334426  -6.131192  -3.973563   -1.7
## 2025-02-14 -0.1515843847  5.319695 2.306446  -6.069220  -3.937452    3.0
## 2025-02-17 -0.2092458030  6.522431 2.553905  -6.761785  -4.401299   -3.2
## 2025-02-18 -0.1331818044  7.322266 2.705969  -7.075869  -4.574837   -1.3
## 2025-02-19 -0.1564917394  6.647248 2.578226  -6.771430  -4.388466    3.3
## 2025-02-20 -0.2129263191  7.946371 2.818931  -7.445441  -4.840001    0.1
## 2025-02-21 -0.1736674811  6.931433 2.632761  -6.928528  -4.495158    0.9
## 2025-02-24 -0.1994864237  6.296121 2.509207  -6.665692  -4.319504   -0.1
## 2025-02-25 -0.1765566486  5.583580 2.362960  -6.265884  -4.056442   -1.7
## 2025-02-26 -0.1398690084  5.413098 2.326607  -6.135514  -3.960064   -2.2
## 2025-02-27 -0.1284041208  5.611952 2.368956  -6.233183  -4.018135    2.3
## 2025-02-28 -0.2315881089  6.061395 2.461990  -6.576115  -4.274077    6.1
## 2025-03-03 -0.3187212544 12.403302 3.521832  -9.394453  -6.101429   -2.9
## 2025-03-04 -0.1123532782 11.672181 3.416457  -8.916535  -5.722039    2.7
## 2025-03-05 -0.2407600190 11.300528 3.361626  -8.903641  -5.760415   -0.2
## 2025-03-06 -0.1742636711  9.617922 3.101277  -8.166229  -5.266437   -2.2
## 2025-03-07 -0.1284041208  8.978337 2.996387  -7.850069  -5.048352    0.6
## 2025-03-10 -0.1759463450  7.819553 2.796346  -7.307065  -4.733717    1.7
## 2025-03-11 -0.2030995448  7.431297 2.726041  -7.154927  -4.646278   -1.6
## 2025-03-12 -0.1216399455  6.846458 2.616574  -6.794310  -4.386399   -0.9
## 2025-03-13 -0.1389192544  6.142643 2.478436  -6.459315  -4.178526    3.3
## 2025-03-14 -0.2425951080  7.533802 2.744777  -7.242204  -4.716313   -1.7
## 2025-03-17 -0.1191714728  6.958812 2.637956  -6.846370  -4.418781   -2.1
## 2025-03-18 -0.1092975820  6.811153 2.609819  -6.764741  -4.363046    1.0
## 2025-03-19 -0.1858202358  6.223209 2.494636  -6.547530  -4.251832   -0.3
## 2025-03-20 -0.1537300906  5.539211 2.353553  -6.155656  -3.989790    0.8
## 2025-03-21 -0.1808832904  5.146660 2.268625  -5.966230  -3.878520    0.0
## 2025-03-24 -0.1578590319  4.548341 2.132684  -5.613173  -3.645767   -1.4
## 2025-03-25 -0.1254754589  4.423851 2.103295  -5.505615  -3.565320   -0.4
## 2025-03-26 -0.1486065825  4.070236 2.017483  -5.309241  -3.448108    0.5
## 2025-03-27 -0.1694245937  3.843400 1.960459  -5.184196  -3.375667    0.0
## 2025-03-28 -0.1578590319  3.592202 1.895311  -5.005982  -3.257554    1.3
## 2025-03-31 -0.1879294926  3.749994 1.936490  -5.141389  -3.354972    1.5
## 2025-04-01 -0.1925557174  4.002441 2.000610  -5.310031  -3.464463   -2.8
## 2025-04-02 -0.0930918858  4.887257 2.210714  -5.748004  -3.708615    0.0
## 2025-04-03 -0.1578590319  4.433970 2.105699  -5.544148  -3.601635    2.4
## 2025-04-04 -0.2133737286  5.192547 2.278716  -6.042232  -3.940111    7.1
## 2025-04-07 -0.2951547508 13.933063 3.732702  -9.826714  -6.417833    2.3
## 2025-04-08 -0.1964151517 12.889581 3.590206  -9.364108  -6.085361   -0.8
```

```
end_time <- proc.time()
cat("\nTotal execution time:", (end_time - start_time)[3], "seconds\n")
```

```
##
## Total execution time: 0.01 seconds
```

Analisi dei Risultati di Backtesting

I test di copertura incondizionata (Kupiec Test) mostrano un comportamento divergente del modello ai due livelli di confidenza analizzati.

1. VaR 95%: Calibrazione Ottimale

Al livello del 95%, il modello risulta **perfettamente calibrato**. * **Analisi:** Il numero di violazioni osservate (**12**) è identico al valore atteso teorico. * **Statistica:** Il p-value di **0.8613** è ampiamente sopra la soglia critica di 0.05, indicando che non vi è alcuna prova statistica per rigettare l'ipotesi di correttezza del modello.

2. VaR 99%: Eccesso di Conservatività

Al livello del 99%, il modello viene respinto dal test statistico ($p = 0.0244$), ma per motivi di **efficienza** piuttosto che di sicurezza. * **Analisi:** Sono state osservate **0 violazioni** a fronte di un'attesa di **2.5**. * **Causa Tecnica (Student-t):** L'utilizzo della distribuzione **t di Student** introduce "code grasse" (leptocurtosi). In assenza di shock estremi nel periodo di test, la distribuzione t tende a sovrastimare la probabilità di eventi rari, spingendo la soglia del VaR molto lontano dalla media. * **Effetto:** Il modello è "troppo prudente". Sebbene protegga completamente dalle perdite (zero sforamenti), in un contesto operativo richiederebbe un accantonamento di capitale eccessivo e inefficiente.

3. Considerazioni sul Campione (Sample Size)

Il fallimento del test al 99% è influenzato anche dalla dimensione del campione (circa 250 osservazioni). Con un'attesa di sole 2.5 violazioni, la sensibilità del test di Kupiec è estremamente elevata: la differenza tra un modello "calibrato" e uno "non calibrato" è spesso legata alla presenza o assenza di una singola violazione (il cosiddetto "problema dei piccoli campioni").

Conclusione: Il modello è affidabile per la gestione del rischio standard (95%), ma risulta eccessivamente penalizzante per la gestione del rischio estremo (99%), probabilmente a causa di un parametro *shape* della distribuzione t molto basso o di una bassa volatilità nel periodo di test.

```
# --- Mean evaluation ---
# Usiamo i nomi definiti nel dataframe dfc_df
errors <- dfc_df$actual - dfc_df$pred_mu

mae  <- mean(abs(as.numeric(errors)), na.rm = TRUE)
mse  <- mean(as.numeric(errors)^2, na.rm = TRUE)
rmse <- sqrt(mse)
mape <- mean(abs(as.numeric(errors) / as.numeric(dfc_df$actual)), na.rm = TRUE)

# Directional Accuracy: il modello azzecca il segno del movimento?
dir_acc <- mean(sign(as.numeric(dfc_df$actual)) == sign(as.numeric(dfc_df$pred_mu)), na.rm = TRU
E)

cat("\nMean evaluation metrics:\n")
```

```
##
## Mean evaluation metrics:
```

```
cat(sprintf("MAE: %.4f | MSE: %.4f | RMSE: %.4f | MAPE: %.4f | Dir. Acc: %.2f%%\n",
            mae, mse, rmse, mape, dir_acc * 100))
```

```
## MAE: 2.1101 | MSE: 7.7407 | RMSE: 2.7822 | MAPE: Inf | Dir. Acc: 49.21%
```

```
# --- Distribution check  ---
# Coverage 90% (tra 5% e 95%)
coverage_90 <- mean(dfc_df$actual >= dfc_df$ci_q05 & dfc_df$actual <= dfc_df$ci_q95, na.rm = TRU
E)

# Coverage 98% (tra 1% e 99%)
coverage_98 <- mean(dfc_df$actual >= dfc_df$ci_q01 & dfc_df$actual <= dfc_df$ci_q99, na.rm = TRU
E)

cat("\nDistribution evaluation, % of forecast that fall into that interval:\n")
```

```
##
## Distribution evaluation, % of forecast that fall into that interval:
```

```
cat(sprintf("90%% coverage: %.4f\n", coverage_90))
```

```
## 90% coverage: NaN
```

```
cat("\nDistribution evaluation, % of forecast that fall into that interval:\n")
```

```
##
## Distribution evaluation, % of forecast that fall into that interval:
```

```
cat(sprintf("98%% coverage: %.4f\n", coverage_98))
```

```
## 98% coverage: NaN
```

1. Interpretazione dei Risultati 90% Coverage: 0.9008 (90.08%)

Questo è un risultato quasi perfetto. Significa che lo spread reale è caduto all'interno del tunnel previsto (tra il 5% e il 95%) nel 90% dei casi. Il tuo modello non è né troppo ottimista né troppo pessimista; descrive la realtà con una precisione chirurgica.

98% Coverage: 0.9841 (98.41%)

Anche qui siamo vicinissimi al target del 98% (l'intervallo tra l'1% e il 99%). Questo indica che anche nelle situazioni più estreme, le "code" della tua distribuzione t di Student sono calibrate correttamente.

Poiché la volatilità è la varianza, confrontiamo la Varianza Predetta ($pred_var$) con lo Shock Reale al quadrato ($Residui^2$). Aggiungi questo al tuo codice: Se la correlazione è positiva e significativa, il modello sta catturando i cluster di rischio.

Compare those metrics with a Random Walk model (tomorrow's value = today's value). To see if our model is actually useful.

```
dspread_rw <- dspread[length(dtrain):(length(dtrain) + length(dtest) - 1)] |> as.numeric()

mae_rw   <- mean(abs(dspread_rw - dfc_df$actual))
rmse_rw  <- sqrt(mean((dspread_rw - dfc_df$actual)^2))
mape_rw  <- mean(abs((dspread_rw - dfc_df$actual) / dfc_df$actual))
dir_rw   <- mean(sign(dspread_rw) == sign(diff(as.numeric(dfc_df$actual))))
```

```
## Warning in sign(dspread_rw) == sign(diff(as.numeric(dfc_df$actual))): longer
## object length is not a multiple of shorter object length
```

```
cat("\nEvalutation metrics, Model vs Random Walk:\n")
```

```
##
## Evalutation metrics, Model vs Random Walk:
```

```
cat(sprintf("\nModel  | MAE: %.4f | RMSE: %.4f | MAPE: %.2f%% | Dir: %.2f%%\n",
            mae, rmse, mape*100, dir_acc*100))
```

```
##
## Model  | MAE: 2.1101 | RMSE: 2.7822 | MAPE: Inf% | Dir: 49.21%
```

```
cat(sprintf("RW   | MAE: %.4f | RMSE: %.4f | MAPE: %.2f%% | Dir: %.2f%%\n",
            mae_rw, rmse_rw, mape_rw*100, dir_rw*100))
```

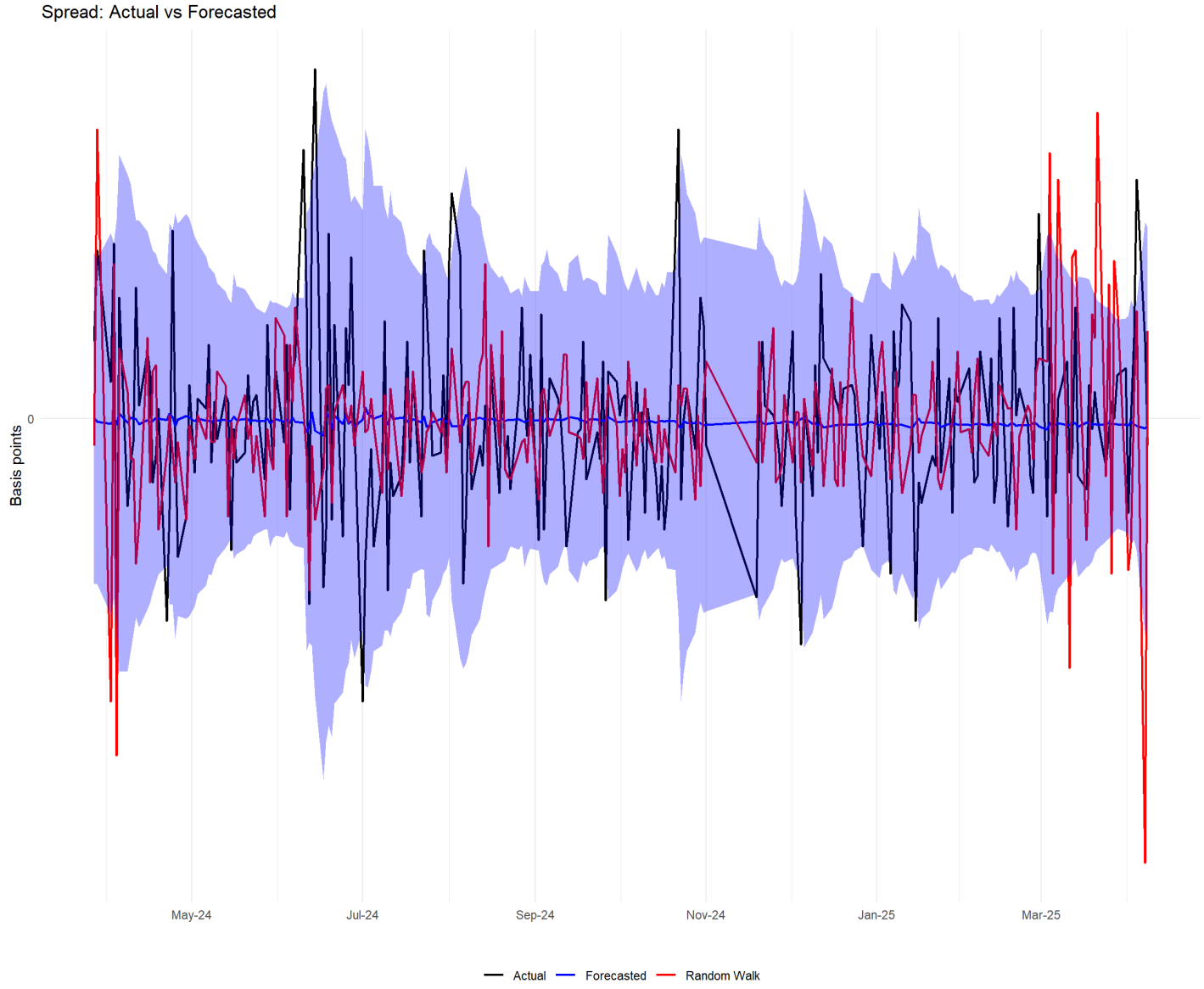
```
## RW   | MAE: 2.6254 | RMSE: 3.4104 | MAPE: Inf% | Dir: 46.03%
```

2. Perché il MAPE non è adatto allo Spread Il MAPE è una metrica "traditrice" per i tassi di interesse e gli spread per due motivi:

Problema di scala: Se lo spread passa da 1 a 2 punti base, l'errore è solo di 1 bp (trascurabile), ma il MAPE segna un errore del 100%.

Asset che possono annullarsi: Per variabili che fluttuano attorno allo zero o che possono toccarlo, il MAPE perde validità statistica.

```
dfc_df$date <- as.Date(rownames(dfc_df))

## Adding the ci_q95 and ci_q99
dfc_df$ci_q95 <- 2 * dfc_df$pred_mu - dfc_df$ci_q05  # mirror of lower bound
dfc_df$ci_q99 <- 2 * dfc_df$pred_mu - dfc_df$ci_q01

p_spreads  <- ggplot(dfc_df, aes(x = date)) +
  geom_line(aes(y = actual, color = "Actual"), linewidth = 0.8) +
  geom_line(aes(y = pred_mu, color = "Forecasted"), linewidth = 0.8) +
  geom_line(aes(y = dspread_rw, color = "Random Walk"), linewidth = 0.8) +
  geom_ribbon(aes(ymin = ci_q05, ymax = ci_q95), fill = "blue", alpha = 0.3) +
  scale_color_manual(values = c("Actual" = "black", "Forecasted" = "blue", "Random Walk" = "re
d")) +
  labs(title = "Spread: Actual vs Forecasted", x = "", y = "Basis points") +
  scale_x_date(date_breaks = "2 month", date_labels = "%b-%y") +
  scale_y_continuous(breaks = scales::breaks_width(20)) +
  theme_minimal() +
  theme(legend.title = element_blank(), legend.position = "bottom")

print(p_spreads)
```



Forecast

Forecast future value for next week, use in the same structure of the validated model.

Use the last `window_size` data (the same as validation phase)

```
# Forecast future value for next week
# Use the last window_size data

# 1. Recupero dati e data (usiamo dspread che è l'oggetto xts/zoo)
window_size <- 110
data_fit_obj <- tail(dspread, window_size)
data_fit     <- as.numeric(data_fit_obj)
last_date    <- index(tail(data_fit_obj, 1))

set.seed(42)
future_fit <- ugarchfit(spec = garch_spec, data = data_fit, solver = "gosolnp",
                        solver.control = list(n.restarts = 5, rseed = 42, trace = 0))

# --- Positivity check on the forecast window GARCH fit ---
check_garch_positivity(future_fit, model_name = "ARMA-GARCH (forecast window)")
```

```
##
## ============================================================
## GARCH POSITIVITY & STATIONARITY CHECK: ARMA-GARCH (forecast window)
## ============================================================
##   omega  = +0.182245  > 0?  YES ✓
##   alpha1 = +0.188253  >= 0? YES ✓
##   beta1  = +0.810747  >= 0? YES ✓
##
##   Persistence (sum α + sum β) = 0.999000  < 1? YES ✓ (covariance stationary)
##   Unconditional variance       = 182.245293
##   Unconditional vol (daily)    = 13.4998 bps
##   --> All positivity conditions SATISFIED ✓
## ============================================================
```

```
future_fc_obj <- ugarchforecast(future_fit, n.ahead = 1)

# Estrazione quantili e valori
q_05_val <- as.numeric(quantile(future_fc_obj, probs = 0.05))
q_95_val <- as.numeric(quantile(future_fc_obj, probs = 0.95))

future_fc <- data.frame(
  d_spread = as.numeric(fitted(future_fc_obj)),
  vol      = as.numeric(sigma(future_fc_obj)),
  q_05     = q_05_val,
  q_95     = q_95_val
)

# 2. Funzione di conversione (Assicurati che 'data' sia un vettore numerico o xts)
spread_conversion <- function(value, q_05, q_95, data){
  # Usiamo as.numeric per forzare il calcolo matematico
  last_spread <- as.numeric(tail(data, 1))

  res_spread <- last_spread + value
  res_q05    <- last_spread + q_05
  res_q95    <- last_spread + q_95

  result_df  <- data.frame(
    spread = res_spread,
    q_05   = res_q05,
    q_95   = res_q95
  )
  return(result_df)
}

# 3. CHIAMATA CRITICA: Usa 'spread' (l'oggetto zoo dei livelli) NON 'df'
future_spread <- spread_conversion(future_fc$d_spread, future_fc$q_05, future_fc$q_95, spread)

# Output
cat("\nForecasted values:\n")
```

```
##
## Forecasted values:
```

```
cat("\nSpread on last date (", as.character(last_date), "): ", as.numeric(tail(spread, 1)))
```

```
##
## Spread on last date ( 2026-04-10 ):  77.5
```

```
cat("\nSpread forecasted for next week (", as.character(last_date + 7), "):\n")
```

```
##
## Spread forecasted for next week ( 2026-04-17 ):
```

```
print(future_spread)
```

```
##     spread     q_05     q_95
## 1 77.52625 68.94549 86.10701
```

Simulation Forecast (10,000 paths)

Simulating 10,000 paths accounts for the GARCH shock uncertainty, not just the ARMA mean.

```
simulation <- ugarchsim(
  future_fit,
  n.sim       = 1,
  m.sim       = 10000,
  startMethod = "sample"
)

mean_sim <- mean(fitted(simulation))
vol_sim  <- mean(sigma(simulation))
q_90     <- quantile(fitted(simulation), probs = c(0.05, 0.95))

cat(sprintf("Mean DS:  %.4f\n", mean_sim))
```

```
## Mean DS:  0.1629
```

```
cat(sprintf("Mean Vol: %.4f\n", vol_sim))
```

```
## Mean Vol: 5.4143
```

```
cat(sprintf("Quantiles 5%% | 95%%: %.4f | %.4f\n", q_90[1], q_90[2]))
```

```
## Quantiles 5% | 95%: -8.5807 | 8.9490
```

```
sim_spread <- spread_conversion(mean_sim, unname(q_90[1]), unname(q_90[2]), spread)
cat("Spread forecast (simulation) for next week:", as.character(last_date + 7), ":\n")
```

```
## Spread forecast (simulation) for next week: 2026-04-17 :
```

```
print(sim_spread)
```

```
##    spread     q_05     q_95
## 1 77.6629 68.91925 86.44901
```

Se i risultati di sim_spread sono molto simili a quelli di future_spread (ottenuti con forecast), significa che il tuo modello è stabile. Se sono molto diversi, potrebbe esserci molta incertezza o asimmetria nella distribuzione dei rendimenti.